

Programación Web

Programación IV



Ing. Alejandro Di Battista

JavaScript: qué es, de dónde viene y dónde corre

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte de la primera clase de JavaScript

Venís de Programación III, donde escribías C# sobre .NET. Ahí el lenguaje y la plataforma se diseñaron juntos, con calma, por un equipo que sabía adónde iba. JavaScript es lo contrario: nació en diez días, para resolver un problema chico, y terminó siendo el lenguaje que corre en cada teléfono, cada navegador y buena parte de los servidores del mundo.

Entender ese origen no es anécdota. Casi todo lo que te va a parecer raro de JavaScript se explica por su historia. Empecemos por ahí.

De dónde viene JavaScript

El problema de 1995

En 1995 la web era un montón de documentos. Escribías HTML, el servidor lo mandaba, el navegador lo dibujaba y ahí terminaba todo. Si querías validar que un campo de un formulario no estuviera vacío, tenías que mandar el formulario al servidor, esperar la respuesta y recibir una página nueva que decía "falta el nombre". Cada interacción costaba una ida y vuelta por la red, con módems de 28.800 bits por segundo.

Netscape, la empresa dueña del navegador más usado, quería resolver eso. Necesitaba un lenguaje que corriera dentro del navegador, en la máquina del usuario, para poder reaccionar a lo que la persona hacía sin ir al servidor.

Diez días de mayo

Netscape contrató a Brendan Eich para escribirlo. La idea original era meter Scheme, un dialecto de Lisp, dentro del navegador. En el camino la empresa cerró un acuerdo con Sun Microsystems, dueña de Java, y la orden cambió: el lenguaje nuevo tenía que parecerse a Java.

Eich escribió el prototipo en diez días, en mayo de 1995. De ahí salió una mezcla poco común:

- la sintaxis de C y Java, con llaves y punto y coma
- las funciones como valores de primera clase, que vienen de Scheme
- los objetos basados en prototipos, que vienen de Self

Esa combinación explica muchas cosas. JavaScript se escribe como Java, pero por dentro piensa como un lenguaje funcional. Los alumnos que esperan encontrar C# con otro nombre chocan justo ahí.

El nombre

El lenguaje se llamó Mocha, después LiveScript y en diciembre de 1995 pasó a llamarse JavaScript. El cambio fue una decisión de marketing: Java era la tecnología del momento y Netscape quería la asociación.

Java y JavaScript no comparten nada más que las cuatro primeras letras. La confusión sigue viva 30 años después.

La guerra de navegadores

Microsoft no se quedó mirando. En 1996 hizo ingeniería inversa del lenguaje y sacó JScript en Internet Explorer 3. Era casi igual, pero no del todo, así que el mismo código funcionaba distinto en cada navegador.

Ese fue el problema central de la web durante 15 años. Escribías una página, andaba en Netscape y se rompía en Explorer. La solución de la época era llenar el código de condicionales según el navegador.

Por qué el lenguaje tiene dos nombres

Sun era dueña de la marca Java, y por lo tanto de JavaScript. Para estandarizar el lenguaje sin pelearse por la marca, Netscape lo llevó en 1996 a ECMA International, un organismo de estandarización. En junio de 1997 salió la primera edición del estándar ECMA-262, con el nombre ECMAScript.

Por eso hoy convive un nombre comercial con un nombre técnico:

- JavaScript es el lenguaje tal como lo conocés y lo nombra todo el mundo
- ECMAScript es el estándar que define qué hace el lenguaje

Cuando alguien dice ES6, ES2015 o ES2022 está hablando de versiones del estándar. Es el mismo lenguaje.

Los años oscuros

La tercera edición, ES3, salió en 1999 y trajo expresiones regulares y manejo de excepciones. Después el lenguaje se quedó quieto diez años.

La cuarta edición fue un intento ambicioso: clases, módulos, tipado estático opcional. El comité no se puso de acuerdo y en 2008 la abandonó. Vale la pena retener esa idea, porque el tipado estático opcional vuelve más adelante en esta historia con otro nombre.

Mientras tanto la web crecía igual. En 1999 Microsoft agregó a Explorer un objeto llamado XMLHttpRequest para que Outlook Web Access pudiera pedir datos al servidor sin recargar la página. Nadie le dio importancia hasta que Google lo usó en Gmail y en Google Maps. En 2005 alguien le puso nombre a esa técnica, AJAX, y quedó claro que el navegador podía ser algo más que un visor de documentos.

En 2006 apareció jQuery. No era un lenguaje nuevo: era una biblioteca que tapaba las diferencias entre navegadores y hacía tolerable manipular la página. Durante casi una década, aprender JavaScript era en la práctica aprender jQuery.

2008: el motor cambia todo

Google lanzó Chrome en septiembre de 2008 con un motor de JavaScript propio llamado V8. V8 no interpretaba el código línea por línea: lo compilaba a código de máquina mientras corría.

El salto de velocidad fue de un orden de magnitud. De golpe se volvió razonable escribir aplicaciones enteras en JavaScript, no ya adornos sobre una página. La competencia respondió y empezó una carrera de rendimiento entre motores que todavía sigue.

En 2009 salió ES5, con modo estricto, soporte de JSON y métodos de arreglos como `map` y `filter`.

2009: JavaScript sale del navegador

Ryan Dahl tomó V8, que ya era un motor rápido y estaba disponible como pieza suelta, y lo sacó del navegador. Le agregó una biblioteca para hablar con el sistema operativo y lo llamó Node.js.

Node convirtió a JavaScript en un lenguaje de propósito general. Podías leer archivos, abrir sockets, escribir un servidor HTTP. En 2010 apareció npm, su gestor de paquetes, y con él el ecosistema de bibliotecas más grande que existe hoy.

Ese es el momento en que aparece la idea de full stack: un solo lenguaje del navegador al servidor. Es la premisa de esta materia.

2012: TypeScript

Microsoft publicó TypeScript en 2012. Lo diseñó Anders Hejlsberg, el mismo que creó C#, así que no te va a sorprender lo que vas a ver cuando lleguemos ahí.

TypeScript es JavaScript con un sistema de tipos encima. Aquella idea de la fallida cuarta edición, tipado estático opcional, volvió por afuera del estándar.

2015: el lenguaje se pone al día

ES6, también llamado ES2015, fue la actualización más grande de la historia del lenguaje: `let` y `const`, funciones flecha, clases, módulos, promesas, plantillas de texto, desestructuración.

Desde entonces el comité publica una versión por año, con los cambios que estén listos. ES2017 trajo `async` / `await`. ES2020 trajo el encadenamiento opcional. Las últimas ediciones agregan mejoras más acotadas, porque el lenguaje ya está maduro.

Dónde estamos hoy

Node dejó de ser el único entorno del lado del servidor: hoy compiten Deno y Bun, y los proveedores de nube ofrecen entornos livianos que corren JavaScript cerca del usuario. TypeScript se volvió el modo habitual de escribir proyectos grandes. Node incluso ejecuta archivos `.ts` de forma directa, aunque enseguida vamos a ver con qué límite.

Una línea de tiempo para tener a mano:

Año	Hito
1995	Brendan Eich escribe el prototipo en Netscape
1996	Microsoft responde con JScript en Internet Explorer 3
1997	Primera edición del estándar ECMA-262
1999	ES3: expresiones regulares y excepciones
2005	AJAX: el navegador pide datos sin recargar la página
2006	jQuery tapa las diferencias entre navegadores
2008	Chrome y V8: compilación a código de máquina
2009	ES5 y Node.js: JavaScript sale del navegador
2012	TypeScript
2015	ES6: el lenguaje moderno, y una versión por año
2017	<code>async</code> / <code>await</code>

Una regla que explica casi todo

El comité que define ECMAScript trabaja con una restricción: no romper la web. Hay miles de millones de páginas publicadas y ninguna se va a corregir. Si un cambio rompe código viejo, no entra.

Por eso JavaScript acumula rarezas en vez de corregirlas. Cuando más adelante veas que `typeof null` devuelve `"object"`, sabé que es un error de 1995 que quedó fijo para siempre. Arreglarlo rompería páginas que hoy funcionan.

Interpretado, compilado y el punto medio

Lo único que entiende la máquina

Un procesador ejecuta números: instrucciones binarias propias de su arquitectura. Cualquier lenguaje que escribas, en algún momento, tiene que terminar en esas instrucciones. La pregunta es cuándo pasa eso.

Hay dos respuestas clásicas.

Un lenguaje compilado traduce todo antes de ejecutar. Escribís el código en C, el compilador produce un ejecutable de código de máquina y después lo corrés. La traducción se hace una vez, con todo el tiempo del mundo para optimizar. El costo es que ese binario sirve para una arquitectura y un sistema operativo.

Un lenguaje interpretado no traduce nada por adelantado. Un programa, el intérprete, lee tu código y lo va ejecutando. Empieza al instante y corre en cualquier lado donde exista el intérprete. El costo es la velocidad: el intérprete vuelve a analizar la misma línea cada vez que pasa por ella, y eso dentro de un bucle se paga caro.

El modelo que ya conocés

.NET no está en ninguno de los dos extremos, y esto ya lo viste en Programación III aunque no le hayas puesto el nombre.

Cuando compilás C#, el compilador no genera código de máquina: genera un lenguaje intermedio, la IL. Ese código intermedio es portable. Recién cuando ejecutás la aplicación, el runtime traduce cada método a código de máquina la primera vez que se lo llama. Esa traducción en el momento de ejecución se llama compilación justo a tiempo, o JIT.

JavaScript llegó al mismo lugar por otro camino.

Qué hace realmente un motor moderno

Decir que JavaScript es interpretado era cierto hasta 2008. Hoy es una simplificación. Un motor como V8 hace algo bastante más interesante.

Cuando le llega tu código, el motor lo analiza y lo convierte a una representación interna, y de ahí a un bytecode propio. Un intérprete empieza a ejecutar ese bytecode de inmediato, porque en la web lo que importa es que la página responda ya.

Mientras ejecuta, el motor observa. Lleva la cuenta de qué funciones se llaman mucho y con qué tipos de datos. Cuando una función se vuelve frecuente, entra un compilador optimizador que la traduce a código de máquina, aprovechando lo que aprendió.

Ese último punto es la clave. Como el motor vio que una función siempre recibió números, puede generar código de máquina que asume números y saltea todas las verificaciones. El resultado es casi tan rápido como C.

La apuesta tiene un plan de contingencia. Si de repente esa función recibe un texto, la suposición se cae. El motor descarta el código optimizado y vuelve al bytecode. A eso se le llama desoptimización.

Por qué esto te importa como programador

De acá sale una consecuencia práctica que conviene entender desde el primer día:

```
// El motor ve siempre números y optimiza a fondo
function sumar(a, b) {
  return a + b;
}

for (let i = 0; i < 1000000; i++) {
  sumar(i, i + 1);
}

sumar("hola", "mundo"); // se cae la suposición y hay que desoptimizar
```

Lo mismo pasa con la forma de los objetos. El motor arma internamente una descripción de cada objeto, parecida a una clase, y la reutiliza mientras todos los objetos tengan las mismas propiedades en el mismo orden:

```
// Todos comparten la misma forma interna: el motor accede rapidísimo
const p1 = { nombre: "Ana", edad: 30 };
const p2 = { nombre: "Luis", edad: 25 };

// Esta forma es distinta y obliga al motor a rehacer el trabajo
const p3 = { edad: 40, nombre: "Sol" };
```

La conclusión no es que optimices a mano. Es esta: un código consistente, donde las variables mantienen su tipo y los objetos su forma, corre mucho más rápido. La disciplina que un lenguaje estático te impone, en JavaScript te conviene igual, aunque nadie te obligue.

Con esto ya podés responder con precisión la pregunta del título. JavaScript no se compila por adelantado y no necesita un paso de compilación para ejecutarse: en ese sentido es interpretado. Pero el motor sí compila a código de máquina mientras corre, guiado por lo que observa. Es un lenguaje interpretado con compilación justo a tiempo.

TypeScript y la transpilación

Compilar y transpilar

Compilar es traducir de un lenguaje a otro de más bajo nivel: de C# a IL, de C a código de máquina.

Transpilar es traducir entre lenguajes del mismo nivel. El resultado sigue siendo código fuente que una persona puede leer. De TypeScript sale JavaScript, y los dos son lenguajes de alto nivel.

Qué hace el compilador de TypeScript

TypeScript hace dos trabajos independientes, y confundirlos es el error más común al empezar.

El primero es verificar los tipos. Lee tus anotaciones, deduce las que faltan y te avisa si hay una incoherencia. Ese trabajo produce errores en tu editor y en la terminal.

El segundo es generar el JavaScript. Y ahí hace algo casi decepcionante: borra los tipos.

```
// TypeScript, lo que escribís
interface Contacto {
  nombre: string;
  email: string;
}

function saludar(contacto: Contacto): string {
  return `Hola ${contacto.nombre}`;
}
```

```
// JavaScript, lo que se ejecuta
function saludar(contacto) {
  return `Hola ${contacto.nombre}`;
}
```

La `interface` desapareció por completo. Las anotaciones también. Lo que queda es el JavaScript que habrías escrito igual.

Los tipos no existen en tiempo de ejecución

Esta es la idea más importante de esta sección, y viene con una diferencia fuerte respecto de C#.

En .NET los tipos viven en el programa que corre. Podés preguntar por el tipo de un objeto en pleno funcionamiento, recorrer sus propiedades y crear instancias por reflexión. El sistema de tipos es parte de la ejecución.

En TypeScript no. Los tipos son andamios: sostienen la obra mientras la construís y se retiran antes de la inauguración. En el momento en que tu código corre, no queda ni rastro de ellos.

De ahí salen dos conclusiones prácticas. La buena: TypeScript no cuesta nada en rendimiento, porque no agrega ni una instrucción. La incómoda: TypeScript no te protege de los datos que entran desde afuera. Volvemos sobre esto al final.

La otra transpilación

Convertir TypeScript a JavaScript no es el único uso de esta técnica.

Durante años el problema fue otro: escribías JavaScript moderno y tenías que ejecutarlo en navegadores viejos que no lo entendían. Herramientas como Babel resolvían eso transpilando JavaScript nuevo a JavaScript viejo, funciones flecha convertidas en funciones comunes y demás.

Hoy los navegadores se actualizan solos y el problema perdió peso, pero las herramientas quedaron y se volvieron más rápidas. Vas a escuchar nombres como esbuild o swc: hacen el mismo trabajo escritos en lenguajes compilados, y por eso tardan milisegundos donde antes tardaban minutos.

Una advertencia sobre Node y los archivos .ts

Node hoy ejecuta archivos `.ts` de forma directa. Conviene saber exactamente qué hace: borra los tipos y ejecuta el JavaScript resultante. No los verifica.

O sea que Node acepta sin protestar un archivo TypeScript lleno de errores de tipo. La verificación sigue siendo trabajo del compilador de TypeScript o de tu editor. Ejecutar no es lo mismo que revisar.

Dónde corre JavaScript

El lenguaje no sabe hacer nada

Acá hay una idea que suele pasarse por alto y que ordena todo lo demás.

El estándar ECMAScript define el lenguaje: variables, funciones, objetos, clases, promesas, `Math`, `JSON`. Nada más. En el estándar no hay ninguna forma de leer un archivo, mostrar un mensaje, abrir una conexión ni dibujar en la pantalla.

Un JavaScript puro no puede comunicarse con el mundo. Todo lo que interactúa con algo afuera se lo da el entorno que lo hospeda.

Por eso `console.log` no es parte del lenguaje, aunque lo uses todo el tiempo. Lo aporta el entorno.

El navegador

En el navegador, el entorno es el navegador mismo. Además del motor, te entrega un conjunto de objetos para trabajar con la página y la red:

- `document` y todo el DOM, para leer y modificar la página
- `window`, `localStorage`, `history`
- `fetch`, para pedir datos por HTTP
- `addEventListener`, para reaccionar a lo que hace el usuario

Cada navegador trae su propio motor. Chrome y Edge usan V8, Firefox usa SpiderMonkey y Safari usa JavaScriptCore. Los tres implementan el mismo estándar, así que el lenguaje se comporta igual. Las diferencias que todavía aparecen están en las APIs del entorno, no en el lenguaje.

Node

Node toma V8 y le arma otro entorno alrededor. No hay `document` ni `window`, porque no hay página. En cambio te da lo que necesita un programa de servidor:

- `fs` para archivos
- `http` para servidores y clientes
- `process` para argumentos, variables de entorno y salida
- acceso al sistema operativo y a la red

Comparemos lo mismo en los dos lados:

```
// Solo en el navegador
document.querySelector("h1").textContent = "Hola";

// Solo en Node
import { readFile } from "node:fs/promises";
const datos = await readFile("contactos.csv", "utf-8");

// En los dos: el lenguaje es el mismo
const contactos = datos.split("\n").map((linea) => linea.split(","));
```

La última línea es la que importa. La lógica es idéntica. Lo que cambia son las herramientas que te presta el entorno.

Un solo hilo

Node hereda del navegador una decisión de diseño: tu código corre en un solo hilo. No hay hilos ni bloqueos como en C#.

A cambio, ninguna operación de entrada y salida detiene la ejecución. Cuando pedís un archivo o una respuesta HTTP, el trabajo se delega y tu código sigue. Cuando el resultado está listo, un mecanismo llamado bucle de eventos ejecuta el código que quedó esperando.

Ese modelo es la razón por la que Node maneja bien miles de conexiones simultáneas con pocos recursos. Le vamos a dedicar una clase entera; por ahora quedate con la idea.

El resto del mapa

JavaScript se hospeda en más lugares de los que se suele pensar:

- Deno y Bun, entornos de servidor más nuevos que compiten con Node
- entornos de borde, que corren tu código en servidores repartidos por el mundo, cerca del usuario
- aplicaciones móviles, con React Native
- aplicaciones de escritorio, con Electron
- bases de datos y programas de escritorio que lo usan como lenguaje de extensión

Un mismo lenguaje, muchos entornos. Cuando algo no funciona, la primera pregunta útil casi siempre es la misma: dónde se está ejecutando esto.

Tipado: estático o dinámico, fuerte o débil

Acá conviene ir despacio, porque son dos preguntas distintas que todo el mundo mezcla.

Primera pregunta: cuándo se verifican los tipos

Un lenguaje de tipado estático verifica los tipos antes de ejecutar. El compilador conoce el tipo de cada variable y rechaza el programa si encuentra una incoherencia. C#, Java, Go y Rust funcionan así.

```
// C#: esto no compila  
int edad = "treinta";
```

Un lenguaje de tipado dinámico verifica los tipos mientras ejecuta. Las variables no tienen tipo: lo tienen los valores. La misma variable puede guardar cualquier cosa en distintos momentos. JavaScript, Python y Ruby funcionan así.

```
// JavaScript: perfectamente válido
let edad = 30;
edad = "treinta";
edad = { años: 30 };
edad = () => 30;
```

Segunda pregunta: cuánto se permite mezclar

Un lenguaje de tipado fuerte no convierte valores por su cuenta. Si mezclás tipos incompatibles, protesta.

Un lenguaje de tipado débil convierte lo que haga falta para que la operación tenga algún resultado, aunque el resultado no tenga sentido.

Python es dinámico y fuerte, y sirve para ver que los dos ejes son independientes:

```
# Python: error, se niega a adivinar
"5" + 3
```

```
// JavaScript: sin quejarse
"5" + 3; // "53"
```

Los cuatro cuadrantes, con ejemplos:

	Fuerte	Débil
Estático	C#, Java, Rust	C
Dinámico	Python, Ruby	JavaScript, PHP

JavaScript vive en el rincón más permisivo: dinámico y débil.

Cómo funciona la conversión implícita

Los ejemplos raros de JavaScript se vuelven predecibles cuando conocés la regla. No es magia ni azar: es un algoritmo, mal elegido pero consistente.

El operador `+` está sobrecargado. Sirve para sumar números y para concatenar textos. Si alguno de los dos operandos es un texto, JavaScript decide que querías concatenar y convierte el otro a texto.

El operador `-` no tiene sentido con textos. Entonces JavaScript convierte los dos operandos a número.

```
"5" + 3;    // "53"    → gana la concatenación
"5" - 3;    // 2       → los dos van a número
"5" * "2";  // 10      → los dos van a número
[] + {};    // "[object Object]"
```

La misma lógica aplica a las comparaciones. El operador `==` convierte antes de comparar; `===` compara sin convertir.

```
1 == "1";    // true
1 === "1";   // false
0 == false;  // true
0 === false; // false
null == undefined; // true
null === undefined; // false
```

De acá sale la primera regla práctica de la materia: usá siempre `===`. La única excepción razonable es `== null`, que atrapa `null` y `undefined` juntos.

Y una curiosidad que ya anticipamos:

```
typeof null; // "object"
```

Es un error de la implementación original de 1995. Sigue ahí porque corregirlo rompería páginas que hoy funcionan.

Qué ganás y qué perdés con cada modelo

El tipado dinámico te da velocidad para arrancar. Escribís menos, probás una idea sin declarar nada, escribís funciones que sirven para cualquier cosa que se comporte como corresponde. Para un guion corto, un prototipo o un análisis exploratorio, es una ventaja real.

El precio aparece con el tamaño. Los errores de tipo no se ven hasta que la línea se ejecuta, y a veces esa línea es un caso raro que llega a producción. El editor no sabe qué tiene un objeto, así que no puede autocompletar ni renombrar con seguridad. La única red de contención son las pruebas automatizadas, que tenés que escribir vos.

Mirá este caso, que es el que convence a todo el mundo:

```
function calcularTotal(pedido) {  
  return pedido.items.reduce((suma, item) => suma + item.precio, 0);  
}
```

Preguntas que el código no responde: ¿qué es un pedido? ¿`items` siempre existe? ¿`precio` es número o texto? Si es texto, `suma + item.precio` concatena y devuelve algo como `"01015"` sin fallar. El error aparece tres pantallas después, cuando alguien intenta mostrar ese total.

El tipado estático invierte el trato. Escribís más al principio y el compilador te contesta antes de ejecutar. Los tipos documentan el código sin que nadie mantenga la documentación. El editor sabe todo, así que autocompleta bien y renombra sin romper nada. Cambiar una función y que el compilador te muestre los quince lugares que hay que tocar es una experiencia que cambia la forma de trabajar en equipo.

El precio es la ceremonia y cierta rigidez. Hay programas correctos que el compilador rechaza porque no logra probar que están bien.

Por qué JavaScript llegó a TypeScript

Cuando JavaScript servía para validar un formulario, el tipado dinámico era la elección correcta. Nadie iba a montar un sistema de tipos para veinte líneas.

Pero después vino Node, vinieron las aplicaciones de una sola página y de golpe había equipos de veinte personas manteniendo cientos de miles de líneas en un lenguaje sin verificación previa. El modelo dejó de alcanzar.

TypeScript es la respuesta a ese problema, y su diseño está condicionado por una restricción: no se puede cambiar JavaScript. Hay demasiado código escrito. Entonces TypeScript no reemplaza el lenguaje, se le suma como una capa que se verifica antes y se borra después.

Tres consecuencias que se ven en el día a día.

El tipado es gradual. Todo JavaScript válido es TypeScript válido. Podés tipar un archivo y dejar el resto como está, y migrar de a poco.

La inferencia hace casi todo el trabajo. No hace falta anotar cada variable, porque el compilador deduce lo que puede:

```
let nombre = "Ana";           // TypeScript infiere string  
nombre = 30;                  // error, no compila  
  
const contactos = ["Ana", "Luis"];  
contactos.map((c) => c.toUpperCase()); // sabe que c es string
```


El sistema de tipos es estructural, no nominal. Esto es una diferencia grande con C# y conviene tenerla presente. En C#, una clase implementa una interfaz solo si lo declara. En TypeScript alcanza con que la forma coincida:

```
interface Punto {
  x: number;
  y: number;
}

// Nunca se declaró como Punto, pero tiene la forma correcta
const posicion = { x: 10, y: 20 };

function dibujar(p: Punto) {
  console.log(p.x, p.y);
}

dibujar(posicion); // válido
```

El límite de TypeScript

Volvemos a la idea de que los tipos se borran, porque tiene una consecuencia que causa muchos errores en producción.

TypeScript verifica lo que vos escribís. No puede verificar lo que entra desde afuera.

```
interface Contacto {
  nombre: string;
  email: string;
}

const respuesta = await fetch("/api/contactos");
const contactos: Contacto[] = await respuesta.json();
```

Esa última línea es una promesa que TypeScript te cree sin comprobar nada. `json()` devuelve datos arbitrarios, y vos afirmaste que son un arreglo de contactos. Si el servidor cambió el campo `email` por `correo`, el compilador no se entera y el programa falla al ejecutarse, igual que en JavaScript puro.

Lo mismo pasa con `JSON.parse`, con lo que llega de un formulario, de una base de datos o de un archivo.

La solución es validar en la frontera: revisar los datos externos en el momento en que entran, con código que corre de verdad. Para eso se usan bibliotecas de validación, y lo vamos a ver cuando armemos la API.

Y hay que nombrar la escotilla de escape. El tipo `any` desactiva la verificación:

```
const datos: any = obtenerAlgo();
datos.loQueSea.tampocoExiste(); // TypeScript no dice nada
```

`any` sirve durante una migración. Usado por costumbre, convierte a TypeScript en JavaScript con más ruido.

Por qué en esta materia empezamos por JavaScript

Podríamos empezar directamente por TypeScript, como hacen muchos cursos. No lo vamos a hacer, por una razón.

TypeScript no es un lenguaje aparte: es JavaScript con verificación previa. Si aprendés TypeScript sin entender qué pasa cuando el código corre, vas a tener un modelo mental equivocado. Vas a creer que los tipos existen en la ejecución, no vas a entender por qué los datos de una API se te escapan del control, y cada error raro va a parecer arbitrario.

Así que primero vamos a entender el lenguaje que se ejecuta. Después le agregamos los tipos y vas a ver, con precisión, qué problema resuelve cada uno.

Para llevarte de esta clase

- JavaScript nació en 1995 en diez días para validar formularios, y sus rarezas se explican por esa historia y por la regla de no romper la web
- ECMAScript es el nombre del estándar; JavaScript es el nombre del lenguaje
- no necesita compilación previa, pero el motor compila a código de máquina mientras ejecuta, guiado por lo que observa: es interpretado con compilación justo a tiempo
- un código con tipos y formas de objeto consistentes corre más rápido, aunque nadie te obligue a mantenerlo así
- el lenguaje no sabe leer archivos ni dibujar en pantalla: todo eso lo aporta el entorno, sea el navegador o Node
- estático o dinámico responde cuándo se verifican los tipos; fuerte o débil responde cuánto convierte el lenguaje por su cuenta. JavaScript es dinámico y débil
- usá siempre `===`
- TypeScript agrega verificación antes de ejecutar y después borra los tipos: no cuesta rendimiento y no protege de los datos que entran desde afuera

Para probar antes de la próxima clase

Abrí la consola del navegador con F12 y probá cada expresión. Antes de apretar Enter, escribí qué creés que va a devolver:

1. Evaluá `"5" + 3` y después `"5" - 3`. Explicá la diferencia con la regla de sobrecarga del operador `+`.
2. Evaluá `0.1 + 0.2`. El resultado no es `0.3`; averiguá qué es un número de punto flotante y por qué pasa lo mismo en C#.
3. Evaluá `[] == false` y `[] === false`. Seguí el camino de conversión que aplica cada operador.
4. Evaluá `NaN === NaN` y buscá qué hace `Number.isNaN`.
5. Declará `let x = 5`, después `x = "cinco"`, y en cada paso evaluá `typeof x`.
6. Escribí en la consola `typeof document` y `typeof window`. Después buscá qué devolverían esas mismas expresiones en Node.

Variables y tipos básicos en JavaScript

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte de la segunda clase de JavaScript

La clase pasada vimos de dónde viene JavaScript y por qué es un lenguaje dinámico y débilmente tipado. Hoy bajamos a la práctica. Vamos a ver cómo se guarda un dato, qué tipos de datos existen y qué podés hacer con cada uno.

El orden es a propósito. Primero el recipiente, después el contenido, al final las reglas que JavaScript aplica cuando mezclás contenidos distintos. Esa última parte es la que más problemas causa en producción, y recién tiene sentido cuando ya conocés los tipos uno por uno.

Qué es una variable

Un programa trabaja con datos que están en la memoria. La memoria son direcciones numéricas, y nadie quiere escribir código pensando en direcciones. Una variable es un nombre que le ponés a un valor para poder referirte a él.

En un lenguaje estático como C#, ese nombre viene con un tipo pegado. `int edad` significa que ahí adentro va a haber un entero y nada más.

En JavaScript no. El nombre no tiene tipo: el tipo lo tiene el valor guardado. La variable es solo una etiqueta que podés mover de un valor a otro.

```
let dato = 42;           // ahora apunta a un número
dato = "cuarenta";      // ahora apunta a un texto
dato = true;            // ahora apunta a un booleano
```

Conviene pensarlo así: la variable no es una caja con un valor adentro, es una flecha que apunta a un valor. Esa imagen se vuelve importante cuando lleguemos a `const`.

Las tres formas de declarar

JavaScript tiene tres palabras para declarar variables. Existen las tres por razones históricas, y solo dos se usan hoy.

var, la forma vieja

`var` es la declaración original de 1995. Tiene dos comportamientos que hoy se consideran errores de diseño.

El primero es su alcance: `var` no respeta los bloques, solo las funciones.

```
function ejemplo() {  
  if (true) {  
    var mensaje = "hola";  
  }  
  console.log(mensaje); // "hola", la variable sobrevivió al bloque  
}
```

El segundo es la elevación, o hoisting. Antes de ejecutar una función, el motor recorre el código y registra todas las declaraciones `var` que encuentra, en cualquier nivel de anidamiento. Las crea al principio con el valor `undefined`. Recién cuando la ejecución llega a la línea original se asigna el valor real.

```
console.log(nombre); // undefined, no falla  
var nombre = "Ana";  
console.log(nombre); // "Ana"
```

Ese `undefined` silencioso escondió errores durante veinte años. Además `var` permite redeclarar la misma variable sin protestar, así que dos programadores podían pisarse el nombre y no enterarse.

En esta materia no vamos a usar `var`. Lo aprendés para poder leer código viejo, nada más.

let, la variable que cambia

`let` llegó en ES6 y corrige las dos cosas.

Respeta el bloque. Una variable declarada entre llaves existe solo entre esas llaves.

```
function ejemplo() {  
  if (true) {  
    let mensaje = "hola";  
    console.log(mensaje); // "hola"  
  }  
  console.log(mensaje); // ReferenceError: mensaje is not defined  
}
```

Y no permite usarla antes de declararla. La declaración también se registra al entrar al bloque, pero la variable queda en un estado inaccesible hasta la línea donde la declarás. Ese tramo se llama zona muerta temporal.

```
console.log(nombre); // ReferenceError: no se puede acceder antes de la declaración
let nombre = "Ana";
```

Fallar es mejor que devolver `undefined`. Un error te detiene en la línea del problema; un `undefined` te aparece tres funciones más adelante.

Tampoco podés redeclararla en el mismo alcance:

```
let x = 1;
let x = 2; // SyntaxError
```

const, la referencia que no se mueve

`const` se comporta igual que `let` en cuanto a alcance y zona muerta, con dos diferencias: tenés que asignarle un valor al declararla y no podés reasignarla después.

```
const PI = 3.14159;
PI = 3; // TypeError: Assignment to constant variable
```

Acá viene el punto que más confunde, y donde sirve la imagen de la flecha. `const` congela la flecha, no lo que hay del otro lado. Si el valor es un objeto o un arreglo, el contenido se puede modificar sin problema.

```
const contacto = { nombre: "Ana" };

contacto.nombre = "Luis"; // válido, modificás el objeto
contacto.email = "l@mail.com"; // válido, agregás una propiedad
contacto = {}; // TypeError, movés la flecha
```

`const` no significa inmutable. Significa que ese nombre va a apuntar siempre al mismo valor. Si querés inmutabilidad real hay otras herramientas, y las vamos a ver más adelante.

Cuál usar

La regla de la materia es simple y es la que se usa en la industria:

- declará todo con `const`
- cambiá a `let` solo cuando necesites reasignar

- no uses `var`

No es una manía estilística. Cuando lees `const`, sabés de una que ese nombre no va a cambiar en las próximas cincuenta líneas, y eso es una cosa menos que rastrear.

Un resumen de las tres

	var	let	const
Alcance	función	bloque	bloque
Antes de declararla	<code>undefined</code>	error	error
Redeclarar	sí	no	no
Reasignar	sí	sí	no
Inicialización obligatoria	no	no	sí

Alcance y vida de una variable

Ya nombramos el alcance varias veces. Vale la pena ordenarlo, porque es un concepto que vas a usar todos los días.

El alcance de una variable es la región del código donde ese nombre se puede usar. En JavaScript hay tres niveles.

El alcance global es el más externo. Una variable declarada fuera de toda función y de todo bloque se ve desde cualquier parte. Usalo lo menos posible: todo lo global es un nombre que cualquiera puede pisar.

El alcance de función abarca el cuerpo entero de una función, incluidos sus bloques internos. Es el que respeta `var`.

El alcance de bloque es cualquier par de llaves: un `if`, un `for`, un `while` o llaves sueltas. Es el que respetan `let` y `const`.

Los alcances se anidan, y desde adentro se ve hacia afuera pero no al revés:


```

const nivel1 = "global";

function externa() {
  const nivel2 = "función";

  if (true) {
    const nivel3 = "bloque";
    console.log(nivel1, nivel2, nivel3); // ve los tres
  }

  console.log(nivel3); // ReferenceError
}

```

Cuando el motor encuentra un nombre, lo busca en el bloque actual. Si no está, sube al alcance que lo contiene, y así hasta el global. Si no lo encuentra en ningún lado, lanza un `ReferenceError`. Esa búsqueda se llama cadena de alcances.

Si un nombre se repite en dos niveles, gana el más cercano. El de afuera queda tapado, y eso se llama sombreado:

```

const usuario = "Ana";

function saludar() {
  const usuario = "Luis"; // sombrea al de afuera
  console.log(usuario);    // "Luis"
}

saludar();
console.log(usuario);      // "Ana"

```

La vida de una variable es cuánto tiempo ocupa memoria. En general nace cuando la ejecución entra a su alcance y muere cuando sale. No la liberás vos: JavaScript tiene recolección de basura, y el motor libera todo valor al que ya no llegue ninguna referencia.

Hay una excepción que vale anticipar. Si una función interna sigue usando una variable de la función externa, esa variable no se libera aunque la externa haya terminado:

```

function crearContador() {
  let cuenta = 0;
  return () => ++cuenta;
}

const contar = crearContador();
contar(); // 1
contar(); // 2, cuenta sigue viva

```

Eso es una clausura, y le vamos a dedicar una clase entera. Por ahora quedate con que el alcance manda más que el momento de la ejecución.

Cómo se nombran las variables

Las reglas del lenguaje

Un nombre válido en JavaScript:

- empieza con una letra, un guion bajo o un signo pesos
- puede seguir con letras, dígitos, guiones bajos o signos pesos
- no puede empezar con un dígito
- no puede ser una palabra reservada del lenguaje, como `let`, `class`, `return` o `function`
- distingue mayúsculas de minúsculas, así que `total` y `Total` son dos variables distintas

El lenguaje acepta caracteres Unicode, o sea que `precioTotalAño` compila. No lo hagas igual: el código se lee y se comparte en inglés técnico, y los acentos traen problemas de codificación en herramientas viejas.

Las convenciones

Las reglas dicen qué acepta el motor. Las convenciones dicen qué espera leer otra persona, y en un equipo eso pesa más.

Convención	Se usa para	Ejemplo
camelCase	variables y funciones	<code>precioTotal</code> , <code>calcularIva</code>
PascalCase	clases y componentes	<code>Contacto</code> , <code>ListaDeContactos</code>
MAYUSCULA_CON_GUIONES	constantes de configuración	<code>IVA_GENERAL</code> , <code>URL_BASE</code>

Ojo con esto si venís de C#: allá los métodos y las propiedades públicas van en PascalCase. En JavaScript son camelCase. Escribir `CalcularTotal()` en JavaScript es tan llamativo como escribir `calcularTotal()` en C#.

Más allá del formato, hay costumbres sobre el contenido del nombre:

- las variables son sustantivos: `contacto`, `total`, `usuarioActual`
- las funciones empiezan con verbo: `calcularTotal`, `buscarContacto`, `enviarMail`
- los booleanos se leen como una afirmación: `esValido`, `tieneStock`, `estaActivo`
- las colecciones van en plural: `contactos`, `precios`

- las constantes de configuración en mayúsculas se reservan para valores fijos del programa, no para todo lo declarado con `const`

Y una regla general: el largo del nombre debería crecer con el alcance. Una `i` dentro de un `for` de tres líneas está bien. Una `d` que vive en todo un módulo, no.

```
// Difícil de leer
const d = new Date();
const x = p * 0.21;

// Se explica solo
const fechaEmision = new Date();
const iva = precio * ALICUOTA_IVA;
```

Los tipos de dato

JavaScript tiene ocho tipos. Siete son primitivos y uno es el objeto.

Tipo	Para qué sirve	Ejemplo
<code>number</code>	todos los números	<code>42</code> , <code>3.14</code>
<code>string</code>	texto	<code>"hola"</code>
<code>boolean</code>	verdadero o falso	<code>true</code>
<code>undefined</code>	una variable sin valor asignado	<code>undefined</code>
<code>null</code>	la ausencia deliberada de valor	<code>null</code>
<code>bigint</code>	enteros más grandes que los que soporta <code>number</code>	<code>9007199254740993n</code>
<code>symbol</code>	identificadores únicos, uso avanzado	<code>Symbol("id")</code>
<code>object</code>	todo lo demás: objetos, arreglos, funciones, fechas	<code>{ a: 1 }</code> , <code>[1, 2]</code>

Para saber qué tipo tiene un valor está el operador `typeof`:

```
typeof 42;           // "number"
typeof "hola";       // "string"
typeof true;         // "boolean"
typeof undefined;    // "undefined"
typeof 10n;          // "bigint"
typeof Symbol();     // "symbol"
typeof { a: 1 };     // "object"
typeof [1, 2];       // "object"
typeof function(){}; // "function"
typeof null;         // "object" ← el error de 1995
```

Dos filas de esa tabla merecen aclaración. `typeof` de un arreglo devuelve `"object"` porque un arreglo es un objeto; para distinguirlos está `Array.isArray()`. Y `typeof null` devuelve `"object"` por aquel error de la primera implementación que ya no se puede corregir.

undefined y null no son lo mismo

Los dos representan ausencia, pero con intenciones distintas.

`undefined` es la ausencia por omisión. Lo pone el lenguaje: una variable declarada y no asignada, una propiedad que no existe, un parámetro que no se pasó, una función que no devuelve nada.

`null` es la ausencia deliberada. Lo ponés vos para decir "acá no hay valor y es a propósito".

```
let sinAsignar;  
console.log(sinAsignar); // undefined, no lo decidió nadie  
  
let contactoSeleccionado = null; // decidiste que todavía no hay ninguno
```

En una base de datos y en una API, la diferencia importa: `null` viaja como un valor y `undefined` directamente no aparece en el JSON.

Primitivos y objetos se copian distinto

Este punto genera errores desde el primer día, así que va ahora.

Un valor primitivo se copia por valor. Cada variable tiene su propia copia y son independientes.

```
let a = 10;  
let b = a;  
b = 20;  
console.log(a); // 10
```

Un objeto se copia por referencia. Las dos variables apuntan al mismo objeto en memoria.

```
const original = { nombre: "Ana" };  
const copia = original;  
copia.nombre = "Luis";  
console.log(original.nombre); // "Luis"
```

Es la misma imagen de la flecha. Al asignar copiás la flecha, no lo que hay al final.

Además, los primitivos son inmutables. No podés modificar el número 5 ni la cadena `"hola"`: solo podés crear otro valor y apuntar la variable ahí.

Booleanos

Para qué sirven y cómo se escriben

El booleano es el tipo de las decisiones. Tiene dos valores literales y ninguno más:

```
const activo = true;
const eliminado = false;
```

Es el resultado de toda comparación, y lo que evalúan `if`, `while` y el operador ternario.

Los operadores de comparación

Operador	Significado
<code>===</code>	igual en valor y tipo
<code>!==</code>	distinto en valor o tipo
<code>==</code>	igual después de convertir
<code>!=</code>	distinto después de convertir
<code><</code> <code>></code> <code><=</code> <code>>=</code>	orden

La regla de la materia ya la dijimos y la repetimos porque es la que más se olvida: usá siempre `===`. `==` convierte antes de comparar y produce resultados que nadie espera. Volvemos sobre eso en la última sección.

Truthy y falsy

Acá está la primera idea propia de JavaScript.

Cuando ponés un valor donde se espera un booleano, el lenguaje no protesta: lo convierte. Y esa conversión sigue una lista corta y cerrada.

Estos ocho valores se convierten en `false`, y se los llama falsy:

```
false
0
-0
0n
""      // cadena vacía
null
undefined
NaN
```

Todo lo demás se convierte en `true`. Todo, sin excepciones. Incluidas estas tres, que sorprenden a todo el mundo:

```
Boolean("0");    // true, es una cadena con un carácter
Boolean("false"); // true, ídem
Boolean([]);     // true, un arreglo vacío es un objeto
Boolean({});     // true, un objeto vacío también
```

La regla de fondo es esa: cualquier objeto es *truthy*, esté vacío o no. Si querés saber si un arreglo tiene elementos, preguntá por `.length`, no por el arreglo.

```
if (contactos.length > 0) { /* ... */ } // correcto
if (contactos) { /* ... */ }           // siempre entra
```

Para convertir a booleano de forma explícita tenés `Boolean(valor)` o el doble negación `!!valor`. Las dos hacen lo mismo; `Boolean()` se lee mejor.

Los operadores lógicos y el cortocircuito

JavaScript tiene tres operadores lógicos: `&&`, `||` y `!`.

Los dos primeros tienen una particularidad. En la mayoría de los lenguajes devuelven un booleano. En JavaScript devuelven uno de los dos operandos, sin convertirlo.

El `&&` evalúa el izquierdo. Si es falsy, lo devuelve y ni mira el derecho. Si es *truthy*, devuelve el derecho.

```
true && "hola";    // "hola"
false && "hola";    // false
0 && "hola";        // 0
"a" && "b";         // "b"
```

El `||` hace lo inverso. Si el izquierdo es *truthy*, lo devuelve y corta. Si no, devuelve el derecho.

```
"Ana" || "Anónimo"; // "Ana"
"" || "Anónimo";    // "Anónimo"
null || "Anónimo";  // "Anónimo"
```

Que corte antes de evaluar el segundo operando se llama cortocircuito, y tiene dos usos muy comunes.

El primero es proteger un acceso que podría fallar:

```
// Si contacto es null, nunca se evalúa contacto.email
if (contacto && contacto.email) {
    enviarMail(contacto.email);
}
```

El segundo es dar un valor por defecto:

```
const nombre = nombreIngresado || "Anónimo";
```

Ese segundo uso tiene una trampa. `||` reemplaza cualquier valor falsy, no solo la ausencia:

```
const cantidad = cantidadIngresada || 1;
// Si el usuario ingresó 0, cantidad queda en 1
```

Para eso está `??`, el operador de coalescencia nula. Solo reemplaza cuando el izquierdo es `null` o `undefined`:

```
const cantidad = cantidadIngresada ?? 1;
// Si el usuario ingresó 0, cantidad queda en 0
```

La regla práctica: si querés cubrir la ausencia de un dato, usá `??`. Usá `||` solo cuando de verdad quieras tratar el `0` y la cadena vacía como si no hubiera dato.

Un pariente cercano es el encadenamiento opcional, `?.`, que corta la expresión si lo de la izquierda es `null` o `undefined`:

```
contacto?.direccion?.ciudad;
// undefined si falta contacto o falta direccion, en vez de fallar
```

Y el ternario, que es la única forma de escribir un `if` que devuelve un valor:

```
const etiqueta = esActivo ? "Activo" : "Inactivo";
```

Números

Un solo tipo numérico

En C# elegís entre `int`, `long`, `float`, `decimal` y varios más. En JavaScript hay un solo tipo `number`, y siempre es un número de punto flotante de 64 bits del estándar IEEE 754.

No existe el entero como tipo aparte. `42` y `42.0` son exactamente el mismo valor.

Eso simplifica mucho y trae dos consecuencias que hay que conocer.

La primera es la precisión decimal:

```
0.1 + 0.2;           // 0.30000000000000004
0.1 + 0.2 === 0.3;   // false
```

No es un error de JavaScript. Es cómo funciona el punto flotante binario, y pasa igual con `double` en C# o en Java. Algunos decimales no tienen representación exacta en binario, del mismo modo que un tercio no la tiene en decimal.

Para comparar decimales, comparás la diferencia contra una tolerancia:

```
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON; // true
```

Y para plata, no uses punto flotante. La solución habitual es trabajar en centavos con enteros, o usar una biblioteca decimal. Lo vamos a aplicar cuando el CRM maneje importes.

La segunda consecuencia es el rango seguro de los enteros. Un `number` representa exactamente los enteros hasta 2 elevado a 53 menos uno, o sea 9.007.199.254.740.991. Más allá de eso empieza a perder precisión:

```
Number.MAX_SAFE_INTEGER; // 9007199254740991
9007199254740992 === 9007199254740993; // true, y no debería
```

Para enteros más grandes existe `bigint`, que se escribe con una `n` al final. No se mezcla con `number` en una misma operación.

```
const grande = 9007199254740993n;
grande + 1n; // 9007199254740994n
grande + 1;  // TypeError
```


Las formas de escribir un número

```
const decimal      = 255;
const conDecimales= 3.14;
const hexadecimal  = 0xff;           // 255
const octal        = 0o377;         // 255
const binario      = 0b11111111;    // 255
const exponencial  = 1.5e6;         // 1500000
const muyChico     = 2e-3;          // 0.002
const legible      = 1_000_000;     // 1000000, el guion bajo lo ignora el motor
```

Los separadores de miles con guion bajo son solo para el ojo humano. El valor es el mismo.

Los tres valores especiales

```
Infinity; // más grande que cualquier número
-Infinity;
NaN;      // "not a number", el resultado de una operación imposible
```

`Infinity` aparece al dividir por cero, que en JavaScript no lanza excepción:

```
1 / 0; // Infinity
-1 / 0; // -Infinity
0 / 0; // NaN
```

`NaN` tiene una propiedad única en todo el lenguaje: no es igual a sí mismo.

```
NaN === NaN; // false
```

Por eso no lo podés detectar comparando. Se detecta con `Number.isNaN()`:

```
Number.isNaN(NaN); // true
Number.isNaN("hola"); // false, no es NaN, es una cadena
isNaN("hola"); // true, convierte antes de preguntar
```

Existen las dos funciones. La global `isNaN()` convierte el argumento antes de evaluarlo y por eso da respuestas engañosas. Usá siempre `Number.isNaN()`.

Operadores aritméticos

Operador	Operación	Ejemplo
<code>+</code>	suma	<code>5 + 3</code> es <code>8</code>
<code>-</code>	resta	<code>5 - 3</code> es <code>2</code>
<code>*</code>	multiplicación	<code>5 * 3</code> es <code>15</code>
<code>/</code>	división	<code>5 / 2</code> es <code>2.5</code>
<code>%</code>	resto	<code>5 % 2</code> es <code>1</code>
<code>**</code>	potencia	<code>5 ** 2</code> es <code>25</code>

Atención con la división: siempre es real. No existe la división entera del `int` de C#.

```
5 / 2;           // 2.5, no 2
Math.trunc(5 / 2); // 2, si querés el entero
```

El `%` devuelve el resto y conserva el signo del dividendo, que no es lo mismo que el módulo matemático:

```
7 % 3;    // 1
-7 % 3;   // -1
```

Están también los operadores unarios y los de incremento:

```
const positivo = +"42"; // convierte a número: 42
const negativo = -x;

let n = 5;
n++;    // devuelve 5 y después incrementa
++n;    // incrementa y después devuelve 7
```

La diferencia entre `n++` y `++n` solo importa si usás el resultado en la misma expresión. Si no, escribí el que se lea mejor.

Y los operadores de asignación compuesta, que son abreviaturas:

```
total += 100; // total = total + 100
total -= 50;
total *= 2;
total /= 4;
total %= 3;
total **= 2;
```

Precedencia de operadores

Cuando una expresión combina varios operadores, el orden de evaluación no es de izquierda a derecha: cada operador tiene una precedencia.

```
2 + 3 * 4;    // 14, no 20
(2 + 3) * 4;  // 20
```

Esta es la tabla, de mayor a menor precedencia, con lo que vas a usar en esta materia:

Nivel	Operadores
1	<code>()</code> agrupación
2	<code>.</code> <code>[]</code> <code>()</code> acceso y llamada
3	<code>++</code> <code>--</code> sufijos
4	<code>!</code> <code>~</code> <code>+</code> <code>-</code> unarios, <code>typeof</code> , <code>++</code> <code>--</code> prefijos
5	<code>**</code>
6	<code>*</code> <code>/</code> <code>%</code>
7	<code>+</code> <code>-</code>
8	<code><<</code> <code>>></code> <code>>>></code>
9	<code><</code> <code>></code> <code><=</code> <code>>=</code>
10	<code>==</code> <code>!=</code> <code>===</code> <code>!==</code>
11	<code>&</code>
12	<code>^</code>
13	<code> </code>
14	<code>&&</code>
15	<code> </code> y <code>??</code>
16	<code>? :</code> ternario
17	<code>=</code> <code>+=</code> <code>-=</code> y demás asignaciones

Tres cosas que conviene retener de esa tabla.

La primera: los operadores de bits tienen menos precedencia que las comparaciones. Eso rompe la intuición de casi todo el mundo.

```
a & b === c;    // se evalúa como a & (b === c)
(a & b) === c;  // casi siempre es lo que querías
```

La segunda: `**` se asocia hacia la derecha, al revés de los demás. Y el lenguaje directamente prohíbe combinarlo con un menos unario sin paréntesis, para que no haya ambigüedad.

```
2 ** 3 ** 2;    // 512, se evalúa 2 ** (3 ** 2)
-2 ** 2;        // SyntaxError
(-2) ** 2;      // 4
```

La tercera: `??` no se puede mezclar con `&&` ni con `||` sin paréntesis, por la misma razón.

```
a || b ?? c;    // SyntaxError
(a || b) ?? c;  // válido
```

La regla práctica es esta: sabé la precedencia para leer código ajeno, y escribí paréntesis para que nadie tenga que saberla al leer el tuyo.

Operadores de bits

Estos operadores trabajan con la representación binaria del número.

Operador	Operación
<code>&</code>	y
<code> </code>	o
<code>^</code>	o exclusivo
<code>~</code>	negación
<code><<</code>	desplazamiento a la izquierda
<code>>></code>	desplazamiento a la derecha con signo
<code>>>></code>	desplazamiento a la derecha sin signo

Acá hay una particularidad importante. Los números de JavaScript son de 64 bits con punto flotante, pero estos operadores no trabajan así. Antes de operar, el motor convierte el valor a un entero con signo de 32 bits, opera, y devuelve otro número de 64 bits.

Dos consecuencias. Los decimales se truncan, y los valores que no entran en 32 bits se desbordan:

```
5.9 | 0;          // 5, trunca
2 ** 31 | 0;      // -2147483648, se desborda
```

Vas a ver `| 0` usado como truncamiento rápido en código viejo. Es frágil por lo anterior; usá `Math.trunc()`.

El uso legítimo de estos operadores son las banderas y las máscaras. Cuando tenés muchas opciones de sí o no, podés guardarlas en un solo número:

```

const LECTURA    = 0b001; // 1
const ESCRITURA  = 0b010; // 2
const BORRADO     = 0b100; // 4

let permisos = LECTURA | ESCRITURA; // 0b011, tiene las dos

permisos & ESCRITURA; // 2, distinto de 0: la tiene
Boolean(permisos & BORRADO); // false: no la tiene

permisos |= BORRADO; // agregar permiso
permisos &= ~ESCRITURA; // quitar permiso
permisos ^= LECTURA; // alternar permiso

```

El otro uso frecuente son los colores, donde cada componente ocupa 8 bits dentro de un mismo número:

```

const color = 0xff8800;
const rojo  = (color >> 16) & 0xff; // 255
const verde = (color >> 8) & 0xff;  // 136
const azul  = color & 0xff;         // 0

```

En desarrollo web no vas a usar bits todos los días. Pero aparecen en permisos, en manejo de imágenes y en protocolos binarios, así que conviene reconocerlos.

La librería estándar de números

Las funciones matemáticas están en dos lugares: el objeto `Math` y el objeto `Number`.

`Math` agrupa las operaciones:

```

Math.round(2.5); // 3, redondea al entero más cercano
Math.round(-2.5); // -2, el medio siempre va hacia arriba
Math.floor(2.9); // 2, hacia abajo
Math.ceil(2.1); // 3, hacia arriba
Math.trunc(-2.9); // -2, corta la parte decimal
Math.abs(-5); // 5
Math.min(3, 1, 2); // 1
Math.max(3, 1, 2); // 3
Math.pow(2, 10); // 1024, equivale a 2 ** 10
Math.sqrt(16); // 4
Math.random(); // decimal entre 0 y 1, sin llegar al 1
Math.PI; // 3.141592653589793

```

Un número entero al azar entre dos valores, que es el ejercicio clásico:

```
function entero(min, max) {  
  return Math.floor(Math.random() * (max - min + 1)) + min;  
}
```

`Number` agrupa las verificaciones y las constantes:

```
Number.isInteger(5.0);    // true  
Number.isFinite(1 / 0);   // false  
Number.isNaN(NaN);       // true  
Number.parseFloat("3.14"); // 3.14  
Number.parseInt("42px");  // 42  
Number.MAX_SAFE_INTEGER;  // 9007199254740991  
Number.EPSILON;          // la menor diferencia representable
```

Y los propios números tienen métodos para mostrarse:

```
const precio = 1234.5678;  
  
precio.toFixed(2);        // "1234.57", devuelve texto  
precio.toString(2);       // en binario  
(255).toString(16);      // "ff"  
  
precio.toLocaleString("es-AR", {  
  style: "currency",  
  currency: "ARS",  
});                        // "$ 1.234,57"
```

Fijate que `toFixed` devuelve una cadena, no un número. Es un error frecuente seguir operando con el resultado.

Cadenas

Para qué sirven y cómo se escriben

Una cadena es texto. Internamente JavaScript la guarda como una secuencia de unidades de 16 bits, en el formato UTF-16.

Hay tres formas de escribirla, y las tres producen el mismo tipo:

```
const simples = 'Hola';  
const dobles  = "Hola";  
const invertidas = `Hola`;
```

Las comillas simples y dobles son intercambiables. Elegí una y mantenela en todo el proyecto; la mayoría de las herramientas de formato usan comillas dobles. La ventaja de tener dos es poder incluir una dentro de la otra sin escapar:

```
const cita = "Ella dijo 'basta'";
const otra = 'El archivo se llama "datos.csv"';
```

Cuando no hay más remedio, se escapan los caracteres con la barra invertida:

```
"Ella dijo \"basta\"";
"Primera línea\nSegunda línea"; // \n es un salto de línea
"Columna1\tColumna2";           // \t es una tabulación
"C:\\Users\\Ana";                // \\ es una barra invertida
"\u00e1";                        // "á" por su código Unicode
```

Las comillas invertidas

Las comillas invertidas, o plantillas de texto, llegaron en ES6 y resuelven dos problemas viejos.

El primero es la interpolación. Podés meter cualquier expresión dentro de `${}` y su resultado se convierte a texto:

```
const nombre = "Ana";
const edad = 30;

// Antes
const viejo = "Hola " + nombre + ", tenés " + edad + " años";

// Ahora
const nuevo = `Hola ${nombre}, tenés ${edad} años`;
```

Adentro de `${}` va una expresión completa, no solo un nombre:

```
`Total: ${precio * cantidad}`;
`El año que viene tenés ${edad + 1}`;
`Estado: ${activo ? "Activo" : "Inactivo"}`;
`Nombre: ${contacto?.nombre ?? "sin datos"}`;
```

El segundo problema es el texto de varias líneas. Con comillas normales tenías que concatenar y meter `\n` a mano. Con invertidas, el salto de línea que escribís es el salto que sale:


```
const html = `

Ese uso lo vas a repetir mucho cuando generemos HTML desde JavaScript. Cuidado con una cosa: la indentación que escribís forma parte de la cadena.



## Las cadenas son inmutables



No podés modificar una cadena. Todos los métodos que parecen modificarla en realidad devuelven una nueva:



```
let saludo = "hola";
saludo[0] = "H"; // no hace nada, no falla
console.log(saludo); // "hola"

saludo = saludo.toUpperCase(); // acá sí, porque reasignás
console.log(saludo); // "HOLA"
```



## Operadores sobre cadenas



El + concatena, y el += acumula:



```
const nombreCompleto = "Ana" + " " + "Pérez";

let lista = "";
lista += "primero, ";
lista += "segundo";
```



Los operadores de orden comparan carácter por carácter según el código UTF-16, no según el alfabeto:



```
"a" < "b"; // true
"Z" < "a"; // true, las mayúsculas van antes en la tabla
"ñ" < "z"; // false, los acentuados van después
"10" < "9"; // true, compara "1" contra "9"
```



Para ordenar texto en español, usá localeCompare, que conoce las reglas del idioma:



38


```

```
["ñandú", "zorro", "ámbar"].sort((a, b) => a.localeCompare(b, "es"));
// ["ámbar", "ñandú", "zorro"]
```

La librería estándar de cadenas

Método	Qué hace	Ejemplo
<code>.length</code>	cantidad de caracteres	<code>"hola".length</code> es <code>4</code>
<code>.at(i)</code>	carácter en la posición, acepta negativos	<code>"hola".at(-1)</code> es <code>"a"</code>
<code>.toUpperCase()</code>	a mayúsculas	<code>"HOLA"</code>
<code>.toLowerCase()</code>	a minúsculas	<code>"hola"</code>
<code>.trim()</code>	saca espacios de los extremos	<code>" a ".trim()</code> es <code>"a"</code>
<code>.includes(t)</code>	si contiene el texto	<code>true</code> o <code>false</code>
<code>.startsWith(t)</code>	si empieza con el texto	<code>true</code> o <code>false</code>
<code>.endsWith(t)</code>	si termina con el texto	<code>true</code> o <code>false</code>
<code>.indexOf(t)</code>	posición, o <code>-1</code> si no está	<code>"hola".indexOf("l")</code> es <code>2</code>
<code>.slice(a, b)</code>	porción, acepta negativos	<code>"hola".slice(1, 3)</code> es <code>"ol"</code>
<code>.split(sep)</code>	corta en un arreglo	<code>"a,b".split(",")</code> es <code>["a","b"]</code>
<code>.replace(a, b)</code>	reemplaza la primera aparición	
<code>.replaceAll(a, b)</code>	reemplaza todas	
<code>.repeat(n)</code>	repite la cadena	<code>"ab".repeat(2)</code> es <code>"abab"</code>
<code>.padStart(n, c)</code>	rellena al principio	<code>"5".padStart(3, "0")</code> es <code>"005"</code>
<code>.padEnd(n, c)</code>	rellena al final	

Un caso de uso que vamos a ver la clase que viene, leyendo un CSV:

```
const linea = " Ana Pérez , ana@mail.com , 3814567890 ";

const campos = linea
  .split(",")
  .map((campo) => campo.trim());

// ["Ana Pérez", "ana@mail.com", "3814567890"]
```

Y una advertencia sobre `.length`. Cuenta unidades de 16 bits, no caracteres percibidos. Con texto latino común no vas a notar diferencia, pero con emojis sí:

```
"hola".length; // 4
"😄".length;    // 2
```

Conversión de tipos

Llegamos a la parte que explica la mitad de los errores raros de JavaScript. Hay dos formas de convertir: la que pedís vos y la que hace el lenguaje solo.

Conversión explícita

Es la que controlás. Se hace con funciones y es la que deberías preferir siempre.

A número:

```
Number("42");      // 42
Number("3.14");     // 3.14
Number("");        // 0      ← ojo con este
Number(" ");       // 0      ← y con este
Number("42px");     // NaN
Number(true);       // 1
Number(false);      // 0
Number(null);       // 0      ← ojo
Number(undefined); // NaN
Number([]);         // 0
Number([5]);        // 5
Number([1, 2]);     // NaN
Number({});         // NaN
```

`parseInt` y `parseFloat` son distintos de `Number`. Leen desde el principio hasta donde entienden y descartan el resto:

```
parseInt("42px");    // 42
Number("42px");     // NaN

parseInt("3.9");     // 3, corta en el punto
parseFloat("3.9kg"); // 3.9

parseInt("");        // NaN
Number("");          // 0

parseInt("ff", 16);  // 255, el segundo argumento es la base
```

Usá `Number()` cuando el texto tiene que ser un número completo, y `parseInt()` cuando esperás un número seguido de otra cosa, como `"20px"`.

A texto:

```
String(42);           // "42"
String(true);         // "true"
String(null);         // "null"
String(undefined);    // "undefined"
String([1, 2]);       // "1,2"
String({});           // "[object Object]"

(42).toString();      // "42"
`${42}`;              // "42"
```

Fijate que un objeto se convierte en `"[object Object]"`. Cuando veas eso en pantalla, ya sabés qué pasó: intentaste mostrar un objeto como texto. Para eso está `JSON.stringify()`.

A booleano, con la lista de falsy que ya vimos:

```
Boolean("hola"); // true
Boolean("");     // false
Boolean(0);      // false
!!"hola";       // true
```

Conversión automática

Es la que hace el motor cuando encuentra tipos que no coinciden. Como JavaScript es débilmente tipado, casi nunca falla: convierte y sigue.

La lógica no es azarosa. El motor mira qué operación estás haciendo y decide a qué tipo convertir.

Con el operador `+` la regla es especial, porque sirve para dos cosas. Si alguno de los dos operandos es una cadena, convierte el otro a cadena y concatena. Si no, convierte los dos a número y suma.

```
"5" + 3;           // "53"    una cadena presente: concatena
"5" + true;        // "5true"
1 + 2 + "3";       // "33"    se evalúa de izquierda a derecha: (1+2) + "3"
"1" + 2 + 3;       // "123"   acá la cadena está primero
```

Con el resto de los operadores aritméticos no hay ambigüedad, así que todo va a número:

```
"5" - 3;    // 2
"5" * "2";  // 10
"10" / "2"; // 5
true + 1;   // 2
null + 1;   // 1
undefined + 1; // NaN
```

Con los operadores de orden, si los dos operandos son cadenas compara texto; si no, convierte a número:

```
"10" < "9";    // true, compara como texto
"10" < 9;       // false, convierte a número
```

En un contexto booleano, o sea dentro de un `if`, un `while`, un `!`, un `&&` o un `||`, se aplica la lista de falsy.

Y dentro de `${}` de una plantilla, todo se convierte a texto.

El caso de la igualdad débil

El operador `==` tiene sus propias reglas, y son las más enredadas del lenguaje.

Si los tipos coinciden, se comporta como `===`. Si no, convierte según esta tabla:

- `null == undefined` es `true`, y ninguno de los dos es igual a nada más
- número contra cadena: la cadena se convierte a número
- booleano contra cualquier cosa: el booleano se convierte a número
- objeto contra primitivo: el objeto se convierte a primitivo

De ahí salen resultados como estos:

```
1 == "1";        // true
0 == false;      // true
0 == "";         // true
"" == false;     // true
null == 0;       // false, null solo se compara con undefined
null == undefined; // true
[] == false;     // true, [] → "" → 0, y false → 0
[1] == 1;        // true
```

Y una contradicción aparente que vale la pena entender, porque muestra que son dos algoritmos distintos:

```
null == 0;    // false, por la regla especial de ==
null >= 0;    // true, los relacionales convierten: Number(null) es 0
```

No hay que memorizar esta tabla. Hay que sacar de acá una sola conclusión: `==` no es confiable, y `===` no tiene ninguna de estas reglas. Usá `===` siempre.

Las reglas que te llevás

- convertí explícitamente cuando cruces un límite: lo que viene de un formulario, de un archivo o de una API siempre llega como texto
- usá `===` y `!==`
- usá `??` para valores por defecto, no `||`
- desconfiá del `+` cuando alguno de los operandos podría ser texto
- si un cálculo te da `NaN`, buscá hacia atrás cuál fue el primer valor que no era número
- si en pantalla aparece `[object Object]`, convertiste un objeto a texto sin querer

Un caso completo, del tipo que vas a encontrar en la práctica:

```
// Lo que devuelve un formulario HTML siempre es texto
const cantidad = document.querySelector("#cantidad").value; // "3"
const precio = document.querySelector("#precio").value;      // "100"

const totalMal = cantidad * precio; // 300, funciona de casualidad
const totalPeor = cantidad + precio; // "3100", concatena

// Lo correcto
const totalBien = Number(cantidad) * Number(precio); // 300
```

Para llevarte de esta clase

- una variable es un nombre que apunta a un valor; el tipo lo tiene el valor, no el nombre
- usá `const` por defecto, `let` cuando necesites reasignar, nunca `var`
- `const` congela la referencia, no el contenido del objeto
- `let` y `const` respetan el bloque y fallan si las usás antes de declararlas
- los primitivos se copian por valor y los objetos por referencia
- hay ocho falsy: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined` y `NaN`; todo lo demás es truthy, incluidos `[]` y `{}`
- `&&` y `||` devuelven un operando, no un booleano, y cortan la evaluación
- `??` cubre solo `null` y `undefined`; `||` cubre todos los falsy
- hay un solo tipo numérico, de punto flotante: no lo uses para plata

- los operadores de bits trabajan sobre 32 bits con signo, no sobre los 64 del número
- las comillas invertidas te dan interpolación y texto de varias líneas
- las cadenas son inmutables: sus métodos devuelven una cadena nueva
- convertí explícitamente y usé `===`

Para probar antes de la próxima clase

1. Declará una constante con un objeto que tenga dos propiedades. Modificá una propiedad y después intentá reasignar la constante. Explicá por qué una operación funciona y la otra no.
2. Escribí un `for` con `var` que guarde tres funciones en un arreglo, cada una imprimiendo el índice. Ejecutalas y observá qué imprimen. Repetí el ejercicio con `let` y explicá la diferencia con lo que sabés de alcance.
3. Evaluá `[] == false`, `[] == 0` y `Boolean([])`. Los tres resultados juntos parecen contradictorios; resolvé la contradicción siguiendo la regla que aplica cada operador.
4. Escribí una función que reciba un precio y devuelva el texto formateado como pesos argentinos con dos decimales.
5. Tomá la cadena `" ana perez ; ANA@MAIL.COM ; 381-456-7890 "` y sacá de ahí un objeto con las propiedades `nombre`, `email` y `telefono`, con el nombre en mayúscula inicial, el email en minúsculas y el teléfono sin guiones.
6. Calculá `0.1 + 0.2 === 0.3` y después escribí una función `sonIguales(a, b)` que compare dos decimales de forma confiable.

Tipos compuestos: arreglos y objetos

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte de la tercera clase de JavaScript

Hasta acá trabajamos con valores sueltos: un número, un texto, un booleano. Un programa real no maneja un contacto, maneja la agenda entera. Necesitamos agrupar.

Hay dos formas de agrupar datos, y la diferencia entre ellas es la pregunta que cada una responde.

Si los elementos son todos lo mismo y lo que importa es el orden, los guardás por posición. Eso es un arreglo, y la pregunta que responde es "cuál es el tercero".

Si los elementos son partes distintas de una misma cosa, los guardás por nombre. Eso es un objeto, y la pregunta que responde es "cuál es el email".

En la práctica se combinan todo el tiempo. Una agenda es un arreglo de objetos, y cada objeto puede tener adentro otro arreglo. Al final de la clase vamos a sumar dos estructuras más, `Set` y `Map`, que resuelven casos donde el arreglo y el objeto quedan cortos.

Arreglos

Qué es un arreglo en JavaScript

Un arreglo es una colección ordenada de valores, numerados desde cero.

Si venís de C#, hay dos diferencias que cambian la forma de trabajar. El arreglo de JavaScript no tiene tamaño fijo: crece y se achica mientras el programa corre. Y no tiene un tipo de elemento: podés mezclar lo que quieras.

```
const mezcla = [1, "dos", true, null, { a: 1 }, [2, 3]];
```

Que puedas mezclar no significa que convenga. Un arreglo con elementos de un solo tipo se lee mejor, y además el motor lo ejecuta más rápido, por lo que vimos en la primera clase sobre las suposiciones que hace el compilador optimizador.

Por dentro, un arreglo es un objeto cuyas claves son números. Eso explica que `typeof [1, 2]` devuelva `"object"`, y que para distinguirlo haya un método aparte:


```
Array.isArray([1, 2]); // true
Array.isArray("hola"); // false
```

Cómo se declara

La forma normal es el literal, con corchetes:

```
const vacio = [];
const numeros = [1, 2, 3];
const nombres = ["Ana", "Luis", "Sol"];
```

Hay otras formas, que sirven para casos puntuales:

```
new Array(5);           // arreglo de largo 5, con 5 huecos vacíos
new Array(5).fill(0);    // [0, 0, 0, 0, 0]
Array.of(5);             // [5], un solo elemento
Array.from("hola");      // ["h", "o", "l", "a"]
Array.from({ length: 5 }, (_, i) => i); // [0, 1, 2, 3, 4]
Array.from({ length: 5 }, (_, i) => i * 10); // [0, 10, 20, 30, 40]
```

`new Array(5)` merece una advertencia. No crea cinco ceros: crea cinco huecos, que se comportan distinto de `undefined` en varios métodos. Si necesitas un arreglo iniciado, agregale siempre `.fill()`.

Notá también que declaramos con `const` y aun así vamos a agregar elementos. Es lo que vimos la clase pasada: `const` congela la referencia, no el contenido.

```
const numeros = [1, 2, 3];
numeros.push(4); // válido
numeros = [];    // TypeError
```

Cómo se accede a los elementos

Con corchetes y el índice, que empieza en cero:

```
const nombres = ["Ana", "Luis", "Sol"];

nombres[0];           // "Ana"
nombres[2];           // "Sol"
nombres.length;       // 3
nombres[nombres.length - 1]; // "Sol", el último
nombres.at(-1);       // "Sol", más corto
nombres.at(-2);       // "Luis"
```

`at()` acepta índices negativos y cuenta desde el final. Es más legible que la resta.

Un índice fuera de rango no lanza error: devuelve `undefined`.

```
nombres[99]; // undefined
```

Esa es una diferencia grande con C#, donde tendrías una excepción. Acá el programa sigue con un `undefined` adentro, y el error aparece más adelante, en otro lugar. Es la misma lógica del tipado débil que vimos en la primera clase: JavaScript prefiere seguir antes que detenerse.

Escribir fuera de rango tampoco falla. Extiende el arreglo y deja huecos en el medio:

```
const a = [1, 2, 3];  
a[6] = 7;  
console.log(a);           // [1, 2, 3, <3 huecos>, 7]  
console.log(a.length);    // 7
```

Y `length` se puede escribir, cosa poco común:

```
const b = [1, 2, 3, 4, 5];  
b.length = 2;  
console.log(b); // [1, 2]
```

Truncar con `length` funciona, pero es poco claro para quien lee. Preferí `b.splice(2)` o directamente crear otro arreglo.

Arreglos de arreglos y matrices

Un elemento de un arreglo puede ser cualquier valor, incluido otro arreglo. Con eso alcanza para representar una matriz.

```
const matriz = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9],  
];  
  
matriz[0];           // [1, 2, 3], la primera fila  
matriz[1][2];        // 6, fila 1, columna 2  
matriz.length;       // 3, cantidad de filas  
matriz[0].length;    // 3, cantidad de columnas de la primera fila
```

JavaScript no tiene matrices verdaderas como el `int[,]` de C#. Lo que tiene son arreglos de arreglos, y eso implica que las filas son independientes: nada garantiza que todas midan lo mismo.

Para crear una matriz vacía de un tamaño dado hay una trampa que cae todos los años:

```
// Mal: las tres filas son el mismo arreglo
const mala = new Array(3).fill([0, 0, 0]);
mala[0][0] = 99;
console.log(mala); // [[99,0,0], [99,0,0], [99,0,0]]
```

`fill` guarda el mismo valor en las tres posiciones, y ese valor es una referencia. Las tres filas apuntan al mismo arreglo. La forma correcta crea una fila nueva por vez:

```
const buena = Array.from({ length: 3 }, () => new Array(3).fill(0));
buena[0][0] = 99;
console.log(buena); // [[99,0,0], [0,0,0], [0,0,0]]
```

Guardá este ejemplo, porque es la primera aparición de un problema que va a volver varias veces hoy: copiar un contenedor no copia lo que hay adentro.

Para recorrer una matriz, dos bucles anidados:

```
for (let fila = 0; fila < matriz.length; fila++) {
  for (let col = 0; col < matriz[fila].length; col++) {
    console.log(`${fila}[${col}] = ${matriz[fila][col]}`);
  }
}
```

Rangos y porciones

Para sacar un tramo del arreglo está `slice`. Recibe la posición inicial y la final, y la final no se incluye.

```
const letras = ["a", "b", "c", "d", "e"];

letras.slice(1, 3); // ["b", "c"]
letras.slice(2);    // ["c", "d", "e"], desde la 2 hasta el final
letras.slice(-2);   // ["d", "e"], las dos últimas
letras.slice();     // copia del arreglo completo
```

`slice` no toca el arreglo original: devuelve uno nuevo.

Su primo `splice` hace lo contrario: modifica el arreglo y devuelve lo que sacó. Sirve para eliminar e insertar en el medio.

```
const dias = ["lun", "mar", "mié", "jue"];

dias.splice(1, 2);           // devuelve ["mar", "mié"]
console.log(dias);           // ["lun", "jue"], quedó modificado

dias.splice(1, 0, "mar", "mié"); // inserta sin eliminar nada
console.log(dias);           // ["lun", "mar", "mié", "jue"]
```

Los nombres se parecen y hacen cosas opuestas. Una forma de recordarlo: `slice` corta una rebanada y te la lleva, `splice` opera sobre el original.

Para buscar dentro del arreglo:

```
const nombres = ["Ana", "Luis", "Sol"];

nombres.indexOf("Luis");    // 1
nombres.indexOf("Pedro");   // -1, no está
nombres.includes("Sol");    // true
nombres.lastIndexOf("Ana"); // 0, busca desde el final
```

`indexOf` compara con `===`, así que sirve para primitivos pero no para buscar objetos por contenido. Para eso están `find` y `findIndex`, que vemos enseguida.

Para unir arreglos:

```
const a = [1, 2];
const b = [3, 4];

a.concat(b); // [1, 2, 3, 4]
[...a, ...b]; // [1, 2, 3, 4], la forma moderna
[0, ...a, 5]; // [0, 1, 2, 5], podés intercalar
```

Los tres puntos son el operador de propagación. Desarma el arreglo en sus elementos sueltos. Lo vas a usar mucho hoy.

Los tres for

Antes de seguir con las operaciones sobre arreglos, hay que ver cómo se recorren. JavaScript tiene tres bucles `for` y elegir mal es una fuente clásica de errores.

El for tradicional

Es el de C, con contador propio:

```
const nombres = ["Ana", "Luis", "Sol"];

for (let i = 0; i < nombres.length; i++) {
  console.log(i, nombres[i]);
}
```

Te da control total: podés ir de a dos, ir para atrás, empezar en el medio, saltar. Usalo cuando el índice sea parte del problema.

```
for (let i = nombres.length - 1; i >= 0; i--) { /* al revés */ }
for (let i = 0; i < nombres.length; i += 2) { /* de a dos */ }
```

Declarar el contador con `let` y no con `var` importa. Con `let` cada vuelta tiene su propia copia de `i`, y eso cambia el comportamiento cuando adentro del bucle creás funciones. Es el ejercicio que quedó de la clase pasada.

El for of

Recorre los valores, sin índice. Es el que vas a usar casi siempre:

```
for (const nombre of nombres) {
  console.log(nombre);
}
```

Se lee mejor, no te podés equivocar con los límites y no hay riesgo de salirte del rango. Si necesitás el índice, pedilo con `entries()`:

```
for (const [i, nombre] of nombres.entries()) {
  console.log(`${i + 1}. ${nombre}`);
}
```

Ese `[i, nombre]` es desestructuración, y la vemos en detalle más abajo.

`for...of` funciona sobre cualquier cosa iterable: arreglos, cadenas, `Set`, `Map` y los resultados de `Object.entries()`. No funciona sobre objetos comunes.

```
for (const letra of "hola") {
  console.log(letra); // h, o, l, a
}
```

Admite `break` y `continue`, así que podés cortar antes de terminar.

El for in

Recorre las claves de un objeto. Este es el que hay que usar con cuidado.

```
const contacto = { nombre: "Ana", email: "ana@mail.com" };

for (const clave in contacto) {
  console.log(clave, contacto[clave]);
}
// nombre ana@mail.com... no: nombre Ana, email ana@mail.com
```

Sobre un arreglo también funciona, pero está mal usarlo ahí, por tres motivos.

Devuelve las claves como texto, no como número:

```
const numeros = [10, 20, 30];

for (const i in numeros) {
  console.log(i, typeof i); // "0" string, "1" string, "2" string
  console.log(i + 1);      // "01", "11", "21", concatena
}
```

Recorre cualquier propiedad que alguien le haya agregado al arreglo, no solo los elementos. Y salta los huecos de los arreglos incompletos.

La regla es corta: `for...in` para objetos, `for...of` para arreglos. Si te acordás de una sola cosa de esta sección, que sea esa.

Cuál usar

Bucle	Recorre	Usalo para
<code>for</code> clásico	índices que controlás vos	saltear, ir al revés, cortar por índice
<code>for...of</code>	valores	recorrer arreglos, cadenas, <code>Set</code> y <code>Map</code>
<code>for...in</code>	claves	recorrer las propiedades de un objeto
<code>.forEach()</code>	valores, con función	transformar cada elemento sin cortar

`forEach` tiene una limitación que conviene saber: no acepta `break`. Si necesitás cortar antes, usá `for...of`.

```
nombres.forEach((nombre, indice) => {
  console.log(indice, nombre);
});
```

Operaciones clásicas sobre arreglos

Acá aparece la forma de trabajar propia de JavaScript. En vez de escribir un bucle y acumular a mano, describís la transformación y el lenguaje la aplica.

map, transformar cada elemento

Devuelve un arreglo nuevo, del mismo largo, con cada elemento transformado.

```
const precios = [100, 200, 300];

const conIva = precios.map((precio) => precio * 1.21);
// [121, 242, 363]

const contactos = [
  { nombre: "Ana", email: "ana@mail.com" },
  { nombre: "Luis", email: "luis@mail.com" },
];

const soloNombres = contactos.map((c) => c.nombre);
// ["Ana", "Luis"]
```

filter, quedarse con algunos

Devuelve un arreglo nuevo, más corto o igual, con los elementos que cumplen la condición.

```
const numeros = [1, 2, 3, 4, 5, 6];

numeros.filter((n) => n % 2 === 0); // [2, 4, 6]

contactos.filter((c) => c.email.endsWith("@mail.com"));
```

reduce, obtener un solo valor

Es el más general de los tres y el que más cuesta al principio, así que vale explicarlo desde el principio.

`reduce` recorre el arreglo llevando un acumulador. En cada vuelta le pasás el acumulador y el elemento actual, y devolvés el acumulador nuevo. Al final te queda ese acumulador.

```
const precios = [100, 200, 300];

const total = precios.reduce((acumulado, precio) => acumulado + precio, 0);
// 600
```

El `0` del final es el valor inicial del acumulador. Paso a paso: empieza en 0, suma 100 y queda 100, suma 200 y queda 300, suma 300 y queda 600.

El acumulador no tiene que ser un número. Puede ser cualquier cosa, y ahí `reduce` muestra su alcance:

```
// Contar apariciones
const palabras = ["a", "b", "a", "c", "a"];

const conteo = palabras.reduce((acc, palabra) => {
  acc[palabra] = (acc[palabra] ?? 0) + 1;
  return acc;
}, {});
// { a: 3, b: 1, c: 1 }

// Agrupar por una propiedad
const porCiudad = contactos.reduce((acc, c) => {
  (acc[c.ciudad] ??= []).push(c);
  return acc;
}, {});
```

Si el acumulador te queda difícil de seguir, escribí un `for...of`. Un `reduce` ilegible no es mejor que un bucle claro.

Búsqueda y verificación

```
const contactos = [
  { id: 1, nombre: "Ana", activo: true },
  { id: 2, nombre: "Luis", activo: false },
];

contactos.find((c) => c.id === 2);           // el objeto de Luis
contactos.findIndex((c) => c.id === 2);      // 1
contactos.some((c) => c.activo);             // true, hay al menos uno
contactos.every((c) => c.activo);            // false, no todos
```

`find` devuelve `undefined` si no encuentra nada, así que conviene chequear antes de usar el resultado.

Ordenar

`sort` tiene dos particularidades que hay que conocer.

La primera es que modifica el arreglo original.

La segunda es que, sin argumentos, convierte todo a texto y ordena alfabéticamente. Con números eso da resultados equivocados:

```
const numeros = [10, 9, 1, 100];

numeros.sort();           // [1, 10, 100, 9] ← ordenó como texto
numeros.sort((a, b) => a - b); // [1, 9, 10, 100]
numeros.sort((a, b) => b - a); // [100, 10, 9, 1]
```

La función que le pasás recibe dos elementos y devuelve un número: negativo si el primero va antes, positivo si va después, cero si da igual.

Para texto en español, usá `localeCompare`, como vimos con las cadenas:

```
contactos.sort((a, b) => a.nombre.localeCompare(b.nombre, "es"));
```

Otras funciones habituales

```
[1, 2, 3].join(" - ");           // "1 - 2 - 3"
[1, 2, 3].reverse();             // [3, 2, 1], modifica el original
[[1, 2], [3, [4]]].flat();       // [1, 2, 3, [4]], un nivel
[[1, 2], [3, [4]]].flat(2);      // [1, 2, 3, 4]
["a b", "c d"].flatMap((s) => s.split(" ")); // ["a","b","c","d"]
```

Se encadenan

La virtud de estos métodos es que cada uno devuelve un arreglo, así que se enganchan uno detrás de otro:

```
const totalActivos = contactos
  .filter((c) => c.activo)
  .map((c) => c.saldo)
  .reduce((total, saldo) => total + saldo, 0);
```

Se lee de arriba abajo como una frase: filtrar los activos, quedarse con el saldo, sumar. Comparalo con la versión con bucle y vas a ver por qué esta forma se impuso.

Mutar o no mutar

Los métodos de arreglo se dividen en dos grupos, y confundirlos causa errores difíciles de encontrar.

Modifican el original	Devuelven uno nuevo
push, pop	map, filter, reduce
shift, unshift	slice, concat
splice	toSpliced
sort	toSorted
reverse	toReversed
fill	with

Los cuatro de la derecha con nombre en infinitivo (`toSorted`, `toReversed`, `toSpliced`, `with`) son recientes, de 2023. Hacen lo mismo que su par de la izquierda pero sin tocar el original.

```
const original = [3, 1, 2];

const ordenadoMal = original.sort();    // modificó original
const ordenadoBien = original.toSorted(); // original queda igual
```

Cómo se hacen las operaciones en forma inmutable

Trabajar sin modificar el original tiene ventajas concretas. Si nadie modifica un arreglo, nadie puede romperlo desde otra parte del programa. Comparar si algo cambió se reduce a comparar dos referencias. Y React, que vamos a ver más adelante, directamente depende de esto para saber cuándo redibujar la pantalla.

Estas son las cuatro operaciones básicas en las dos formas:

```

const contactos = [{ id: 1, nombre: "Ana" }, { id: 2, nombre: "Luis" }];

// Agregar
contactos.push(nuevo); // muta
const conNuevo = [...contactos, nuevo]; // inmutable
const alPrincipio = [nuevo, ...contactos]; // inmutable

// Eliminar
contactos.splice(1, 1); // muta
const sinLuis = contactos.filter((c) => c.id !== 2); // inmutable

// Actualizar uno
contactos[0].nombre = "Ana María"; // muta
const actualizados = contactos.map((c) =>
  c.id === 1 ? { ...c, nombre: "Ana María" } : c
); // inmutable

// Reemplazar por posición
const reemplazado = contactos.with(0, otroContacto); // inmutable

```

Fijate el patrón de la actualización: `map` recorre todo, y para el elemento que querés cambiar devolvés un objeto nuevo con las propiedades del viejo más el cambio. Para los demás devolvés el mismo objeto. Ese patrón lo vas a escribir cien veces en esta materia.

Referencia y copia

Retomamos algo de la clase pasada, porque con arreglos se vuelve más grave.

Asignar un arreglo a otra variable no lo copia. Copia la flecha:

```

const a = [1, 2, 3];
const b = a;

b.push(4);
console.log(a); // [1, 2, 3, 4], cambió también
console.log(a === b); // true, es el mismo arreglo

```

Para copiar de verdad hay tres formas equivalentes:

```

const copia1 = [...a];
const copia2 = a.slice();
const copia3 = Array.from(a);

copia1.push(99);
console.log(a); // sin cambios

```

Pero atención, porque acá aparece otra vez el problema de la matriz. Estas tres formas hacen una copia superficial: copian el arreglo de afuera, no lo que hay adentro.

```
const contactos = [{ nombre: "Ana" }, { nombre: "Luis" }];
const copia = [...contactos];

copia.push({ nombre: "Sol" });
console.log(contactos.length); // 2, el arreglo externo es otro

copia[0].nombre = "Ana María";
console.log(contactos[0].nombre); // "Ana María", el objeto es el mismo
```

El arreglo se duplicó. Los objetos de adentro, no: las dos listas apuntan a los mismos objetos.

Para una copia profunda, donde nada se comparta, está `structuredClone`:

```
const copiaProfunda = structuredClone(contactos);
copiaProfunda[0].nombre = "Ana María";
console.log(contactos[0].nombre); // "Ana", intacto
```

Vas a ver también este truco en código viejo:

```
const copia = JSON.parse(JSON.stringify(contactos));
```

Funciona, pero pierde por el camino las fechas, las funciones, los `undefined` y falla con referencias circulares. Usá `structuredClone`.

La comparación sigue la misma lógica. `===` sobre arreglos compara referencias, no contenido:

```
[1, 2] === [1, 2]; // false, son dos arreglos distintos
```

Para comparar contenido tenés que recorrer, o comparar sus representaciones en texto si el caso es simple.

Desestructuración de arreglos

Desestructurar es sacar valores de una estructura y ponerlos en variables sueltas, en una sola línea.

```
const coordenadas = [10, 20, 30];

// Sin desestructurar
const x1 = coordenadas[0];
const y1 = coordenadas[1];

// Desestructurando
const [x, y, z] = coordenadas;
```

Las variables se asignan por posición, no por nombre. Podés saltar posiciones dejando el lugar vacío:

```
const [, , tercero] = coordenadas; // 30
```

Podés juntar el resto en un arreglo con los tres puntos:

```
const [primero, ...resto] = [1, 2, 3, 4, 5];
// primero = 1, resto = [2, 3, 4, 5]
```

Podés poner valores por defecto para cuando falte un elemento:

```
const [a = 0, b = 0, c = 0] = [10, 20];
// a = 10, b = 20, c = 0
```

Funciona con estructuras anidadas, lo que sirve para matrices:

```
const matriz = [[1, 2], [3, 4]];
const [[a1, a2], [b1, b2]] = matriz;
// a1=1, a2=2, b1=3, b2=4
```

Sirve para intercambiar dos variables sin una tercera:

```
let x = 1;
let y = 2;
[x, y] = [y, x];
// x = 2, y = 1
```

Y aparece constantemente en los bucles y en los parámetros de funciones:

```
for (const [indice, nombre] of nombres.entries()) { /* ... */ }

function distancia([x1, y1], [x2, y2]) {
  return Math.hypot(x2 - x1, y2 - y1);
}

distancia([0, 0], [3, 4]); // 5
```

Objetos

Qué es un objeto

Un objeto agrupa valores y les pone nombre. Cada par de nombre y valor es una propiedad.

```
const contacto = {
  nombre: "Ana Pérez",
  email: "ana@mail.com",
  edad: 30,
  activo: true,
};
```

El nombre de la propiedad, o clave, siempre es texto. El valor puede ser cualquier cosa, incluidos otros objetos, arreglos y funciones.

Una aclaración para quien viene de C#: esto no es una instancia de una clase. Es un objeto suelto, creado en el momento, sin ninguna definición previa. JavaScript también tiene clases, pero la mayoría de los objetos que vas a manejar en desarrollo web nacen así, como literales o como resultado de leer un JSON.

Los objetos se anidan sin límite:

```
const contacto = {
  nombre: "Ana",
  direccion: {
    calle: "Rivadavia 123",
    ciudad: "Tucumán",
  },
  telefonos: ["3814567890", "3811234567"],
};
```

El punto y los corchetes

Hay dos formas de acceder a una propiedad.

La sintaxis de punto es la habitual:

```
contacto.nombre;           // "Ana"
contacto.direccion.ciudad; // "Tucumán"
contacto.telefonos[0];     // "3814567890"
```

La sintaxis de corchetes usa la clave como texto:

```
contacto["nombre"]; // "Ana"
```

Con una clave fija son equivalentes y siempre se prefiere el punto. Pero hay tres casos en que los corchetes son obligatorios.

Cuando la clave no es un identificador válido:

```
const config = {
  "api-key": "abc123",
  "max intentos": 3,
};

config["api-key"]; // única forma
config.api-key;    // error de sintaxis, lo lee como una resta
```

Cuando la clave está en una variable:

```
const campo = "email";

contacto[campo]; // "ana@mail.com"
contacto.campo;  // undefined, busca una propiedad llamada "campo"
```

Y cuando la clave se calcula en el momento:

```
for (const campo of ["nombre", "email"]) {
  console.log(`${campo}: ${contacto[campo]}`);
}
```

Esa diferencia entre `contacto[campo]` y `contacto.campo` es el error más común del tema. El punto toma lo que escribiste literalmente; los corchetes evalúan la expresión.

Agregar, modificar y eliminar

Los objetos son abiertos: agregás propiedades cuando querés.

```
const contacto = { nombre: "Ana" };

contacto.email = "ana@mail.com"; // agrega
contacto.nombre = "Ana María";   // modifica
delete contacto.email;           // elimina
```

Pedir una propiedad que no existe devuelve `undefined`, sin error:

```
contacto.telefono; // undefined
```

Y pedir una propiedad de algo que es `undefined` sí falla:

```
contacto.direccion.ciudad; // TypeError si no hay direccion
contacto.direccion?.ciudad; // undefined, sin fallar
```

Para preguntar si una propiedad existe:

```
"nombre" in contacto;           // true
Object.hasOwn(contacto, "nombre"); // true, solo propiedades propias
contacto.nombre !== undefined;   // sirve, salvo que valga undefined
```

Las formas abreviadas

Hay tres atajos de sintaxis que aparecen en todo el código moderno.

El primero es la abreviatura de propiedad. Cuando la clave y la variable se llaman igual, escribís el nombre una sola vez:

```
const nombre = "Ana";
const email = "ana@mail.com";

// Forma larga
const contacto = { nombre: nombre, email: email };

// Forma abreviada
const contacto = { nombre, email };
```

Es exactamente lo mismo. Lo vas a ver a la salida de casi toda función que arme un objeto.

El segundo es la abreviatura de método. Si el valor es una función, podés omitir `function` y los dos puntos:


```
// Forma larga
const agenda = {
  contactos: [],
  agregar: function (c) { this.contactos.push(c); },
};

// Forma abreviada
const agenda = {
  contactos: [],
  agregar(c) { this.contactos.push(c); },
};
```

El tercero son las claves calculadas. Con corchetes dentro del literal, la clave sale de una expresión:

```
const campo = "email";
const valor = "ana@mail.com";

const objeto = { [campo]: valor };
// { email: "ana@mail.com" }

// Útil para construir objetos dinámicos
function crearFiltro(campo, valor) {
  return { [campo]: valor, [`${campo}Exacto`]: true };
}
```

Y el operador de propagación, que ya usamos con arreglos, también funciona con objetos:

```
const base = { nombre: "Ana", activo: true };

const copia = { ...base };
const conEmail = { ...base, email: "ana@mail.com" };
const modificado = { ...base, nombre: "Ana María" };
```

El orden importa: lo que va después pisa lo que va antes.

```
{ ...base, activo: false }; // activo queda en false
{ activo: false, ...base }; // activo vuelve a true
```

Combinando dos objetos:

```
const porDefecto = { pagina: 1, orden: "nombre", limite: 10 };
const delUsuario = { limite: 50 };

const opciones = { ...porDefecto, ...delUsuario };
// { pagina: 1, orden: "nombre", limite: 50 }
```

Ese patrón de valores por defecto más valores del usuario lo vas a usar en cada función de configuración que escribas.

Desestructuración de objetos

Igual que con arreglos, pero por nombre en vez de por posición.

```
const contacto = { nombre: "Ana", email: "ana@mail.com", edad: 30 };

const { nombre, email } = contacto;
// nombre = "Ana", email = "ana@mail.com"
```

El orden no importa y podés tomar solo lo que necesitás.

Para usar otro nombre de variable, con dos puntos:

```
const { nombre: nombreContacto, email: correo } = contacto;
```

Valores por defecto para lo que falte:

```
const { nombre, telefono = "sin teléfono" } = contacto;
```

Se puede combinar todo:

```
const { nombre: n = "Anónimo", ciudad = "Tucumán" } = contacto;
```

Anidada, para llegar a propiedades internas:

```
const contacto = {
  nombre: "Ana",
  direccion: { calle: "Rivadavia 123", ciudad: "Tucumán" },
};

const { direccion: { ciudad } } = contacto;
// ciudad = "Tucumán", y direccion no queda como variable
```

Y el resto, que junta lo que no nombraste:

```
const { nombre, ...otrosDatos } = contacto;  
// nombre = "Ana"  
// otrosDatos = { email: ..., edad: ... }
```

Ese último uso sirve para sacar una propiedad de un objeto sin modificarlo:

```
const { password, ...contactoPublico } = usuario;  
// contactoPublico tiene todo menos password
```

El lugar donde más se usa la desestructuración es en los parámetros de las funciones:

```
// Sin desestructurar: hay que recordar el orden  
function crearContacto(nombre, email, edad, activo) { /* ... */ }  
crearContacto("Ana", "ana@mail.com", 30, true);  
  
// Desestructurando: los argumentos se nombran  
function crearContacto({ nombre, email, edad = 0, activo = true }) {  
  return { nombre, email, edad, activo };  
}  
  
crearContacto({ email: "ana@mail.com", nombre: "Ana" });
```

La segunda versión no depende del orden, documenta qué recibe y permite valores por defecto. Es la forma habitual cuando una función tiene más de tres parámetros.

Recorrer un objeto

Un objeto no es iterable, así que `for...of` no funciona directamente sobre él. Hay dos caminos.

El primero es `for...in`, que ya vimos:

```
for (const clave in contacto) {  
  console.log(clave, contacto[clave]);  
}
```

El segundo, más recomendable, es convertir el objeto en un arreglo y recorrerlo con las herramientas que ya conocés:

```
Object.keys(contacto);    // ["nombre", "email", "edad"]  
Object.values(contacto);  // ["Ana", "ana@mail.com", 30]  
Object.entries(contacto); // [["nombre","Ana"], ["email","ana@mail.com"], ...]
```

Con `entries` y desestructuración queda muy legible:

```
for (const [clave, valor] of Object.entries(contacto)) {  
  console.log(`${clave}: ${valor}`);  
}
```

Y como son arreglos, podés encadenar métodos:

```
// Quedarse con las propiedades que tienen valor  
const limpio = Object.fromEntries(  
  Object.entries(contacto).filter(([, valor]) => valor !== null)  
);
```

`Object.fromEntries` es el camino inverso: de arreglo de pares a objeto.

Otras funciones frecuentes

```
Object.assign({}, a, b); // combina, forma anterior al spread  
Object.freeze(contacto); // impide modificarlo  
JSON.stringify(contacto); // objeto a texto  
JSON.parse(texto); // texto a objeto
```

`JSON.stringify` es la forma de mostrar un objeto legible en consola, en vez del `[object Object]` que vimos la clase pasada:

```
console.log(JSON.stringify(contacto, null, 2)); // con sangría de 2
```

Y la comparación funciona como con los arreglos: `===` compara referencias.

```
{ a: 1 } === { a: 1 }; // false
```

Set

Un `Set` es una colección de valores sin repetidos y sin claves. Responde una sola pregunta: si un valor está o no está.

```
const etiquetas = new Set();

etiquetas.add("cliente");
etiquetas.add("activo");
etiquetas.add("cliente"); // ignorado, ya estaba

etiquetas.size; // 2
etiquetas.has("activo"); // true
etiquetas.delete("activo"); // true, lo eliminó
etiquetas.clear(); // vacía todo
```

Se puede crear desde un arreglo, y de ahí sale su uso más frecuente:

```
const numeros = [1, 2, 2, 3, 3, 3];

const sinRepetir = [...new Set(numeros)]; // [1, 2, 3]
```

Es iterable, así que se recorre con `for...of` y mantiene el orden en que agregaste los elementos:

```
for (const etiqueta of etiquetas) {
  console.log(etiqueta);
}
```

Dos detalles a tener en cuenta.

La comparación es por identidad, igual que `===`, con una excepción: `NaN` sí se considera igual a sí mismo dentro de un `Set`. Y como los objetos se comparan por referencia, dos objetos con el mismo contenido son dos elementos distintos:

```
const s = new Set();
s.add({ a: 1 });
s.add({ a: 1 });
s.size; // 2
```

Comparado con un arreglo, `Set` gana cuando la pregunta es la pertenencia. Buscar en un arreglo con `includes` recorre todos los elementos; en un `Set` es prácticamente instantáneo, sin importar el tamaño.

Map

Un `Map` guarda pares de clave y valor, como un objeto, pero con tres diferencias importantes.

```
const contactosPorId = new Map();

contactosPorId.set(1, { nombre: "Ana" });
contactosPorId.set(2, { nombre: "Luis" });

contactosPorId.get(1);    // { nombre: "Ana" }
contactosPorId.has(2);    // true
contactosPorId.delete(2); // true
contactosPorId.size;      // 1
contactosPorId.clear();
```

La primera diferencia es que la clave puede ser cualquier valor, no solo texto. Números, booleanos, objetos, funciones.

```
const m = new Map();
m.set(1, "número uno");
m.set("1", "texto uno");
m.get(1);    // "número uno", son claves distintas
```

En un objeto común, `obj[1]` y `obj["1"]` son la misma propiedad, porque las claves siempre se convierten a texto.

La segunda es que `size` te da la cantidad directamente, sin tener que pedir `Object.keys().length`.

La tercera es que es iterable y conserva el orden de inserción:

```
for (const [clave, valor] of contactosPorId) {
  console.log(clave, valor);
}

contactosPorId.keys();    // iterador de claves
contactosPorId.values();  // iterador de valores
contactosPorId.entries(); // iterador de pares
[...contactosPorId];      // [[1, {...}], [2, {...}]]
```

Se puede crear desde un arreglo de pares, lo que lo conecta con `Object.entries`:

```
const desdeObjeto = new Map(Object.entries({ a: 1, b: 2 }));
const deVueltaAObjeto = Object.fromEntries(desdeObjeto);
```

Un uso típico es indexar una lista para poder buscar rápido:

```
const contactos = [
  { id: 1, nombre: "Ana" },
  { id: 2, nombre: "Luis" },
];

const indice = new Map(contactos.map((c) => [c.id, c]));

indice.get(2); // acceso directo, sin recorrer el arreglo
```

Y contar frecuencias, la versión con `Map` del ejemplo de `reduce`:

```
const conteo = new Map();

for (const palabra of palabras) {
  conteo.set(palabra, (conteo.get(palabra) ?? 0) + 1);
}
```

Objeto o Map

	Objeto	Map
Claves	texto y símbolos	cualquier valor
Cantidad	<code>Object.keys().length</code>	<code>.size</code>
Orden	números primero, después inserción	inserción siempre
Iterable	no, hay que convertirlo	sí
Se convierte a JSON	sí	no directamente
Altas y bajas frecuentes	más lento	más rápido

La regla práctica: usá un objeto cuando represente una entidad con propiedades conocidas, como un contacto. Usá un `Map` cuando sea un diccionario con claves que cambian durante la ejecución.

Cuándo usar cada estructura

Estructura	Usala cuando
Arreglo	los elementos son del mismo tipo y el orden importa
Objeto	son propiedades distintas de una misma entidad
Set	solo te importa si un valor está, y no querés repetidos
Map	necesitás un diccionario con claves dinámicas o no textuales

Para llevarte de esta clase

- el arreglo agrupa por posición y el objeto por nombre
- un índice fuera de rango devuelve `undefined` en vez de fallar
- no hay matrices verdaderas: son arreglos de arreglos, y `fill` con un arreglo comparte la misma referencia en todas las filas
- `slice` devuelve una porción nueva, `splice` modifica el original
- `for...of` para arreglos, `for...in` para objetos, `for` clásico cuando el índice importa
- `map`, `filter` y `reduce` devuelven algo nuevo; `push`, `sort` y `reverse` modifican
- `sort` sin comparador ordena como texto
- asignar copia la referencia; `[...a]` copia el arreglo pero no los objetos de adentro; `structuredClone` copia todo
- la desestructuración toma por posición en arreglos y por nombre en objetos
- usá corchetes cuando la clave esté en una variable o se calcule
- `{ nombre }` es lo mismo que `{ nombre: nombre }`
- `Set` para valores únicos, `Map` para diccionarios con claves de cualquier tipo

Para probar antes de la próxima clase

1. Creá una matriz de 3 por 3 llena de ceros usando `new Array(3).fill([...])` y otra usando `Array.from`. Modificá el elemento `[0][0]` en las dos y explicá la diferencia.
2. Partiendo de un arreglo de contactos, escribí en una sola cadena de métodos el listado de los emails de los contactos activos, ordenados alfabéticamente.
3. Escribí dos versiones de "agregar un contacto a la agenda": una que modifique el arreglo y otra inmutable. Verificá con `===` que la segunda devuelve un arreglo distinto.
4. Copiá un arreglo de objetos con el operador de propagación, modificá una propiedad de un objeto de la copia y comprobá qué pasó con el original. Después repetilo con `structuredClone`.
5. Escribí una función que reciba un objeto de configuración con desestructuración y valores por defecto, y devuelva la configuración completa.
6. Dado un arreglo de contactos con una propiedad `ciudad`, armá un `Map` que asocie cada ciudad con el arreglo de contactos de esa ciudad. Después resolvélo de nuevo con `reduce` y un objeto, y comparalos.

Arreglos

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte de la tercera clase de JavaScript, 1 de 4

Hasta acá trabajamos con valores sueltos: un número, un texto, un booleano. Un programa real no maneja un contacto, maneja la agenda entera. Necesitamos agrupar.

Hay dos formas de agrupar, y la diferencia está en la pregunta que responde cada una. Si los elementos son todos lo mismo y lo que importa es el orden, los guardás por posición: eso es un arreglo, y responde "cuál es el tercero". Si son partes distintas de una misma cosa, los guardás por nombre: eso es un objeto, y responde "cuál es el email".

Este apunte cubre los arreglos. Los otros tres de esta clase son objetos, Set y Map.

Qué es un arreglo en JavaScript

Un arreglo es una colección ordenada de valores, numerados desde cero.

Si venís de C#, hay dos diferencias que cambian la forma de trabajar. El arreglo de JavaScript no tiene tamaño fijo: crece y se achica mientras el programa corre. Y no tiene un tipo de elemento: podés mezclar lo que quieras.

```
const mezcla = [1, "dos", true, null, { a: 1 }, [2, 3]];
```

Que puedas mezclar no significa que convenga. Un arreglo con elementos de un solo tipo se lee mejor, y además el motor lo ejecuta más rápido, por lo que vimos en la primera clase sobre las suposiciones que hace el compilador optimizador.

Por dentro, un arreglo es un objeto cuyas claves son números. Eso explica que `typeof [1, 2]` devuelva `"object"`, y que para distinguirlo haya un método aparte:

```
Array.isArray([1, 2]); // true
Array.isArray("hola"); // false
Array.isArray({ 0: "a", length: 1 }); // false
```

Cómo se declara

La forma normal es el literal, con corchetes:

```
const vacio = [];  
const numeros = [1, 2, 3];  
const nombres = ["Ana", "Luis", "Sol"];
```

Hay otras formas, que sirven para casos puntuales:

```
new Array(5); // arreglo de largo 5, con 5 huecos vacíos  
new Array(5).fill(0); // [0, 0, 0, 0, 0]  
Array.of(5); // [5], un solo elemento  
Array.from("hola"); // ["h", "o", "l", "a"]  
Array.from({ length: 5 }, (_, i) => i); // [0, 1, 2, 3, 4]  
Array.from({ length: 5 }, (_, i) => i * 10); // [0, 10, 20, 30, 40]
```

`new Array(5)` merece una advertencia. No crea cinco ceros: crea cinco huecos, que se comportan distinto de `undefined` en varios métodos. Si necesitas un arreglo iniciado, agréale siempre `.fill()`.

`Array.from` es la navaja para convertir a arreglo cualquier cosa que se le parezca: una cadena, un `Set`, un `Map`, los `NodeList` que devuelve el DOM, o un objeto con `length`. Es la forma más general de decir "quiero esto como arreglo".

Notá también que declaramos con `const` y aun así vamos a agregar elementos. Es lo que vimos la clase pasada: `const` congela la referencia, no el contenido.

```
const numeros = [1, 2, 3];  
numeros.push(4); // válido  
numeros = []; // TypeError
```

Cómo se accede a los elementos

Con corchetes y el índice, que empieza en cero:

```
const nombres = ["Ana", "Luis", "Sol"];  
  
nombres[0]; // "Ana"  
nombres[2]; // "Sol"  
nombres.length; // 3  
nombres[nombres.length - 1]; // "Sol", el último  
nombres.at(-1); // "Sol", más corto  
nombres.at(-2); // "Luis"
```

`at()` acepta índices negativos y cuenta desde el final. Es más legible que la resta.

Un índice fuera de rango no lanza error: devuelve `undefined`.

```
nombres[99]; // undefined
```

Esa es una diferencia grande con C#, donde tendrías una excepción. Acá el programa sigue con un `undefined` adentro, y el error aparece más adelante, en otro lugar. Es la misma lógica del tipado débil que vimos en la primera clase: JavaScript prefiere seguir antes que detenerse.

Escribir fuera de rango tampoco falla. Extiende el arreglo y deja huecos en el medio:

```
const a = [1, 2, 3];  
a[6] = 7;  
console.log(a);          // [1, 2, 3, <3 huecos>, 7]  
console.log(a.length); // 7
```

Un hueco no es lo mismo que un `undefined` guardado. Al leerlo devuelve `undefined`, pero `map`, `filter` y `forEach` lo saltan, mientras que `for...of` lo recorre. Un arreglo con huecos se llama disperso y conviene evitarlo: es una fuente de diferencias sutiles entre métodos.

Y `length` se puede escribir, cosa poco común:

```
const b = [1, 2, 3, 4, 5];  
b.length = 2;  
console.log(b); // [1, 2]
```

Truncar con `length` funciona, pero es poco claro para quien lee. Preferí `b.splice(2)` o directamente crear otro arreglo.

Agregar y quitar por los extremos

Cuatro métodos cubren el uso del arreglo como pila o como cola. Los cuatro modifican el arreglo original.

```
const cola = ["Ana", "Luis"];  
  
cola.push("Sol");      // agrega al final, devuelve el nuevo largo (3)  
cola.pop();            // saca el último y lo devuelve ("Sol")  
cola.unshift("Eva");   // agrega al principio, devuelve el nuevo largo  
cola.shift();          // saca el primero y lo devuelve ("Eva")
```

`push` y `pop` trabajan sobre el final y son rápidos. `shift` y `unshift` trabajan sobre el principio y obligan a recorrer todo el arreglo para reenumerar, así que con muchos elementos son más lentos.

Combinando `push` y `shift` tenés una cola. Combinando `push` y `pop`, una pila.

```
const pila = [];  
pila.push("a");  
pila.push("b");  
pila.pop();    // "b", el último que entró es el primero que sale
```

`push` acepta varios argumentos, y con propagación podés volcar un arreglo entero adentro de otro:

```
const destino = [1, 2];  
destino.push(3, 4);    // [1, 2, 3, 4]  
destino.push(...[5, 6]); // [1, 2, 3, 4, 5, 6]
```

Arreglos de arreglos y matrices

Un elemento de un arreglo puede ser cualquier valor, incluido otro arreglo. Con eso alcanza para representar una matriz.

```
const matriz = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9],  
];  
  
matriz[0];    // [1, 2, 3], la primera fila  
matriz[1][2]; // 6, fila 1, columna 2  
matriz.length; // 3, cantidad de filas  
matriz[0].length; // 3, cantidad de columnas de la primera fila
```

JavaScript no tiene matrices verdaderas como el `int[,]` de C#. Lo que tiene son arreglos de arreglos, y eso implica que las filas son independientes: nada garantiza que todas midan lo mismo.

Para crear una matriz vacía de un tamaño dado hay una trampa que cae todos los años:

```
// Mal: las tres filas son el mismo arreglo  
const mala = new Array(3).fill([0, 0, 0]);  
mala[0][0] = 99;  
console.log(mala); // [[99,0,0], [99,0,0], [99,0,0]]
```

`fill` guarda el mismo valor en las tres posiciones, y ese valor es una referencia. Las tres filas apuntan al mismo arreglo. La forma correcta crea una fila nueva por vez:

```
const buena = Array.from({ length: 3 }, () => new Array(3).fill(0));
buena[0][0] = 99;
console.log(buena); // [[99,0,0], [0,0,0], [0,0,0]]
```

Guardá este ejemplo, porque es la primera aparición de un problema que va a volver varias veces hoy: copiar un contenedor no copia lo que hay adentro.

Para recorrer una matriz, dos bucles anidados:

```
for (let fila = 0; fila < matriz.length; fila++) {
  for (let col = 0; col < matriz[fila].length; col++) {
    console.log(`[${fila}][${col}] = ${matriz[fila][col]}`);
  }
}
```

Con `for...of` queda más corto, aunque perdés los índices:

```
for (const fila of matriz) {
  for (const valor of fila) {
    console.log(valor);
  }
}
```

Rangos y porciones

Para sacar un tramo del arreglo está `slice`. Recibe la posición inicial y la final, y la final no se incluye.

```
const letras = ["a", "b", "c", "d", "e"];

letras.slice(1, 3); // ["b", "c"]
letras.slice(2);    // ["c", "d", "e"], desde la 2 hasta el final
letras.slice(-2);   // ["d", "e"], las dos últimas
letras.slice();     // copia del arreglo completo
```

`slice` no toca el arreglo original: devuelve uno nuevo.

Su primo `splice` hace lo contrario: modifica el arreglo y devuelve lo que sacó. Sirve para eliminar e insertar en el medio.

```
const dias = ["lun", "mar", "mié", "jue"];

dias.splice(1, 2);           // devuelve ["mar", "mié"]
console.log(dias);          // ["lun", "jue"], quedó modificado

dias.splice(1, 0, "mar", "mié"); // inserta sin eliminar nada
console.log(dias);          // ["lun", "mar", "mié", "jue"]
```

Los tres argumentos de `splice` son: desde qué posición, cuántos elementos elimino, y qué inserto en su lugar. Con eso solo hacés cualquier operación en el medio del arreglo.

Los nombres se parecen y hacen cosas opuestas. Una forma de recordarlo: `slice` corta una rebanada y te la lleva, `splice` opera sobre el original.

Buscar dentro del arreglo

Para valores primitivos alcanza con estos tres:

```
const nombres = ["Ana", "Luis", "Sol"];

nombres.indexOf("Luis");    // 1
nombres.indexOf("Pedro");   // -1, no está
nombres.includes("Sol");    // true
nombres.lastIndexOf("Ana"); // 0, busca desde el final
```

`indexOf` compara con `===`. Por eso no sirve para buscar objetos por contenido, y tampoco encuentra `NaN`. `includes` usa una comparación apenas distinta que sí lo encuentra:

```
[NaN].indexOf(NaN);    // -1
[NaN].includes(NaN);   // true
```

Cuando los elementos son objetos, la búsqueda va por condición:

```
const contactos = [
  { id: 1, nombre: "Ana", activo: true },
  { id: 2, nombre: "Luis", activo: false },
];

contactos.find((c) => c.id === 2);    // el objeto de Luis
contactos.findIndex((c) => c.id === 2); // 1
contactos.findLast((c) => c.activo);   // el último que cumple
contactos.some((c) => c.activo);       // true, hay al menos uno
contactos.every((c) => c.activo);      // false, no todos
```

`find` devuelve `undefined` si no encuentra nada, así que conviene chequear antes de usar el resultado, o encadenar con `?.`:

```
const luis = contactos.find((c) => c.id === 2);
console.log(luis?.nombre ?? "no está");
```

Si vas a buscar muchas veces en la misma lista, el arreglo no es la estructura adecuada: cada `find` la recorre entera. Ahí conviene armar un Map como índice o un Set para pertenencia.

Unir, separar y aplanar

```
const a = [1, 2];
const b = [3, 4];

a.concat(b);    // [1, 2, 3, 4]
[...a, ...b];   // [1, 2, 3, 4], la forma moderna
[0, ...a, 5];   // [0, 1, 2, 5], podés intercalar
```

Los tres puntos son el operador de propagación. Desarma el arreglo en sus elementos sueltos. Lo vas a usar mucho hoy.

Entre arreglo y texto:

```
[1, 2, 3].join(" - ");    // "1 - 2 - 3"
"a,b,c".split(",");       // ["a", "b", "c"]
["Ana", "Luis"].join(", "); // "Ana, Luis"
```

Y para aplanar estructuras anidadas:

```
[[1, 2], [3, [4]]].flat();    // [1, 2, 3, 4]], un nivel
[[1, 2], [3, [4]]].flat(2);   // [1, 2, 3, 4]
[[1, [2, [3]]]].flat(Infinity); // [1, 2, 3], todos los niveles
["a b", "c d"].flatMap((s) => s.split(" ")); // ["a","b","c","d"]
```

`flatMap` es `map` seguido de `flat(1)`. Sirve cuando cada elemento produce cero, uno o varios resultados.

Cómo se recorre

JavaScript tiene tres bucles `for`, y para arreglos solo dos son correctos.

El `for` clásico es el de C, con contador propio:

```
const nombres = ["Ana", "Luis", "Sol"];

for (let i = 0; i < nombres.length; i++) {
  console.log(i, nombres[i]);
}
```

Te da control total: podés ir de a dos, ir para atrás, empezar en el medio, saltar. Usalo cuando el índice sea parte del problema.

```
for (let i = nombres.length - 1; i >= 0; i--) { /* al revés */ }
for (let i = 0; i < nombres.length; i += 2) { /* de a dos */ }
```

Declarar el contador con `let` y no con `var` importa. Con `let` cada vuelta tiene su propia copia de `i`, y eso cambia el comportamiento cuando adentro del bucle creás funciones. Es el ejercicio que quedó de la clase pasada.

El `for...of` recorre los valores, sin índice. Es el que vas a usar casi siempre:

```
for (const nombre of nombres) {
  console.log(nombre);
}
```

Se lee mejor, no te podés equivocar con los límites y no hay riesgo de salirte del rango. Si necesitás el índice, pedilo con `entries()`:

```
for (const [i, nombre] of nombres.entries()) {
  console.log(`${i + 1}. ${nombre}`);
}
```

Ese `[i, nombre]` es desestructuración, y la vemos más abajo.

`for...of` funciona sobre cualquier cosa iterable: arreglos, cadenas, `Set`, `Map` y los resultados de `Object.entries()`. Admite `break` y `continue`, así que podés cortar antes de terminar.

El `for...in` recorre claves y es para objetos. Sobre un arreglo funciona, pero está mal usarlo ahí, por tres motivos.

Devuelve las claves como texto, no como número:


```
const numeros = [10, 20, 30];

for (const i in numeros) {
  console.log(i, typeof i); // "0" string, "1" string, "2" string
  console.log(i + 1);       // "01", "11", "21", concatena
}
```

Recorre cualquier propiedad que alguien le haya agregado al arreglo, no solo los elementos. Y salta los huecos.

La regla es corta: `for...in` para objetos, `for...of` para arreglos. Si te acordás de una sola cosa de esta sección, que sea esa.

Queda un cuarto recorrido, `forEach`, que es un método y no un bucle:

```
nombres.forEach((nombre, indice) => {
  console.log(indice, nombre);
});
```

Tiene dos limitaciones. No acepta `break` ni `continue`: si necesitás cortar antes, usá `for...of`. Y no espera funciones asíncronas, cosa que va a importar cuando lleguemos a las llamadas de red.

Bucle	Recorre	Usalo para
<code>for</code> clásico	índices que controlás vos	saltear, ir al revés, cortar por índice
<code>for...of</code>	valores	recorrer arreglos, cadenas, <code>Set</code> y <code>Map</code>
<code>for...in</code>	claves	recorrer las propiedades de un objeto
<code>.forEach()</code>	valores, con función	aplicar algo a cada elemento sin cortar

Transformar: map, filter y reduce

Acá aparece la forma de trabajar propia de JavaScript. En vez de escribir un bucle y acumular a mano, describís la transformación y el lenguaje la aplica.

`map` devuelve un arreglo nuevo, del mismo largo, con cada elemento transformado.

```
const precios = [100, 200, 300];

const conIva = precios.map((precio) => precio * 1.21);
// [121, 242, 363]

const contactos = [
  { nombre: "Ana", email: "ana@mail.com" },
  { nombre: "Luis", email: "luis@mail.com" },
];

const soloNombres = contactos.map((c) => c.nombre);
// ["Ana", "Luis"]
```

La función recibe también el índice como segundo argumento:

```
nombres.map((nombre, i) => `${i + 1}. ${nombre}`);
```

`filter` devuelve un arreglo nuevo, más corto o igual, con los elementos que cumplen la condición.

```
const numeros = [1, 2, 3, 4, 5, 6];

numeros.filter((n) => n % 2 === 0); // [2, 4, 6]

contactos.filter((c) => c.email.endsWith("@mail.com"));
```

`reduce` es el más general de los tres y el que más cuesta al principio, así que vale explicarlo desde el principio.

`reduce` recorre el arreglo llevando un acumulador. En cada vuelta le pasás el acumulador y el elemento actual, y devolvés el acumulador nuevo. Al final te queda ese acumulador.

```
const precios = [100, 200, 300];

const total = precios.reduce((acumulado, precio) => acumulado + precio, 0);
// 600
```

El `0` del final es el valor inicial del acumulador. Paso a paso: empieza en 0, suma 100 y queda 100, suma 200 y queda 300, suma 300 y queda 600.

Ese valor inicial no es opcional en la práctica. Sin él, `reduce` toma el primer elemento como acumulador, y sobre un arreglo vacío lanza un error:

```
[].reduce((a, b) => a + b);      // TypeError
[].reduce((a, b) => a + b, 0);   // 0
```

El acumulador no tiene que ser un número. Puede ser cualquier cosa, y ahí `reduce` muestra su alcance:

```
// Contar apariciones
const palabras = ["a", "b", "a", "c", "a"];

const conteo = palabras.reduce((acc, palabra) => {
  acc[palabra] = (acc[palabra] ?? 0) + 1;
  return acc;
}, {});
// { a: 3, b: 1, c: 1 }

// Agrupar por una propiedad
const porCiudad = contactos.reduce((acc, c) => {
  (acc[c.ciudad] ??= []).push(c);
  return acc;
}, {});
```

Ese caso de agrupar es tan común que desde 2024 existe una función que lo hace sola:

```
const porCiudad = Object.groupBy(contactos, (c) => c.ciudad);
```

Si el acumulador te queda difícil de seguir, escribí un `for...of`. Un `reduce` ilegible no es mejor que un bucle claro.

Ordenar

`sort` tiene dos particularidades que hay que conocer.

La primera es que modifica el arreglo original.

La segunda es que, sin argumentos, convierte todo a texto y ordena alfabéticamente. Con números eso da resultados equivocados:

```
const numeros = [10, 9, 1, 100];

numeros.sort();           // [1, 10, 100, 9] ← ordenó como texto
numeros.sort((a, b) => a - b); // [1, 9, 10, 100]
numeros.sort((a, b) => b - a); // [100, 10, 9, 1]
```

La función que le pasás recibe dos elementos y devuelve un número: negativo si el primero va antes, positivo si va después, cero si da igual.

Para texto en español, usá `localeCompare`, como vimos con las cadenas. Es lo único que ordena bien los acentos y la ñe:

```
contactos.sort((a, b) => a.nombre.localeCompare(b.nombre, "es"));
```

El orden es estable: los elementos que empatan quedan en el orden en que estaban. Eso permite ordenar por varios criterios encadenando dos `sort`, o combinándolos en una sola función:

```
contactos.sort(
  (a, b) => Number(b.activo) - Number(a.activo) || a.nombre.localeCompare(b.nombre,
  "es")
);
```

Si no querés modificar el original, está `toSorted`, que devuelve una copia ordenada.

Se encadenan

La virtud de estos métodos es que cada uno devuelve un arreglo, así que se enganchan uno detrás de otro:

```
const totalActivos = contactos
  .filter((c) => c.activo)
  .map((c) => c.saldo)
  .reduce((total, saldo) => total + saldo, 0);
```

Se lee de arriba abajo como una frase: filtrar los activos, quedarse con el saldo, sumar. Comparalo con la versión con bucle y vas a ver por qué esta forma se impuso.

El costo es que cada eslabón crea un arreglo intermedio. Con listas de la escala que vamos a manejar no importa. Con cientos de miles de elementos, un solo `for...of` es más eficiente.

Mutar o no mutar

Los métodos de arreglo se dividen en dos grupos, y confundirlos causa errores difíciles de encontrar.

Modifican el original	Devuelven uno nuevo
push, pop	map, filter, reduce
shift, unshift	slice, concat
splice	toSpliced
sort	toSorted
reverse	toReversed
fill	with

Los cuatro de la derecha con nombre en infinitivo (`toSorted`, `toReversed`, `toSpliced`, `with`) son recientes, de 2023. Hacen lo mismo que su par de la izquierda pero sin tocar el original.

```
const original = [3, 1, 2];

const ordenadoMal = original.sort();      // modificó original
const ordenadoBien = original.toSorted(); // original queda igual
```

Trabajar sin modificar el original tiene ventajas concretas. Si nadie modifica un arreglo, nadie puede romperlo desde otra parte del programa. Comparar si algo cambió se reduce a comparar dos referencias. Y React, que vamos a ver más adelante, directamente depende de esto para saber cuándo redibujar la pantalla.

Estas son las cuatro operaciones básicas en las dos formas:

```
const contactos = [{ id: 1, nombre: "Ana" }, { id: 2, nombre: "Luis" }];

// Agregar
contactos.push(nuevo);                // muta
const conNuevo = [...contactos, nuevo]; // inmutable
const alPrincipio = [nuevo, ...contactos]; // inmutable

// Eliminar
contactos.splice(1, 1);                // muta
const sinLuis = contactos.filter((c) => c.id !== 2); // inmutable

// Actualizar uno
contactos[0].nombre = "Ana María";    // muta
const actualizados = contactos.map((c) =>
  c.id === 1 ? { ...c, nombre: "Ana María" } : c
);                                     // inmutable

// Reemplazar por posición
const reemplazado = contactos.with(0, otroContacto); // inmutable
```

Fijate el patrón de la actualización: `map` recorre todo, y para el elemento que querés cambiar devolvés un objeto nuevo con las propiedades del viejo más el cambio. Para los demás devolvés el mismo objeto. Ese patrón lo vas a escribir cien veces en esta materia.

Referencia y copia

Retomamos algo de la clase pasada, porque con arreglos se vuelve más grave.

Asignar un arreglo a otra variable no lo copia. Copia la flecha:

```
const a = [1, 2, 3];
const b = a;

b.push(4);
console.log(a);      // [1, 2, 3, 4], cambió también
console.log(a === b); // true, es el mismo arreglo
```

Para copiar de verdad hay tres formas equivalentes:

```
const copia1 = [...a];
const copia2 = a.slice();
const copia3 = Array.from(a);

copia1.push(99);
console.log(a); // sin cambios
```

Pero atención, porque acá aparece otra vez el problema de la matriz. Estas tres formas hacen una copia superficial: copian el arreglo de afuera, no lo que hay adentro.

```
const contactos = [{ nombre: "Ana" }, { nombre: "Luis" }];
const copia = [...contactos];

copia.push({ nombre: "Sol" });
console.log(contactos.length); // 2, el arreglo externo es otro

copia[0].nombre = "Ana María";
console.log(contactos[0].nombre); // "Ana María", el objeto es el mismo
```

El arreglo se duplicó. Los objetos de adentro, no: las dos listas apuntan a los mismos objetos.

Para una copia profunda, donde nada se comparta, está `structuredClone`:

```
const copiaProfunda = structuredClone(contactos);
copiaProfunda[0].nombre = "Ana María";
console.log(contactos[0].nombre); // "Ana", intacto
```

Vas a ver también este truco en código viejo:

```
const copia = JSON.parse(JSON.stringify(contactos));
```

Funciona, pero pierde por el camino las fechas, las funciones, los `undefined` y falla con referencias circulares. Usá `structuredClone`.

La comparación sigue la misma lógica. `===` sobre arreglos compara referencias, no contenido:

```
[1, 2] === [1, 2]; // false, son dos arreglos distintos
```

Para comparar contenido tenés que recorrer:

```
const iguales = a.length === b.length && a.every((v, i) => v === b[i]);
```

Eso sirve para primitivos. Si los elementos son objetos, la comparación tiene que entrar en cada uno.

Desestructuración

Desestructurar es sacar valores de una estructura y ponerlos en variables sueltas, en una sola línea.

```
const coordenadas = [10, 20, 30];

// Sin desestructurar
const x1 = coordenadas[0];
const y1 = coordenadas[1];

// Desestructurando
const [x, y, z] = coordenadas;
```

Las variables se asignan por posición, no por nombre. Podés saltar posiciones dejando el lugar vacío:

```
const [, , tercero] = coordenadas; // 30
```

Podés juntar el resto en un arreglo con los tres puntos:

```
const [primero, ...resto] = [1, 2, 3, 4, 5];  
// primero = 1, resto = [2, 3, 4, 5]
```

Podés poner valores por defecto para cuando falte un elemento:

```
const [a = 0, b = 0, c = 0] = [10, 20];  
// a = 10, b = 20, c = 0
```

El valor por defecto entra solo cuando el elemento es `undefined`, no cuando es `null` ni `0`.

Funciona con estructuras anidadas, lo que sirve para matrices:

```
const matriz = [[1, 2], [3, 4]];  
const [[a1, a2], [b1, b2]] = matriz;  
// a1=1, a2=2, b1=3, b2=4
```

Sirve para intercambiar dos variables sin una tercera:

```
let x = 1;  
let y = 2;  
[x, y] = [y, x];  
// x = 2, y = 1
```

Y aparece constantemente en los bucles y en los parámetros de funciones:

```
for (const [indice, nombre] of nombres.entries()) { /* ... */ }  
  
function distancia([x1, y1], [x2, y2]) {  
  return Math.hypot(x2 - x1, y2 - y1);  
}  
  
distancia([0, 0], [3, 4]); // 5
```

Como la desestructuración funciona sobre cualquier iterable, también sirve con cadenas y con `Set`:

```
const [primeraLetra] = "hola"; // "h"
```


Errores frecuentes

Ordenar números sin comparador. `[10, 9].sort()` devuelve `[10, 9]`, porque compara como texto.

Usar `map` cuando no te interesa el resultado. Si solo querés ejecutar algo por cada elemento, usá `for...of`. `map` construye un arreglo que después tirás.

Passar una función que espera más argumentos de los que creés:

```
["1", "2", "3"].map(parseInt); // [1, NaN, NaN]
```

`map` le pasa a la función el valor y el índice, y `parseInt` toma ese índice como base de numeración. La forma correcta es explicitar el argumento: `.map((s) => parseInt(s, 10))`.

Confundir `slice` con `splice`. Uno copia, el otro modifica.

Olvidar que `find` puede devolver `undefined` y usar el resultado directo.

Copiar un arreglo de objetos con propagación y creer que los objetos también se copiaron.

Modificar el arreglo mientras lo recorrés con `for` clásico. Si eliminás elementos, los índices se corren y salteás elementos. Filtrá y quedate con el arreglo nuevo.

Para llevarte

- el arreglo agrupa por posición, y el índice arranca en cero
- un índice fuera de rango devuelve `undefined` en vez de fallar
- `at(-1)` es la forma legible de pedir el último
- `push` y `pop` trabajan sobre el final; `shift` y `unshift` sobre el principio y son más lentos
- no hay matrices verdaderas: son arreglos de arreglos, y `fill` con un arreglo comparte la misma referencia en todas las filas
- `slice` devuelve una porción nueva, `splice` modifica el original
- `for...of` para arreglos, `for` clásico cuando el índice importa, nunca `for...in`
- `map`, `filter` y `reduce` devuelven algo nuevo; `push`, `sort` y `reverse` modifican
- `reduce` siempre con valor inicial
- `sort` sin comparador ordena como texto, y modifica el original
- asignar copia la referencia; `[...a]` copia el arreglo pero no los objetos de adentro; `structuredClone` copia todo
- `===` sobre arreglos compara referencias, no contenido
- la desestructuración toma por posición y admite valores por defecto y resto

Para probar

1. Creá una matriz de 3 por 3 llena de ceros usando `new Array(3).fill([...])` y otra usando `Array.from`. Modificá el elemento `[0][0]` en las dos y explicá la diferencia.
2. Partiendo de un arreglo de contactos, escribí en una sola cadena de métodos el listado de los emails de los contactos activos, ordenados alfabéticamente.
3. Escribí dos versiones de "agregar un contacto a la agenda": una que modifique el arreglo y otra inmutable. Verificá con `===` que la segunda devuelve un arreglo distinto.
4. Copiá un arreglo de objetos con el operador de propagación, modificá una propiedad de un objeto de la copia y comprobá qué pasó con el original. Después repetilo con `structuredClone`.
5. Implementá `map`, `filter` y `find` con un `for...of` y comparalos con los originales.
6. Ordená un arreglo de contactos primero por activo y después por nombre, en una sola llamada a `sort`.
7. Explicá por qué `["1", "2", "3"].map(parseInt)` no devuelve `[1, 2, 3]` y arreglalo.

Objetos

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte de la tercera clase de JavaScript, 2 de 4

El arreglo agrupa por posición y responde "cuál es el tercero". El objeto agrupa por nombre y responde "cuál es el email". Cuando los datos son partes distintas de una misma cosa, el objeto es la estructura correcta.

En desarrollo web el objeto es la moneda corriente. Todo lo que llega de una API llega como JSON, y JSON son objetos y arreglos anidados. Los otros apuntes de esta clase son arreglos, Set y Map.

Qué es un objeto

Un objeto agrupa valores y les pone nombre. Cada par de nombre y valor es una propiedad.

```
const contacto = {  
  nombre: "Ana Pérez",  
  email: "ana@mail.com",  
  edad: 30,  
  activo: true,  
};
```

El nombre de la propiedad, o clave, siempre es texto. El valor puede ser cualquier cosa, incluidos otros objetos, arreglos y funciones.

Que la clave sea siempre texto tiene una consecuencia que sorprende: si usás un número, se convierte.

```
const puntajes = {};  
puntajes[1] = "uno";  
puntajes["1"]; // "uno", es la misma propiedad  
Object.keys(puntajes); // ["1"], la clave quedó como texto
```

Cuando necesites claves que no sean texto, la estructura correcta es Map.

Una aclaración para quien viene de C#: esto no es una instancia de una clase. Es un objeto suelto, creado en el momento, sin ninguna definición previa. JavaScript también tiene clases, pero la mayoría de los objetos que vas a manejar en desarrollo web nacen así, como literales o como resultado de leer un JSON.

Los objetos se anidan sin límite:

```
const contacto = {  
  nombre: "Ana",  
  direccion: {  
    calle: "Rivadavia 123",  
    ciudad: "Tucumán",  
  },  
  telefonos: ["3814567890", "3811234567"],  
};
```

El punto y los corchetes

Hay dos formas de acceder a una propiedad.

La sintaxis de punto es la habitual:

```
contacto.nombre;           // "Ana"  
contacto.direccion.ciudad; // "Tucumán"  
contacto.telefonos[0];     // "3814567890"
```

La sintaxis de corchetes usa la clave como texto:

```
contacto["nombre"]; // "Ana"
```

Con una clave fija son equivalentes y siempre se prefiere el punto. Pero hay tres casos en que los corchetes son obligatorios.

Cuando la clave no es un identificador válido:

```
const config = {  
  "api-key": "abc123",  
  "max intentos": 3,  
};  
  
config["api-key"]; // única forma  
config.api-key;    // error de sintaxis, lo lee como una resta
```

Cuando la clave está en una variable:

```
const campo = "email";

contacto[campo];    // "ana@mail.com"
contacto.campo;     // undefined, busca una propiedad llamada "campo"
```

Y cuando la clave se calcula en el momento:

```
for (const campo of ["nombre", "email"]) {
  console.log(`${campo}: ${contacto[campo]}`);
}
```

Esa diferencia entre `contacto[campo]` y `contacto.campo` es el error más común del tema. El punto toma lo que escribiste literalmente; los corchetes evalúan la expresión.

Agregar, modificar y eliminar

Los objetos son abiertos: agregás propiedades cuando querés.

```
const contacto = { nombre: "Ana" };

contacto.email = "ana@mail.com"; // agrega
contacto.nombre = "Ana María";   // modifica
delete contacto.email;           // elimina
```

Pedir una propiedad que no existe devuelve `undefined`, sin error:

```
contacto.telefono; // undefined
```

Y pedir una propiedad de algo que es `undefined` sí falla:

```
contacto.direccion.ciudad; // TypeError si no hay direccion
contacto.direccion?.ciudad; // undefined, sin fallar
```

El encadenamiento opcional tiene tres formas, según qué venga después:

```
contacto?.direccion;      // por si contacto no existe
contacto.telefonos?.[0];  // acceso por índice
contacto.saludar?.();     // llamada a un método que puede no estar
```

Para preguntar si una propiedad existe:

```
"nombre" in contacto;           // true
Object.hasOwn(contacto, "nombre"); // true, solo propiedades propias
contacto.nombre !== undefined;   // sirve, salvo que valga undefined
```

La diferencia entre `in` y `Object.hasOwn` importa cuando usás el objeto como diccionario. `in` también encuentra las propiedades heredadas, y todo objeto hereda unas cuantas:

```
const diccionario = {};
"toString" in diccionario;           // true, viene heredada
Object.hasOwn(diccionario, "toString"); // false, es lo que querías preguntar
```

Si vas a usar un objeto como diccionario de claves que vienen de afuera, usá `Object.hasOwn`, o directamente usá un `Map`, que no tiene este problema.

Las formas abreviadas

Hay tres atajos de sintaxis que aparecen en todo el código moderno.

El primero es la abreviatura de propiedad. Cuando la clave y la variable se llaman igual, escribís el nombre una sola vez:

```
const nombre = "Ana";
const email = "ana@mail.com";

// Forma larga
const contacto = { nombre: nombre, email: email };

// Forma abreviada
const contacto = { nombre, email };
```

Es exactamente lo mismo. Lo vas a ver a la salida de casi toda función que arme un objeto.

El segundo es la abreviatura de método. Si el valor es una función, podés omitir `function` y los dos puntos:

```
// Forma larga
const agenda = {
  contactos: [],
  agregar: function (c) { this.contactos.push(c); },
};

// Forma abreviada
const agenda = {
  contactos: [],
  agregar(c) { this.contactos.push(c); },
};
```

Adentro de un método, `this` es el objeto que está antes del punto en la llamada. Lo vemos en detalle cuando lleguemos a funciones; por ahora alcanza con saber que `agenda.agregar(c)` hace que `this` sea `agenda`.

El tercero son las claves calculadas. Con corchetes dentro del literal, la clave sale de una expresión:

```
const campo = "email";
const valor = "ana@mail.com";

const objeto = { [campo]: valor };
// { email: "ana@mail.com" }

// Útil para construir objetos dinámicos
function crearFiltro(campo, valor) {
  return { [campo]: valor, [`${campo}Exacto`]: true };
}
```

Propagación y combinación

El operador de propagación, que ya usamos con arreglos, también funciona con objetos:

```
const base = { nombre: "Ana", activo: true };

const copia = { ...base };
const conEmail = { ...base, email: "ana@mail.com" };
const modificado = { ...base, nombre: "Ana María" };
```

El orden importa: lo que va después pisa lo que va antes.

```
{ ...base, activo: false }; // activo queda en false
{ activo: false, ...base }; // activo vuelve a true
```

Combinando dos objetos:

```
const porDefecto = { pagina: 1, orden: "nombre", limite: 10 };
const delUsuario = { limite: 50 };

const opciones = { ...porDefecto, ...delUsuario };
// { pagina: 1, orden: "nombre", limite: 50 }
```

Ese patrón de valores por defecto más valores del usuario lo vas a usar en cada función de configuración que escribas.

Ojo con una cosa: la combinación es superficial. Si una propiedad es un objeto, se pisa entera, no se mezcla:

```
const base = { direccion: { calle: "Rivadavia 123", ciudad: "Tucumán" } };
const cambio = { direccion: { ciudad: "Salta" } };

{ ...base, ...cambio };
// { direccion: { ciudad: "Salta" } }, se perdió la calle
```

Para actualizar una propiedad anidada sin perder el resto, hay que propagar en cada nivel:

```
const actualizado = {
  ...contacto,
  direccion: { ...contacto.direccion, ciudad: "Salta" },
};
```

Ese patrón es incómodo y por eso más adelante vamos a ver herramientas que lo simplifican. Por ahora, acordate de la regla: un nivel de propagación por cada nivel que quieras conservar.

Desestructuración

Igual que con arreglos, pero por nombre en vez de por posición.

```
const contacto = { nombre: "Ana", email: "ana@mail.com", edad: 30 };

const { nombre, email } = contacto;
// nombre = "Ana", email = "ana@mail.com"
```


El orden no importa y podés tomar solo lo que necesitás.

Para usar otro nombre de variable, con dos puntos:

```
const { nombre: nombreContacto, email: correo } = contacto;
```

Valores por defecto para lo que falte:

```
const { nombre, telefono = "sin teléfono" } = contacto;
```

El valor por defecto entra cuando la propiedad es `undefined`, no cuando es `null` ni cadena vacía.

Se puede combinar todo:

```
const { nombre: n = "Anónimo", ciudad = "Tucumán" } = contacto;
```

Anidada, para llegar a propiedades internas:

```
const contacto = {  
  nombre: "Ana",  
  direccion: { calle: "Rivadavia 123", ciudad: "Tucumán" },  
};  
  
const { direccion: { ciudad } } = contacto;  
// ciudad = "Tucumán", y direccion no queda como variable
```

Cuidado con la desestructuración anidada: si `direccion` no existe, la línea falla. Poné un valor por defecto para el nivel intermedio:

```
const { direccion: { ciudad } = {} } = contacto;
```

Y el resto, que junta lo que no nombraste:

```
const { nombre, ...otrosDatos } = contacto;  
// nombre = "Ana"  
// otrosDatos = { email: ..., edad: ... }
```

Ese último uso sirve para sacar una propiedad de un objeto sin modificarlo, que es la versión inmutable de `delete`:

```
const { password, ...contactoPublico } = usuario;  
// contactoPublico tiene todo menos password, y usuario queda intacto
```

El lugar donde más se usa la desestructuración es en los parámetros de las funciones:

```
// Sin desestructurar: hay que recordar el orden  
function crearContacto(nombre, email, edad, activo) { /* ... */  
  crearContacto("Ana", "ana@mail.com", 30, true);  
  
// Desestructurando: los argumentos se nombran  
function crearContacto({ nombre, email, edad = 0, activo = true }) {  
  return { nombre, email, edad, activo };  
}  
  
crearContacto({ email: "ana@mail.com", nombre: "Ana" });
```

La segunda versión no depende del orden, documenta qué recibe y permite valores por defecto. Es la forma habitual cuando una función tiene más de tres parámetros.

Recorrer un objeto

Un objeto no es iterable, así que `for...of` no funciona directamente sobre él. Hay dos caminos.

El primero es `for...in`, que recorre las claves:

```
for (const clave in contacto) {  
  console.log(clave, contacto[clave]);  
}
```

Tiene el mismo problema que `Object.hasOwn` resuelve: también recorre propiedades heredadas. Con objetos literales no pasa, pero con objetos que vienen de una clase o de una librería, sí.

El segundo camino, más recomendable, es convertir el objeto en un arreglo y recorrerlo con las herramientas que ya conocés:

```
Object.keys(contacto);    // ["nombre", "email", "edad"]  
Object.values(contacto); // ["Ana", "ana@mail.com", 30]  
Object.entries(contacto); // [["nombre","Ana"], ["email","ana@mail.com"], ...]
```

Con `entries` y desestructuración queda muy legible:

```
for (const [clave, valor] of Object.entries(contacto)) {  
  console.log(`${clave}: ${valor}`);  
}
```

Y como son arreglos, podés encadenar métodos:

```
// Quedarse con las propiedades que tienen valor  
const limpio = Object.fromEntries(  
  Object.entries(contacto).filter(([, valor]) => valor !== null)  
);  
  
// Transformar todos los valores  
const enMayusculas = Object.fromEntries(  
  Object.entries(contacto).map(([k, v]) => [k, String(v).toUpperCase()])  
);
```

`Object.fromEntries` es el camino inverso: de arreglo de pares a objeto.

Sobre el orden de las claves, hay una regla que conviene conocer. Las claves que parecen números enteros salen primero, ordenadas de menor a mayor. El resto sale en el orden en que las agregaste.

```
const raro = { b: 1, 2: "dos", a: 3, 1: "uno" };  
Object.keys(raro); // ["1", "2", "b", "a"]
```

Si el orden te importa, no uses un objeto: usá un arreglo o un Map.

Copiar y comparar

Vale lo mismo que para los arreglos, porque el mecanismo es el mismo: la variable guarda una referencia.

```
const a = { nombre: "Ana" };  
const b = a;  
  
b.nombre = "Luis";  
console.log(a.nombre); // "Luis", es el mismo objeto
```

Para copiar en forma superficial:

```
const copia1 = { ...a };
const copia2 = Object.assign({}, a); // forma anterior a la propagación
```

Las dos copian un solo nivel. Lo que esté anidado se sigue compartiendo:

```
const original = { direccion: { ciudad: "Tucumán" } };
const copia = { ...original };

copia.direccion.ciudad = "Salta";
console.log(original.direccion.ciudad); // "Salta", el objeto interno es el mismo
```

Para una copia profunda, `structuredClone`:

```
const copiaProfunda = structuredClone(original);
```

Copia objetos, arreglos, fechas, `Map` y `Set`. No copia funciones: si el objeto tiene métodos, falla.

Y la comparación funciona como con los arreglos: `===` compara referencias.

```
{ a: 1 } === { a: 1 }; // false, son dos objetos distintos
```

Para comparar contenido hay que recorrer las claves, o comparar sus representaciones en texto si el caso es simple y el orden de las claves coincide.

Congelar y proteger

```
const config = Object.freeze({ pagina: 1, limite: 10 });

config.limite = 50; // no hace nada, en modo estricto lanza error
Object.isFrozen(config); // true
```

`Object.freeze` también es superficial: congela el objeto de afuera, no los objetos anidados.

Objetos y JSON

Es la conversión que más vas a usar en toda la materia, porque es cómo viajan los datos por la red.

```
JSON.stringify(contacto); // objeto a texto
JSON.parse(texto);        // texto a objeto
```

`JSON.stringify` es también la forma de mostrar un objeto legible en consola, en vez del `[object Object]` que vimos la clase pasada:

```
console.log(JSON.stringify(contacto, null, 2)); // con sangría de 2
```

Tiene pérdidas que conviene conocer desde ahora. Descarta las propiedades cuyo valor es `undefined` o una función. Convierte las fechas en texto, así que al volver con `parse` ya no son fechas. Y falla con referencias circulares.

```
JSON.stringify({ a: undefined, b: () => 1, c: new Date() });
// {"c":"2026-08-15T..."}
```

Errores frecuentes

Escribir `contacto.campo` cuando la clave está en la variable `campo`.

Crear que la propagación combina en profundidad. Pisa el nivel superior y listo.

Usar `for...in` sobre un arreglo. Para arreglos, `for...of`.

Preguntar `if (contacto.activo)` cuando la propiedad puede valer `0` o cadena vacía. Ahí conviene `if ("activo" in contacto)` o `!== undefined`.

Usar un objeto como diccionario con claves que vienen del usuario, sin `Object.hasOwn`. Una clave llamada `toString` o `constructor` da respuestas inesperadas.

Comparar dos objetos con `===` esperando comparar contenido.

Para llevarte

- el objeto agrupa por nombre, y la clave siempre es texto
- el punto toma la clave literal; los corchetes evalúan la expresión
- pedir una propiedad inexistente devuelve `undefined`, pero pedirle una propiedad a `undefined` falla
- usá `?.` para atravesar estructuras que pueden estar incompletas
- `{ nombre }` es lo mismo que `{ nombre: nombre }`
- la propagación combina, respeta el orden y es superficial

- la desestructuración toma por nombre, admite renombrar, valores por defecto y resto
- `Object.keys`, `values` y `entries` convierten el objeto en arreglo para recorrerlo
- copiar es superficial salvo con `structuredClone`, y `===` compara referencias
- `JSON.stringify` pierde funciones, `undefined` y el tipo de las fechas

Para probar

1. Escribí una función que reciba un objeto de configuración con desestructuración y valores por defecto, y devuelva la configuración completa.
2. Dado un contacto con dirección anidada, escribí la versión inmutable de cambiar solo la ciudad, sin perder la calle.
3. Escribí una función `sinVacios(objeto)` que devuelva un objeto nuevo sin las propiedades cuyo valor sea `null`, `undefined` o cadena vacía.
4. Escribí `sonIguales(a, b)` que compare dos objetos planos por contenido y explicá por qué `===` no sirve.
5. Convertí un arreglo de contactos en un objeto indexado por id usando `Object.fromEntries`, y después hacé lo mismo con un Map. Compará las dos versiones.
6. Probá `JSON.parse(JSON.stringify(x))` sobre un objeto que tenga una fecha y una función, y explicá qué se perdió.

Set

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte de la tercera clase de JavaScript, 3 de 4

El arreglo guarda por posición y el objeto por nombre. Quedan casos que ninguno de los dos resuelve bien, y para eso están las dos estructuras que agregó ES2015: `Set` y `Map`.

Un `Set` es una colección de valores sin repetidos y sin claves. Responde una sola pregunta: si un valor está o no está.

Cómo se crea y se usa

```
const etiquetas = new Set();

etiquetas.add("cliente");
etiquetas.add("activo");
etiquetas.add("cliente"); // ignorado, ya estaba

etiquetas.size;           // 2
etiquetas.has("activo");  // true
etiquetas.delete("activo"); // true, lo eliminó
etiquetas.clear();        // vacía todo
```

Toda la interfaz son esos cinco métodos más `size`. No hay índices, no hay `length`, no se puede pedir "el tercero".

`add` devuelve el mismo `Set`, así que se puede encadenar:

```
const permisos = new Set().add("leer").add("escribir");
```

Se puede crear desde cualquier iterable, y de ahí sale su uso más frecuente:

```
const numeros = [1, 2, 2, 3, 3, 3];
const sinRepetir = [...new Set(numeros)]; // [1, 2, 3]

new Set("hola"); // {"h", "o", "l", "a"}
new Set(mapaDeContactos.keys());
```

Ese `[...new Set(x)]` es el modismo estándar para quitar duplicados de un arreglo. Vale la pena memorizarlo.

Cómo se recorre

Es iterable, así que se recorre con `for...of` y mantiene el orden en que agregaste los elementos:

```
for (const etiqueta of etiquetas) {  
  console.log(etiqueta);  
}  
  
etiquetas.forEach((e) => console.log(e));
```

Para volver a arreglo y usar `map`, `filter` y compañía:

```
[...etiquetas];           // arreglo  
Array.from(etiquetas);    // lo mismo  
[...etiquetas].sort();    // ordenado
```

Un `Set` no tiene `map` ni `filter` propios. La forma de transformarlo es convertirlo a arreglo, operar y volver:

```
const enMayusculas = new Set([...etiquetas].map((e) => e.toUpperCase()));
```

Cómo compara los valores

La comparación es prácticamente la de `===`, con una excepción: `NaN` sí se considera igual a sí mismo dentro de un `Set`. Esa regla se llama `SameValueZero`, y también trata a `0` y `-0` como el mismo valor.

```
const s = new Set();  
s.add(NaN);  
s.add(NaN);  
s.size;    // 1, aunque NaN !== NaN
```

Como los objetos se comparan por referencia, dos objetos con el mismo contenido son dos elementos distintos:


```
const s = new Set();
s.add({ a: 1 });
s.add({ a: 1 });
s.size; // 2
```

Esto limita mucho el uso de `Set` con objetos. Si querés eliminar duplicados de una lista de objetos, tenés que elegir una clave y usarla:

```
const contactos = [
  { id: 1, nombre: "Ana" },
  { id: 1, nombre: "Ana" },
  { id: 2, nombre: "Luis" },
];

// Con Set, quedándote solo con la clave
const idsUnicos = new Set(contactos.map((c) => c.id)); // {1, 2}

// Con Map, si querés los objetos completos
const unicos = [...new Map(contactos.map((c) => [c.id, c])).values()];
```

Por qué existe: la pertenencia

Comparado con un arreglo, `Set` gana cuando la pregunta es la pertenencia.

Buscar en un arreglo con `includes` recorre los elementos uno por uno. En el peor caso los mira todos. En un `Set`, `has` va directo al valor, sin importar cuántos elementos haya.

```
const lista = ["a", "b", "c" /* ... 100000 elementos */];
const conjunto = new Set(lista);

lista.includes("z"); // recorre todo antes de decir que no
conjunto.has("z"); // respuesta inmediata
```

Con diez elementos no se nota. Con diez mil, y adentro de un bucle, la diferencia es de segundos contra milisegundos. El caso típico es filtrar una lista contra otra:

```
// Lento: por cada contacto, recorre todos los ids bloqueados
const permitidos = contactos.filter((c) => !idsBloqueados.includes(c.id));

// Rápido: el Set se arma una vez y cada consulta es directa
const bloqueados = new Set(idsBloqueados);
const permitidos = contactos.filter((c) => !bloqueados.has(c.id));
```

Operaciones de conjuntos

Desde 2024 los navegadores incorporaron los métodos de la teoría de conjuntos. Reciben otro `Set` y devuelven uno nuevo, sin modificar los originales:

```
const a = new Set([1, 2, 3]);
const b = new Set([3, 4]);

a.union(b);           // {1, 2, 3, 4}
a.intersection(b);    // {3}
a.difference(b);       // {1, 2}
a.symmetricDifference(b); // {1, 2, 4}

a.isSubsetOf(b);       // false
a.isSupersetOf(new Set([1])); // true
a.isDisjointFrom(b);   // false, comparten el 3
```

Son recientes, así que en entornos viejos hay que hacerlas a mano:

```
const union = new Set([...a, ...b]);
const interseccion = new Set([...a].filter((x) => b.has(x)));
const diferencia = new Set([...a].filter((x) => !b.has(x)));
```

Vale la pena escribir las tres a mano una vez: se entiende mejor qué hace cada una y por qué el `Set` es la estructura natural para esto.

Casos de uso habituales

Quitar duplicados de una lista:

```
const ciudades = [...new Set(contactos.map((c) => c.ciudad))];
```

Etiquetas o permisos de una entidad, donde solo importa si están:

```
const permisos = new Set(["leer", "escribir"]);
if (permisos.has("borrar")) { /* ... */ }
```

Marcar lo ya procesado, para no repetir trabajo ni entrar en ciclos:

```
const visitados = new Set();

for (const nodo of grafo) {
  if (visitados.has(nodo.id)) continue;
  visitados.add(nodo.id);
  // procesar
}
```

Filtrar una lista contra otra, como vimos arriba.

Set y WeakSet

Existe una variante, `WeakSet`, que solo acepta objetos y no impide que el recolector de basura los libere. No es iterable ni tiene `size`. Sirve para marcar objetos sin retenerlos en memoria, por ejemplo para recordar cuáles ya se procesaron sin evitar que se descarten.

Es un caso raro en desarrollo web común. Alcanza con saber que existe.

Cuándo usar Set

Necesito	Estructura
orden y elementos repetidos	arreglo
propiedades con nombre de una entidad	objeto
saber si un valor está, sin repetidos	<code>Set</code>
asociar una clave a un valor	<code>Map</code>

Dos límites a tener presentes. Un `Set` no se convierte solo a JSON: `JSON.stringify(new Set([1,2]))` devuelve `{}`, así que hay que pasarlo a arreglo antes de mandarlo por la red. Y no sirve para deduplicar objetos por contenido, por lo que vimos sobre la comparación por referencia.

Para llevarte

- `Set` guarda valores únicos y responde una sola pregunta: si un valor está
- la interfaz es `add`, `has`, `delete`, `clear` y `size`
- `[...new Set(arreglo)]` es la forma estándar de quitar duplicados
- es iterable y conserva el orden de inserción
- compara con `SameValueZero`: como `===`, salvo que `NaN` es igual a sí mismo

- los objetos se comparan por referencia, así que no deduplica por contenido
- `has` es inmediato; `includes` de un arreglo recorre todo
- las operaciones de conjuntos ya son métodos, y también se escriben con `filter` y propagación
- no se serializa a JSON: convertirlo a arreglo antes

Para probar

1. Escribí una función `duplicados(arreglo)` que devuelva los valores que aparecen más de una vez, usando un `Set` de vistos.
2. Dado un arreglo de contactos, obtené la lista de ciudades distintas ordenada alfabéticamente.
3. Implementá unión, intersección y diferencia con `filter` y propagación, y compará el resultado con los métodos nuevos.
4. Medí con `console.time` la diferencia entre buscar 10000 veces en un arreglo con `includes` y en un `Set` con `has`.
5. Explicá por qué `new Set([{'a':1}, {'a':1}]).size` da 2 y escribí una alternativa que deduplique por contenido.
6. Convertí un `Set` a JSON y de vuelta a `Set`.

Map

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte de la tercera clase de JavaScript, 4 de 4

Un `Map` guarda pares de clave y valor, igual que un objeto. La diferencia es para qué está pensado cada uno.

El objeto representa una entidad con propiedades conocidas de antemano: un contacto tiene nombre, email y edad. El `Map` representa un diccionario: claves que aparecen y desaparecen mientras el programa corre, y que no necesariamente son texto.

Los otros apuntes de esta clase son arreglos, objetos y Set.

Cómo se crea y se usa

```
const contactosPorId = new Map();

contactosPorId.set(1, { nombre: "Ana" });
contactosPorId.set(2, { nombre: "Luis" });

contactosPorId.get(1);      // { nombre: "Ana" }
contactosPorId.get(99);     // undefined, no está
contactosPorId.has(2);      // true
contactosPorId.delete(2);   // true
contactosPorId.size;        // 1
contactosPorId.clear();
```

`set` devuelve el mismo `Map`, así que se puede encadenar:

```
const config = new Map()
  .set("pagina", 1)
  .set("limite", 10);
```

Se puede crear desde un arreglo de pares, lo que lo conecta con `Object.entries`:

```
const desdeArreglo = new Map([["a", 1], ["b", 2]]);
const desdeObjeto = new Map(Object.entries({ a: 1, b: 2 }));
const deVueltaAObjeto = Object.fromEntries(desdeObjeto);
```

Las tres diferencias con el objeto

La primera es que la clave puede ser cualquier valor, no solo texto: números, booleanos, objetos, funciones.

```
const m = new Map();
m.set(1, "número uno");
m.set("1", "texto uno");
m.get(1);    // "número uno", son claves distintas
```

En un objeto común, `obj[1]` y `obj["1"]` son la misma propiedad, porque las claves siempre se convierten a texto.

Usar un objeto como clave abre un uso que el objeto literal no permite: asociar datos a una entidad sin tocarla.

```
const metadatos = new Map();
metadatos.set(contactoAna, { seleccionado: true });
metadatos.get(contactoAna);
```

La comparación de claves es la misma de `Set`, `SameValueZero`: como `===`, salvo que `NaN` funciona como clave y `0` y `-0` son la misma. Y los objetos se comparan por referencia, así que `m.get({id: 1})` nunca encuentra lo que guardaste con otro objeto de igual contenido.

La segunda diferencia es que `size` te da la cantidad directamente, sin tener que pedir `Object.keys().length`.

La tercera es que es iterable y conserva el orden de inserción, siempre, sin la regla rara de las claves numéricas del objeto.

Cómo se recorre

```
for (const [clave, valor] of contactosPorId) {
  console.log(clave, valor);
}

contactosPorId.forEach((valor, clave) => console.log(clave, valor));
```

Fijate que en `forEach` el valor viene primero y la clave después, al revés de lo que uno espera. Es una fuente clásica de confusión.

Tres métodos devuelven iteradores:

```
contactosPorId.keys();    // iterador de claves
contactosPorId.values();  // iterador de valores
contactosPorId.entries(); // iterador de pares
```

Un iterador no es un arreglo: no tiene `map` ni `filter`. Para operar con las herramientas que ya conocés, convertilo:

```
[...contactosPorId];          // [[1, {...}], [2, {...}]]
[...contactosPorId.keys()];    // [1, 2]
[...contactosPorId.values()];  // [{...}, {...}]

const activos = [...contactosPorId.values()].filter((c) => c.activo);
```

Y para transformar un `Map` en otro, el camino de ida y vuelta:

```
const duplicados = new Map(
  [...precios].map(([clave, valor]) => [clave, valor * 2])
);
```

Patrones habituales

Indexar una lista para poder buscar rápido. Es el uso más frecuente:

```
const contactos = [
  { id: 1, nombre: "Ana" },
  { id: 2, nombre: "Luis" },
];

const indice = new Map(contactos.map((c) => [c.id, c]));

indice.get(2); // acceso directo, sin recorrer el arreglo
```

Si vas a buscar una sola vez, `find` alcanza. Si vas a buscar muchas veces, armar el índice una vez y consultarlo es mucho más rápido, por la misma razón que vimos con `Set`.

Contar frecuencias:

```
const conteo = new Map();

for (const palabra of palabras) {
  conteo.set(palabra, (conteo.get(palabra) ?? 0) + 1);
}
```

Ese `?? 0` cubre la primera vez, cuando `get` devuelve `undefined`. Es el patrón que hay que reconocer en todo acumulador con `Map`.

Agrupar por una propiedad:

```
const porCiudad = new Map();

for (const c of contactos) {
  if (!porCiudad.has(c.ciudad)) porCiudad.set(c.ciudad, []);
  porCiudad.get(c.ciudad).push(c);
}
```

Desde 2024 hay una función que lo hace directamente:

```
const porCiudad = Map.groupBy(contactos, (c) => c.ciudad);
```

Existe también `Object.groupBy`, que devuelve un objeto en vez de un `Map`. Usá la versión con `Map` cuando las claves puedan no ser texto o cuando te importe el orden.

Guardar resultados ya calculados, para no repetir el trabajo:

```
const cache = new Map();

function traerContacto(id) {
  if (cache.has(id)) return cache.get(id);
  const contacto = buscarEnLaBase(id);
  cache.set(id, contacto);
  return contacto;
}
```


Objeto o Map

	Objeto	Map
Claves	texto y símbolos	cualquier valor
Cantidad	<code>Object.keys().length</code>	<code>.size</code>
Orden	números primero, después inserción	inserción siempre
Iterable	no, hay que convertirlo	sí
Se convierte a JSON	sí	no directamente
Altas y bajas frecuentes	más lento	más rápido
Claves heredadas	sí, cuidado con <code>toString</code>	no, está limpio
Sintaxis	<code>obj.clave</code> , breve	<code>map.get(clave)</code> , más larga

La regla práctica: usá un objeto cuando represente una entidad con propiedades conocidas, como un contacto. Usá un `Map` cuando sea un diccionario con claves que cambian durante la ejecución.

Hay una razón más para preferir `Map` como diccionario. Un objeto hereda propiedades que pueden confundirse con datos:

```
const dic = {};  
dic["toString"];           // una función, aunque nunca la guardaste  
  
const m = new Map();  
m.get("toString");         // undefined, como corresponde
```

Map y JSON

Un `Map` no se serializa solo:

```
JSON.stringify(new Map([["a", 1]])); // "{}", se pierde todo
```

Hay que convertirlo antes, y reconstruirlo al recibirlo:

```
const texto = JSON.stringify([...miMapa]);           // arreglo de pares  
const recuperado = new Map(JSON.parse(texto));
```

Si las claves son texto, la otra opción es pasar por objeto:

```
JSON.stringify(Object.fromEntries(miMapa));
```

Esta es la razón principal por la que los datos que van y vienen de una API se manejan como objetos, y el `Map` queda para el trabajo interno del programa.

Map y WeakMap

`WeakMap` es una variante donde las claves solo pueden ser objetos y no impiden que el recolector de basura los libere. No es iterable y no tiene `size`.

Sirve para asociar datos a un objeto mientras ese objeto viva, sin provocar pérdidas de memoria. Es lo que usan varias librerías para guardar estado privado de un componente. Alcanza con saber que existe.

Errores frecuentes

Usar corchetes en vez de `get` y `set`:

```
const m = new Map();
m["clave"] = 1;    // no da error, pero no usa el Map
m.get("clave");    // undefined
m.size;            // 0
```

Lo que hiciste fue agregarle una propiedad común al objeto `Map`. El `Map` sigue vacío.

Buscar con un objeto distinto del que guardaste. Las claves objeto se comparan por referencia.

Esperar que `JSON.stringify` conserve el contenido.

Invertir el orden de los parámetros en `forEach`: primero llega el valor.

Olvidar el valor por defecto al acumular: `conteo.get(x) + 1` da `NaN` la primera vez.

Para llevarte

- `Map` es el diccionario: clave a valor, con claves de cualquier tipo
- la interfaz es `set`, `get`, `has`, `delete`, `clear` y `size`
- compara claves con `SameValueZero`, y los objetos por referencia
- es iterable y conserva el orden de inserción

- se crea desde un arreglo de pares, y `Object.entries` y `Object.fromEntries` son el puente con el objeto
- indexar una lista con `new Map(lista.map(x => [x.id, x]))` convierte cada búsqueda en un acceso directo
- para contar y agrupar, el patrón es `map.get(k) ?? valorInicial`
- `Map.groupBy` resuelve el agrupamiento en una línea
- no se serializa a JSON: convertilo a arreglo de pares o a objeto
- usá objeto para entidades, `Map` para diccionarios dinámicos

Para probar

1. Dado un arreglo de contactos con una propiedad `ciudad`, armá un `Map` que asocie cada ciudad con el arreglo de contactos de esa ciudad. Después resóvelo de nuevo con `reduce` y un objeto, y comparalos.
2. Escribí una función `contarPalabras(texto)` que devuelva un `Map` de palabra a cantidad, ordenado de mayor a menor.
3. Convertí un objeto de configuración a `Map`, agregale una clave numérica y volvé a objeto. Explicá qué pasó con esa clave.
4. Implementá una caché con `Map` para una función lenta y medí la diferencia con `console.time`.
5. Guardá un `Map` en JSON y recuperalo, verificando que las claves numéricas sigan siendo números.
6. Explicá por qué `m["clave"] = 1` no funciona como esperás y qué pasó realmente.

Estructuras de control

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte de la cuarta clase de JavaScript

Hasta acá los programas que escribimos se ejecutan de arriba abajo, una línea después de la otra. Con eso solo se puede hacer muy poco. Un programa útil tiene que poder decidir y tiene que poder repetir.

Hay un resultado clásico de 1966, el teorema del programa estructurado, que dice algo fuerte: cualquier algoritmo que se pueda escribir se puede expresar combinando tres esquemas.

La secuencia, que es hacer una cosa después de la otra. La selección, que es elegir entre caminos. Y la iteración, que es repetir mientras se cumpla una condición.

Todo lo que veas hoy es alguna forma de esos tres. Las estructuras que trae el lenguaje no agregan poder de cómputo, agregan legibilidad. Por eso, cuando dudes entre dos formas de escribir algo, el criterio casi siempre es cuál se entiende mejor.

Al final vamos a ver las excepciones, que son el único mecanismo de la clase de hoy que rompe el flujo normal y salta de un lugar a otro.

El if

La forma básica

```
if (saldo > 0) {  
  console.log("Hay saldo disponible");  
}
```

La condición se evalúa y se convierte a booleano. Ojo con eso, porque es la lista de falsy de la segunda clase la que decide, no una comparación:

```
if (nombre) { }           // entra si nombre no es "", null, undefined  
if (contactos.length) { } // entra si el arreglo tiene elementos  
if (cantidad) { }         // NO entra si cantidad vale 0
```

Ese último caso es la trampa de siempre. Si el cero es un valor válido, comparará explícitamente:

```
if (cantidad !== undefined) { }  
if (cantidad != null) { } // cubre null y undefined juntos
```

Las llaves son opcionales cuando hay una sola instrucción. Ponelas igual, siempre.

```
// Válido, pero no lo hagas  
if (esValido) guardar();  
  
// Así  
if (esValido) {  
    guardar();  
}
```

No es una manía. En 2014 Apple tuvo una falla grave de seguridad en la verificación de certificados: una línea duplicada por error quedó fuera de un `if` sin llaves, y el código de validación dejó de ejecutarse. El compilador no vio nada raro. Las llaves cuestan dos caracteres.

else y el encadenamiento

`else` cubre el caso contrario:

```
if (saldo > 0) {  
    habilitarCompra();  
} else {  
    mostrarAviso();  
}
```

Y acá hay algo que conviene entender bien, porque después explica varias cosas: `else if` no existe como palabra del lenguaje. Lo que existe es un `if` dentro del `else`, y como el `else` admite una sola instrucción, se pueden omitir sus llaves.

```
// Lo que escribís
if (a) {
  uno();
} else if (b) {
  dos();
} else {
  tres();
}

// Lo que el lenguaje ve
if (a) {
  uno();
} else {
  if (b) {
    dos();
  } else {
    tres();
  }
}
```

De ahí sale la propiedad más importante de una cadena de `else if`: las condiciones se evalúan en orden y en cuanto una da verdadero, el resto ni se mira.

```
function categoria(edad) {
  if (edad < 13) {
    return "niño";
  } else if (edad < 18) {
    return "adolescente";
  } else if (edad < 65) {
    return "adulto";
  } else {
    return "adulto mayor";
  }
}
```

Fijate que la segunda condición no necesita escribir `edad >= 13`. Si llegó ahí es porque la primera falló, así que ya lo sabe. Escribir la condición completa sería redundante y además invitaría a que las dos se desincronicen cuando alguien cambie un número.

La contracara es que el orden importa. Si ponés la condición más general primero, las siguientes quedan muertas:

```
// Mal: nunca llega a los otros casos
if (edad < 65) {
  return "adulto";
} else if (edad < 18) {
  return "adolescente"; // inalcanzable
}
```

Regla: ordená de la condición más específica a la más general.

El anidamiento y su costo

Un `if` puede contener otro `if`. El problema es que cada nivel suma indentación y suma cosas para recordar mientras leés.

```
function procesarPedido(pedido) {
  if (pedido !== null) {
    if (pedido.items.length > 0) {
      if (pedido.cliente.activo) {
        if (hayStock(pedido.items)) {
          confirmar(pedido);
        } else {
          return "sin stock";
        }
      } else {
        return "cliente inactivo";
      }
    } else {
      return "pedido vacío";
    }
  } else {
    return "pedido inexistente";
  }
}
```

Cuando llegás a `confirmar(pedido)`, que es lo único que de verdad hace esta función, tenés cuatro condiciones abiertas en la cabeza. Y los `else` que explican cada caso quedan lejísimos de la condición que los provocó.

Esa forma se conoce como flecha o pirámide, por el dibujo que hace el código contra el margen.

Las guardas

La solución es invertir el orden. En vez de anidar el camino correcto adentro de las condiciones, chequeás lo que puede salir mal y salís de inmediato.

```
function procesarPedido(pedido) {
  if (pedido == null) return "pedido inexistente";
  if (pedido.items.length === 0) return "pedido vacío";
  if (!pedido.cliente.activo) return "cliente inactivo";
  if (!hayStock(pedido.items)) return "sin stock";

  confirmar(pedido);
}
```

Es el mismo algoritmo. Cambió la forma y cambió todo lo demás.

Cada condición está pegada a su consecuencia, así que se lee de a una y se olvida. No hay indentación. El camino correcto queda al final, sin condiciones encima. Y agregar una validación nueva es agregar una línea, no reacomodar un bloque.

Esas condiciones de salida temprana se llaman guardas o cláusulas de guarda. Es probablemente el hábito que más mejora la legibilidad del código, y no cuesta nada adoptarlo.

Dos aclaraciones. Necesitás estar dentro de una función para poder usar `return`; dentro de un bucle el equivalente es `continue` o `break`. Y si el bloque no tiene salida posible, no hay guarda que valga: ahí conviene extraer el interior a otra función.

Simplificaciones frecuentes

Hay varias formas de escribir un `if` que se pueden acortar sin perder claridad.

No compares booleanos con `true`:

```
if (esValido === true) { } // redundante
if (esValido) { }          // así
```

No armes un `if` para devolver un booleano:


```
// Innecesario
function esMayor(edad) {
  if (edad >= 18) {
    return true;
  } else {
    return false;
  }
}

// La condición ya es el resultado
function esMayor(edad) {
  return edad >= 18;
}
```

Para asignar uno de dos valores, el ternario:

```
// Con if
let etiqueta;
if (activo) {
  etiqueta = "Activo";
} else {
  etiqueta = "Inactivo";
}

// Con ternario
const etiqueta = activo ? "Activo" : "Inactivo";
```

El ternario gana porque además te deja usar `const`. Lo que no conviene es encadenarlos: tres ternarios anidados son ilegibles y para eso está el `else if`.

Para valores por defecto y accesos protegidos, lo que ya vimos:

```
// En vez de if
const nombre = nombreIngresado ?? "Anónimo";
const ciudad = contacto?.direccion?.ciudad;
```

Y cuando la cadena de `else if` compara siempre lo mismo contra valores fijos, conviene reemplazarla por una tabla:

```
// Cadena larga
function traducirEstado(estado) {
  if (estado === "pending") return "Pendiente";
  else if (estado === "paid") return "Pagado";
  else if (estado === "shipped") return "Enviado";
  else if (estado === "cancelled") return "Cancelado";
  else return "Desconocido";
}

// Tabla de despacho
const ESTADOS = {
  pending: "Pendiente",
  paid: "Pagado",
  shipped: "Enviado",
  cancelled: "Cancelado",
};

function traducirEstado(estado) {
  return ESTADOS[estado] ?? "Desconocido";
}
```

La segunda versión es datos en vez de código. Agregar un estado es agregar una línea a un objeto, y ese objeto lo podés mover a un archivo de configuración, exportarlo o recorrerlo. Cuando la cadena de condiciones es toda del tipo "si vale esto, entonces aquello", casi siempre conviene esta forma.

El while

La forma básica

```
while (condicion) {
  // se repite mientras la condición sea verdadera
}
```

El `while` evalúa la condición antes de cada vuelta. Si es falsa desde el principio, el cuerpo no se ejecuta ni una vez.

Todo bucle necesita tres cosas, aunque el `while` no te obligue a escribirlas juntas: algo que inicialice, una condición y algo que avance hacia el final. Si falta el avance, el bucle no termina más.

```
let i = 0;           // inicialización
while (i < 5) {       // condición
  console.log(i);
  i++;               // avance
}
```

Cuándo usar while

El `while` es la elección natural cuando no sabés de antemano cuántas vueltas van a ser.

Procesar una cola hasta vaciarla:

```
const pendientes = [tarea1, tarea2, tarea3];

while (pendientes.length > 0) {
  const tarea = pendientes.shift();
  ejecutar(tarea);
  // ejecutar puede agregar tareas nuevas a la cola
}
```

Reintentar hasta que salga bien o se agoten los intentos:

```
let intentos = 0;
let resultado = null;

while (resultado === null && intentos < 3) {
  resultado = intentarConexion();
  intentos++;
}
```

Avanzar por una estructura hasta llegar al final, como recorrer una lista enlazada o subir por un árbol:

```
let elemento = nodoInicial;

while (elemento !== null) {
  procesar(elemento);
  elemento = elemento.siguiente;
}
```

Y los algoritmos que reducen un valor en cada vuelta:

```
// Dígitos de un número
let n = 12345;
let suma = 0;

while (n > 0) {
  suma += n % 10;
  n = Math.floor(n / 10);
}
// suma = 15
```

El bucle infinito

Si la condición nunca se vuelve falsa, el programa se cuelga. En el navegador, la pestaña deja de responder, porque JavaScript corre en un solo hilo y ese hilo está ocupado.

```
let i = 0;
while (i < 5) {
  console.log(i);
  // falta i++, no termina nunca
}
```

Antes de escribir un `while`, preguntate qué línea hace que la condición se acerque a ser falsa. Si no la podés señalar, todavía no está terminado.

Hay un caso donde el bucle infinito es intencional: cuando la salida está adentro, con un `break`.

```
while (true) {
  const linea = leerSiguiente();
  if (linea === null) break;
  procesar(linea);
}
```

Es legítimo, pero cada `break` que agregues suma una salida más que rastrear. Con más de dos, conviene replantear la condición.

El do while

La variante `do while` evalúa la condición al final, así que el cuerpo se ejecuta al menos una vez.

```
do {
  // se ejecuta siempre, mínimo una vez
} while (condicion);
```

Fijate el punto y coma final, que en el `while` común no va.

Sirve cuando la primera vuelta es obligatoria porque de ella sale el dato que evalúa la condición. El caso típico es pedir algo hasta que sea válido:

```
let opcion;

do {
  opcion = mostrarMenu();
} while (!OPCIONES_VALIDAS.includes(opcion));
```

Con un `while` común habría que repetir la lectura antes del bucle:

```
let opcion = mostrarMenu(); // duplicado
while (!OPCIONES_VALIDAS.includes(opcion)) {
  opcion = mostrarMenu();    // duplicado
}
```

Esa duplicación es la señal de que corresponde un `do while`.

Otro caso son los reintentos con espera creciente, donde el primer intento se hace siempre:

```
let espera = 100;
let exito = false;
let intentos = 0;

do {
  exito = intentar();
  if (!exito) {
    esperar(espera);
    espera *= 2;
    intentos++;
  }
} while (!exito && intentos < 5);
```

En la práctica el `do while` se usa poco. Reconocerlo alcanza; forzarlo donde no corresponde no.

break y continue

`break` corta el bucle y sigue después de él. `continue` abandona la vuelta actual y pasa a la siguiente.

```
// break: buscar y salir
for (const contacto of contactos) {
  if (contacto.id === buscado) {
    encontrado = contacto;
    break;    // no tiene sentido seguir
  }
}

// continue: saltar los que no interesan
for (const linea of lineas) {
  if (linea.trim() === "") continue;
  if (linea.startsWith("#")) continue;
  procesar(linea);
}
```

Ese uso de `continue` es la versión de las guardas dentro de un bucle: en vez de meter todo el cuerpo adentro de un `if`, descartás lo que no sirve y seguís.

Los dos afectan solo al bucle más interno. Cuando hay bucles anidados y querés salir de los dos, JavaScript tiene etiquetas:

```
buscar:
for (let fila = 0; fila < matriz.length; fila++) {
  for (let col = 0; col < matriz[fila].length; col++) {
    if (matriz[fila][col] === objetivo) {
      posicion = [fila, col];
      break buscar;    // sale de los dos bucles
    }
  }
}
```

Existen, y hay que saber leerlas. Pero antes de usarlas, mirá si no conviene extraer la búsqueda a una función y salir con `return`, que se entiende sin explicación:

```
function buscarEnMatriz(matriz, objetivo) {
  for (let fila = 0; fila < matriz.length; fila++) {
    for (let col = 0; col < matriz[fila].length; col++) {
      if (matriz[fila][col] === objetivo) return [fila, col];
    }
  }
  return null;
}
```

Y recordá que `forEach` no acepta `break` ni `continue`. Si necesitás cortar, usá `for...of` o los métodos que ya cortan solos, como `find` y `some`.

El switch

La forma básica

`switch` compara un valor contra varias opciones fijas.

```
switch (estado) {  
  case "pending":  
    mostrar("Pendiente");  
    break;  
  case "paid":  
    mostrar("Pagado");  
    break;  
  case "shipped":  
    mostrar("Enviado");  
    break;  
  default:  
    mostrar("Desconocido");  
}
```

La comparación es estricta, con `===`. No convierte tipos, así que el `1` del `case` no coincide con el `"1"` que llega:

```
switch ("1") {  
  case 1:  
    console.log("nunca entra acá");  
    break;  
}
```

`default` cubre lo que no coincidió con ningún caso. Puede ir en cualquier posición, pero ponelo al final, porque es donde todo el mundo lo busca.

El fallthrough

Acá está lo que distingue al `switch` de una cadena de `if`, y lo que más problemas causa.

El `switch` no elige un bloque: elige un punto de entrada. Una vez que encuentra el caso que coincide, empieza a ejecutar desde ahí y sigue de largo por los casos siguientes hasta que encuentre un `break` o se termine el `switch`.

Esa continuación se llama fallthrough, o caída.

```

switch (2) {
  case 1:
    console.log("uno");
  case 2:
    console.log("dos");    // entra acá
  case 3:
    console.log("tres");   // y también ejecuta esto
  case 4:
    console.log("cuatro"); // y esto
}
// Imprime: dos, tres, cuatro

```

Por eso el `break` no es decoración. Olvidarlo es el error clásico del `switch`, y es difícil de detectar porque el código sigue funcionando: solo hace de más.

Ahora bien, el `fallthrough` tiene un uso deliberado y muy útil. Cuando varios casos comparten el mismo tratamiento, los apilás sin cuerpo:

```

switch (dia) {
  case "sábado":
  case "domingo":
    return "fin de semana";
  case "lunes":
  case "martes":
  case "miércoles":
  case "jueves":
  case "viernes":
    return "día hábil";
  default:
    return "día inválido";
}

```

Los primeros dos casos no tienen cuerpo, así que caen al siguiente. Es la forma de escribir "o esto o aquello" en un `switch`, y es la razón por la que el lenguaje mantiene este comportamiento.

Cuando la caída es intencional pero el caso sí tiene cuerpo, dejá un comentario. Es una convención que además reconocen las herramientas de análisis:


```
switch (nivel) {
  case "admin":
    permisos.push("borrar");
    // fallthrough intencional
  case "editor":
    permisos.push("editar");
    // fallthrough intencional
  case "lector":
    permisos.push("leer");
    break;
}
```

Ese ejemplo aprovecha la caída para acumular permisos: un admin termina con los tres.

El alcance dentro del switch

Todos los `case` comparten un mismo bloque. Eso trae un problema al declarar variables:

```
switch (tipo) {
  case "a":
    let x = 1; // SyntaxError al llegar al siguiente
    break;
  case "b":
    let x = 2; // redeclaración en el mismo alcance
    break;
}
```

La solución son llaves propias para el caso:

```
switch (tipo) {
  case "a": {
    let x = 1;
    break;
  }
  case "b": {
    let x = 2;
    break;
  }
}
```

Cuándo usar switch

Usalo cuando compares un mismo valor contra varias constantes y quieras agrupar casos. Con tres o cuatro opciones se lee mejor que la cadena de `else if`.

Vas a encontrar también este truco para manejar rangos:

```
switch (true) {  
  case edad < 13: return "niño";  
  case edad < 18: return "adolescente";  
  default: return "adulto";  
}
```

Funciona, porque cada `case` se compara con `true`. Pero es una forma indirecta de escribir un `else if`, y se entiende menos. Preferí el `else if`.

Y si el `switch` solo mapea valores a valores, sin lógica, volvé a la tabla de despacho que vimos con el `if`. El objeto es más corto y no tiene `break` que olvidar.

El for

Las tres partes

El `for` junta en una línea las tres partes que en el `while` estaban desparramadas.

```
for (inicializacion; condicion; avance) {  
  // cuerpo  
}
```

Y esa es exactamente la equivalencia entre los dos. Todo `for` se puede escribir como `while`:

```
// for  
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}  
  
// el mismo while  
let i = 0;  
while (i < 5) {  
  console.log(i);  
  i++;  
}
```

El orden de ejecución es este: la inicialización corre una sola vez, antes de todo. Después se evalúa la condición; si es falsa, termina. Si es verdadera, corre el cuerpo, después el avance, y vuelve a la condición.

Las tres partes son opcionales. Si las sacás todas, queda un bucle infinito:

```
for (;;) {  
  // equivalente a while (true)  
}
```

Y se pueden manejar varias variables con la coma:

```
for (let i = 0, j = arr.length - 1; i < j; i++, j--) {  
  [arr[i], arr[j]] = [arr[j], arr[i]]; // invierte el arreglo  
}
```

Cuándo for y cuándo while

La diferencia no es técnica, es de comunicación. Las tres partes del `for` están juntas y a la vista, así que quien lee sabe de entrada cuántas vueltas serán y cómo avanza.

Usá `for` cuando la inicialización, la condición y el avance hablen de lo mismo, típicamente un contador o un índice. Usá `while` cuando el avance dependa de algo que pasa adentro del cuerpo, o cuando no haya contador.

```
// for: el índice manda  
for (let i = 0; i < contactos.length; i++) { }  
  
// while: el avance depende de lo que se lea  
while ((linea = leer()) !== null) { }
```

El for of

Recorre los valores de cualquier cosa iterable. Es el que vas a usar casi siempre.

```
for (const contacto of contactos) {  
  console.log(contacto.nombre);  
}
```

Iterable significa que la estructura sabe entregar sus elementos de a uno. Son iterables los arreglos, las cadenas, los `Set`, los `Map` y el resultado de `Object.entries()`. Los objetos comunes no.

```
for (const letra of "hola") { }  
for (const valor of new Set([1, 2, 3])) { }  
for (const [clave, valor] of new Map()) { }  
for (const [clave, valor] of Object.entries(config)) { }  
for (const [i, valor] of arreglo.entries()) { }
```

Acepta `break` y `continue`, que es lo que lo diferencia de `forEach`.

El for in

Recorre las claves de un objeto. Ya vimos la clase pasada por qué no hay que usarlo con arreglos: entrega las claves como texto, incluye propiedades agregadas y saltea los huecos.

```
for (const clave in configuracion) {  
  console.log(clave, configuracion[clave]);  
}
```

Además recorre las propiedades heredadas por la cadena de prototipos, que es un tema que veremos más adelante. Si querés solo las propias:

```
for (const clave in objeto) {  
  if (!Object.hasOwn(objeto, clave)) continue;  
  // ...  
}
```

En la práctica, casi siempre conviene `Object.entries()` con `for...of`, que no tiene ninguno de estos problemas.

Un resumen de todos los bucles

Estructura	Recorre	Cortás con <code>break</code>	Usalo para
<code>while</code>	mientras se cumpla algo	sí	cantidad de vueltas desconocida
<code>do while</code>	ídem, mínimo una vez	sí	cuando la primera vuelta es obligatoria
<code>for</code>	un contador	sí	índices, saltos, recorridos al revés
<code>for...of</code>	valores de un iterable	sí	recorrer colecciones
<code>for...in</code>	claves de un objeto	sí	propiedades de un objeto
<code>.forEach()</code>	valores, con función	no	aplicar algo a cada elemento

Las excepciones

El problema que resuelven

Una función tiene un contrato: recibe algo y devuelve algo. La pregunta es qué hace cuando no puede cumplirlo.

Una opción es devolver un valor especial. Es lo que hace buena parte de la biblioteca estándar:

`indexOf` devuelve `-1`, `find` devuelve `undefined`, `Number("abc")` devuelve `NaN`.

Esa opción tiene dos problemas. El primero es que se puede ignorar en silencio: nada te obliga a chequear el `-1`, y si no lo hacés el programa sigue con un valor equivocado. El segundo es que si el error tiene que llegar tres niveles más arriba, cada nivel intermedio tiene que recibirlo y reenviarlo, contaminando funciones que no tienen nada que ver.

La excepción resuelve las dos cosas. Interrumpe la ejecución en el punto del problema y sube por la pila de llamadas hasta que alguien la atrape. Si nadie la atrapa, el programa se detiene y muestra el error.

O sea que es un salto: rompe el flujo estructurado a propósito, y por eso hay que usarla con criterio.

Lanzar

```
function retirar(cuenta, monto) {
  if (monto <= 0) {
    throw new Error("El monto debe ser positivo");
  }
  if (monto > cuenta.saldo) {
    throw new Error("Saldo insuficiente");
  }
  cuenta.saldo -= monto;
}
```

En JavaScript podés lanzar cualquier valor, no solo errores. Es una libertad que no deberías usar:

```
throw "algo salió mal";           // válido pero malo
throw new Error("algo salió mal"); // así
```

La diferencia es que un `Error` trae el nombre, el mensaje y la traza de la pila, que es la lista de funciones por las que pasó. Sin eso, depurar se vuelve adivinanza.

El lenguaje trae varios tipos de error, y los motores los usan para sus propias fallas:

Tipo	Cuándo aparece
Error	el genérico, y la base de los demás
TypeError	una operación sobre un tipo que no corresponde
RangeError	un valor fuera del rango permitido
ReferenceError	una variable que no existe
SyntaxError	código mal escrito, o <code>JSON.parse</code> con texto inválido

Un objeto de error tiene estas propiedades:

```
const error = new Error("Saldo insuficiente");

error.name;    // "Error"
error.message; // "Saldo insuficiente"
error.stack;   // el recorrido de llamadas
```

Atrapar

```
try {
  retirar(cuenta, 5000);
  console.log("Retiro exitoso");
} catch (error) {
  console.error("No se pudo retirar:", error.message);
} finally {
  cerrarSesion();
}
```

El bloque `try` contiene el código que puede fallar. Si se lanza algo, la ejecución salta al `catch` de inmediato: las líneas que quedaban en el `try` no se ejecutan.

El `finally` corre siempre, haya error o no, y también si el `try` tiene un `return`. Sirve para liberar lo que haya que liberar: cerrar una conexión, apagar un indicador de carga, destrabar un botón.

```
function cargarDatos() {
  mostrarCargando(true);
  try {
    return leerDatos();
  } catch (error) {
    registrar(error);
    return null;
  } finally {
    mostrarCargando(false); // pase lo que pase
  }
}
```

Si no te interesa el objeto del error, podés omitirlo:

```
try {
  JSON.parse(texto);
} catch {
  return null;
}
```

Distinguir tipos de error

Un `catch` atrapa todo lo que pase por ahí, así que muchas veces hay que ver qué llegó:

```
try {
  procesar(datos);
} catch (error) {
  if (error instanceof SyntaxError) {
    avisar("El archivo está mal formado");
  } else if (error instanceof RangeError) {
    avisar("Un valor está fuera de rango");
  } else {
    throw error; // no es mío, que suba
  }
}
```

Esa última línea importa. Si no sabés manejar un error, relanzalo. Atrapar todo y no hacer nada es la forma más eficaz de esconder un problema.

Errores propios

Podés definir tus propios tipos extendiendo `Error`. Sirve para que quien atrapa pueda distinguir y para llevar datos extra:

```

class ErrorDeValidacion extends Error {
  constructor(campo, mensaje) {
    super(mensaje);
    this.name = "ErrorDeValidacion";
    this.campo = campo;
  }
}

function validarContacto(contacto) {
  if (!contacto.email) {
    throw new ErrorDeValidacion("email", "El email es obligatorio");
  }
  if (!contacto.email.includes("@")) {
    throw new ErrorDeValidacion("email", "El email no es válido");
  }
}

try {
  validarContacto(datos);
} catch (error) {
  if (error instanceof ErrorDeValidacion) {
    marcarCampo(error.campo, error.message);
  } else {
    throw error;
  }
}

```

Cuando atrapás un error y lanzás otro, conservá el original con `cause`. Si no, perdés la causa real:

```

try {
  await guardarEnBase(contacto);
} catch (error) {
  throw new Error("No se pudo guardar el contacto", { cause: error });
}

```

Cuándo no usar excepciones

Acá está el criterio que da nombre a esta sección, y conviene tenerlo claro.

Una excepción es cara y rompe el flujo. Justifica ese costo cuando el problema es realmente excepcional: algo que no debería pasar, o que la función no puede resolver ni describir con un valor de retorno.

Cuando la falla es parte del funcionamiento normal, devolvé un valor.


```
// Mal: que un contacto no exista es lo más común del mundo
function buscarContacto(id) {
  const c = contactos.find((c) => c.id === id);
  if (!c) throw new Error("No encontrado");
  return c;
}

// Bien
function buscarContacto(id) {
  return contactos.find((c) => c.id === id) ?? null;
}
```

Una prueba práctica: si quien llama a tu función va a tener que envolverla en un `try` siempre, entonces eso no era una excepción, era un resultado posible.

Lo mismo con la validación de un formulario. Que el usuario escriba mal un email es lo esperable, no algo excepcional. Devolvé la lista de problemas:

```
function validar(contacto) {
  const errores = [];
  if (!contacto.nombre) errores.push({ campo: "nombre", mensaje: "Obligatorio" });
  if (!contacto.email) errores.push({ campo: "email", mensaje: "Obligatorio" });
  return errores; // vacío si está todo bien
}
```

Otra forma habitual es devolver un objeto que describa el resultado, con éxito o con error:

```
function convertirAFecha(texto) {
  const fecha = new Date(texto);
  if (Number.isNaN(fecha.getTime())) {
    return { ok: false, error: "Fecha inválida" };
  }
  return { ok: true, valor: fecha };
}

const resultado = convertirAFecha(entrada);
if (!resultado.ok) {
  mostrarError(resultado.error);
  return;
}
usarFecha(resultado.valor);
```

Ese patrón hace visible el error en la firma de la función. Nadie se lo puede olvidar, porque para llegar al valor tiene que pasar por el chequeo.

Resumiendo el criterio:

Situación	Qué corresponde
Un contacto no existe	devolver <code>null</code>
Un formulario tiene campos vacíos	devolver la lista de errores
Un texto no se puede convertir	devolver un resultado con <code>ok</code>
El servidor no responde	lanzar
Un archivo de configuración está corrupto	lanzar
Un parámetro rompe una condición que nunca debería romperse	lanzar

Errores frecuentes al usar excepciones

El `catch` vacío es el peor de todos. Silencia el problema y deja el programa en un estado inconsistente, sin que nadie se entere:

```
try {
  guardar(datos);
} catch (e) {
  // ← nunca dejes esto vacío
}
```

Si de verdad querés ignorar el error, escribí un comentario que diga por qué. Si no lo podés justificar, no lo ignores.

El `try` gigante es el otro. Si el bloque tiene cuarenta líneas y algo falla, no sabés qué falló:

```
// Difícil de diagnosticar
try {
  const datos = leer();
  const validos = validar(datos);
  const guardados = guardar(validos);
  notificar(guardados);
} catch (error) {
  console.error("algo falló");
}
```

Envolvé el tramo más chico posible, y en cada `catch` decí de qué se trata.

Por último, una advertencia sobre lo que viene. En código asíncrono, un `try` común no atrapa lo que falla dentro de una promesa ni dentro de una función que se ejecuta más tarde:

```
try {
  setTimeout(() => { throw new Error("no lo atrapa nadie"); }, 100);
} catch (error) {
  // nunca entra acá
}
```

La razón es que para cuando la función interna se ejecuta, el `try` ya terminó. Lo vamos a ver en detalle cuando lleguemos a promesas y a `async`.

Para llevarte de esta clase

- todo se construye con secuencia, selección e iteración; el resto es legibilidad
- poné llaves siempre, aunque el `if` tenga una sola línea
- `else if` es un `if` dentro del `else`: se evalúa en orden y la primera condición verdadera gana
- ordená las condiciones de la más específica a la más general
- reemplazá el anidamiento profundo por guardas que salgan temprano
- cuando la cadena de `if` solo mapea valores, usá un objeto como tabla de despacho
- `while` cuando no sabés cuántas vueltas; `do while` cuando la primera es obligatoria
- todo `for` es un `while` con las tres partes juntas
- `switch` compara con `===` y elige un punto de entrada, no un bloque: sin `break` sigue de largo
- el fallthrough sin cuerpo es la forma correcta de agrupar casos
- `for...of` para colecciones, `for...in` para objetos, `forEach` cuando no necesites cortar
- lanzá excepciones solo para lo excepcional; para lo esperable, devolvé un valor
- nunca dejes un `catch` vacío, y si no sabés manejar un error, relanzalo

Para probar antes de la próxima clase

1. Tomá una función con tres niveles de `if` anidados y reescribirla con guardas. Contá las líneas y los niveles de indentación de cada versión.
2. Escribí una cadena de seis `else if` que traduzca códigos de estado a texto, y después reescribirla como tabla de despacho.
3. Escribí un `switch` que devuelva la cantidad de días de un mes, agrupando los casos con fallthrough. Después agregale el caso de febrero en año bisiesto.
4. Convertí a `while` este `for`: `for (let i = 10; i > 0; i -= 2)`. Después convertí a `for` un `while` que recorra un arreglo saltando los elementos nulos.
5. Escribí una función `buscarEnMatriz` que devuelva la posición de un valor. Resolvela primero con etiquetas y `break`, después con `return`, y decidí cuál dejarías en el proyecto.

6. Escribí `convertirANumero(texto)` en dos versiones: una que lance una excepción y otra que devuelva `{ ok, valor, error }`. Escribí el código que las usa y compará cuál obliga a manejar el error.

Fundamentos de la computación digital

Apunte de clase

La idea central

Con dos símbolos y tres operaciones se puede construir cualquier comportamiento que sepamos describir con precisión.

Los dos símbolos son 0 y 1. Las tres operaciones son NOT, AND y OR. Todo lo demás (números, texto, imágenes, sumas, productos, decisiones, un procesador entero) sale de combinar esas piezas.

El recorrido de la clase tiene cuatro pasos:

- codificar cualquier información con ceros y unos
- describir el comportamiento deseado como una función de esos bits
- convertir esa descripción en una expresión con NOT, AND y OR
- construir esa expresión con dispositivos físicos

Codificar la información en binario

Un bit tiene 2 valores posibles. Con n bits hay 2^n combinaciones, así que alcanza con elegir una combinación distinta para cada cosa que queramos representar.

Ejemplo del dado. Un dado tiene 6 caras. Con 2 bits solo llegamos a 4 combinaciones, así que necesitamos 3 bits, que dan 8:

Cara	Código
1	000
2	001
3	010
4	011
5	100
6	101

Sobran dos códigos, 110 y 111, que no corresponden a ninguna cara. Más adelante sirven: como nunca se van a dar, podemos elegir la salida que más nos convenga para simplificar el circuito.

La codificación es arbitraria. Podríamos haber asignado los códigos en cualquier orden. Elegimos este porque coincide con la numeración binaria y después nos deja hacer cuentas.

Lo mismo vale para cualquier otra cosa: letras, colores, notas musicales, píxeles. Codificar es solo ponerse de acuerdo en una tabla.

Funciones booleanas y tablas de verdad

Una función booleana toma n bits de entrada y devuelve un bit de salida. Como hay una cantidad finita de entradas posibles, la podemos definir por enumeración: escribimos las 2^n combinaciones y al lado la salida que queremos. Eso es una tabla de verdad.

Ejemplo de los dos dados. Tiramos dos dados y queremos una función que valga 1 cuando la suma da 7:

- entradas: 3 bits del primer dado y 3 del segundo, 6 bits en total
- filas de la tabla: $2^6 = 64$
- filas con salida 1: las 6 combinaciones que suman 7, es decir 1 y 6, 2 y 5, 3 y 4, 4 y 3, 5 y 2, 6 y 1

La tabla de verdad es una definición perfecta, pero no explica nada ni se puede construir directamente. Además crece rápido: cada entrada nueva duplica la cantidad de filas.

Vale la pena ver cuánta variedad hay acá. Con n entradas, cada tabla se distingue por su columna de salida, que tiene 2^n casilleros. Entonces hay $2^{(2^n)}$ funciones distintas. Con 2 entradas son 16. Con 3, son 256. Con 6 entradas, como en los dados, hay unos $1,8 \times 10^{19}$ comportamientos posibles. Todos ellos se arman con NOT, AND y OR.

Los tres operadores básicos

Notación: escribimos $\neg a$ para la negación, $a \cdot b$ para la conjunción y $a + b$ para la disyunción. Los símbolos de producto y suma no son casuales, como se ve más abajo.

NOT invierte el valor:

a	$\neg a$
0	1
1	0

AND vale 1 solo si las dos entradas valen 1:

a	b	a · b
0	0	0
0	1	0
1	0	0
1	1	1

OR vale 1 si al menos una entrada vale 1:

a	b	a + b
0	0	0
0	1	1
1	0	1
1	1	1

Leídos en castellano: "no", "y", "o". Son los conectivos con los que ya razonamos todos los días.

De la tabla de verdad a la expresión

Existe un método mecánico para pasar de cualquier tabla de verdad a una expresión con NOT, AND y OR. Se llama forma canónica disyuntiva. También se la conoce como suma de productos canónica o desarrollo en mintérminos.

El método tiene tres pasos.

1. Mirá solo las filas donde la salida vale 1. Las filas con salida 0 se ignoran.
2. Por cada una de esas filas escribí un AND de todas las variables, negando las que en esa fila valen 0 y dejando tal cual las que valen 1. Ese producto se llama mintérmino.
3. Uní todos los mintérminos con OR.

Por qué funciona. Cada mintérmino vale 1 en una sola fila de la tabla, justamente la fila que lo generó, y vale 0 en todas las demás. Al unirlos con OR, la expresión completa vale 1 exactamente en las filas que elegimos y en ninguna otra.

De acá sale un resultado importante: NOT, AND y OR alcanzan para expresar cualquier función booleana, sin excepción. Se dice que forman un conjunto funcionalmente completo. No hay comportamiento digital, por complicado que sea, que necesite un cuarto operador.

Hay un método espejo, la forma canónica conjuntiva, que mira las filas con salida 0, arma una suma por cada una negando las variables que valen 1, y une todo con AND. Conviene cuando los ceros son pocos.

Ejemplo completo con dados de 4 caras

El caso de los dos dados de 6 caras necesita una tabla de 64 filas, incómoda para el pizarrón. Con dados de 4 caras la cuenta es la misma y entra en 16 filas. Ganamos si la suma da 5.

Cada dado necesita 2 bits. Codificamos 1 como 00, 2 como 01, 3 como 10 y 4 como 11. Llamamos a1 a0 a los bits del primer dado y b1 b0 a los del segundo.

a1	a0	b1	b0	A	B	A+B	gané
0	0	0	0	1	1	2	0
0	0	0	1	1	2	3	0
0	0	1	0	1	3	4	0
0	0	1	1	1	4	5	1
0	1	0	0	2	1	3	0
0	1	0	1	2	2	4	0
0	1	1	0	2	3	5	1
0	1	1	1	2	4	6	0
1	0	0	0	3	1	4	0
1	0	0	1	3	2	5	1
1	0	1	0	3	3	6	0
1	0	1	1	3	4	7	0
1	1	0	0	4	1	5	1
1	1	0	1	4	2	6	0
1	1	1	0	4	3	7	0
1	1	1	1	4	4	8	0

Aplicamos el método a las 4 filas que valen 1:

$$\text{gané} = \neg a1 \cdot \neg a0 \cdot b1 \cdot b0 + \neg a1 \cdot a0 \cdot b1 \cdot \neg b0 + a1 \cdot \neg a0 \cdot \neg b1 \cdot b0 + a1 \cdot a0 \cdot \neg b1 \cdot \neg b0$$

Son 4 productos de 4 literales cada uno. Funciona, pero es caro: mucha compuerta para algo que intuitivamente es simple. Acá entra el álgebra.

El álgebra de Boole

Las expresiones booleanas se pueden manipular con reglas, igual que las expresiones numéricas. Eso permite reescribir una función en una forma equivalente más corta, o sea con menos compuertas.

Leyes básicas, cada una con su versión dual:

Ley	Con AND	Con OR
Identidad	$a \cdot 1 = a$	$a + 0 = a$
Anulación	$a \cdot 0 = 0$	$a + 1 = 1$
Idempotencia	$a \cdot a = a$	$a + a = a$
Complemento	$a \cdot \neg a = 0$	$a + \neg a = 1$
Conmutativa	$a \cdot b = b \cdot a$	$a + b = b + a$
Asociativa	$(a \cdot b) \cdot c = a \cdot (b \cdot c)$	$(a + b) + c = a + (b + c)$
Distributiva	$a \cdot (b + c) = a \cdot b + a \cdot c$	$a + b \cdot c = (a + b) \cdot (a + c)$
Absorción	$a \cdot (a + b) = a$	$a + a \cdot b = a$
De Morgan	$\neg(a \cdot b) = \neg a + \neg b$	$\neg(a + b) = \neg a \cdot \neg b$

Y la doble negación: $\neg\neg a = a$.

La distributiva del AND sobre el OR es la que ya conocemos de la aritmética, y por eso llamamos producto al AND y suma al OR. La otra distributiva, la del OR sobre el AND, no tiene análogo con números: $2 + 3 \times 4$ no es $(2+3) \times (2+4)$. En el álgebra de Boole sí vale, y es una herramienta de simplificación muy potente.

Volvamos al ejemplo. Agrupamos el primer término con el segundo, que comparten $\neg a1 \cdot b1$, y el tercero con el cuarto, que comparten $a1 \cdot \neg b1$:

$$\text{gané} = \neg a1 \cdot b1 \cdot (\neg a0 \cdot b0 + a0 \cdot \neg b0) + a1 \cdot \neg b1 \cdot (\neg a0 \cdot b0 + a0 \cdot \neg b0)$$

Ahora el paréntesis es común a los dos sumandos, así que lo sacamos como factor:

$$\text{gané} = (\neg a1 \cdot b1 + a1 \cdot \neg b1) \cdot (\neg a0 \cdot b0 + a0 \cdot \neg b0)$$

La expresión $\neg x \cdot y + x \cdot \neg y$ vale 1 cuando x e y son distintos. Es tan frecuente que tiene nombre propio, XOR, y se escribe $x \oplus y$. Entonces:

$$\text{gané} = (a1 \oplus b1) \cdot (a0 \oplus b0)$$

Pasamos de 4 productos de 4 literales a dos XOR y un AND. Y el resultado se lee en castellano: ganamos cuando los dos códigos difieren en todos sus bits. Tiene sentido, porque sumar 5 con dados de 4 caras significa que un dado tiene que dar exactamente lo que le falta al otro.

La tabla de verdad enumera, el álgebra explica. Ese es el salto.

Las leyes de De Morgan

Las dos leyes de De Morgan merecen párrafo aparte porque las vamos a usar todo el tiempo:

$$\begin{aligned}\neg(a \cdot b) &= \neg a + \neg b \\ \neg(a + b) &= \neg a \cdot \neg b\end{aligned}$$

En palabras: negar un AND es lo mismo que hacer un OR de las partes negadas, y viceversa. "No es cierto que ambas cosas pasen" equivale a "alguna de las dos no pasa".

Sirven para tres cosas:

- mover las negaciones hacia adentro o hacia afuera de una expresión
- convertir cualquier expresión con AND en una con OR, y al revés
- justificar que NAND y NOR alcanzan solas para todo, como vemos al final

Del papel al circuito

Todo lo anterior es matemática. El paso decisivo es que se puede construir con materia.

Necesitamos un dispositivo que deje pasar o corte la corriente según una señal de control. El relé es el ejemplo más fácil de ver: una bobina que, al recibir corriente, atrae un contacto y cierra o abre el circuito. Es un interruptor manejado por otra señal, no por una mano.

Con eso, las tres operaciones son configuraciones de cableado:

- AND: dos contactos en serie. La corriente llega al final solo si ambos están cerrados.
- OR: dos contactos en paralelo. Alcanza con que uno esté cerrado.
- NOT: un contacto normalmente cerrado, que se abre cuando la bobina recibe señal.

Como AND, OR y NOT alcanzan para cualquier función, y cada uno se puede armar con relés, se sigue que cualquier función booleana se puede construir físicamente. Ese fue el resultado de la tesis de maestría de Claude Shannon en 1937: la conexión entre el álgebra de Boole, de 1854, y los circuitos de conmutación.

El relé después se reemplazó por la válvula, y la válvula por el transistor. El dispositivo cambió, la velocidad cambió en varios órdenes de magnitud, el álgebra es la misma. Un procesador actual es esta idea repetida decenas de miles de millones de veces.

Al circuito que implementa una operación básica lo llamamos compuerta.

Números en base 2

Hasta acá los bits eran códigos arbitrarios. Si elegimos la codificación con cuidado, además podemos calcular.

En base 10 cada posición vale 10 veces más que la anterior, y usamos 10 dígitos. En base 2 cada posición vale 2 veces más que la anterior, y usamos 2 dígitos. Es la misma idea con otro tamaño de paquete.

$$1011 \text{ en base } 2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 11 \text{ en base } 10$$

Con n bits representamos los números de 0 a $2^n - 1$. Con 8 bits, de 0 a 255. Con 32 bits, más de 4 mil millones.

La conversión al revés se hace dividiendo por 2 y anotando los restos de abajo hacia arriba. Es el mismo procedimiento que usaríamos en decimal dividiendo por 10.

Suma binaria

Sumamos como en la escuela: columna por columna, de derecha a izquierda, arrastrando el acarreo. La única diferencia es que la tabla de sumar tiene 4 entradas en vez de 100:

$$\begin{array}{l} 0 + 0 = 0 \\ 0 + 1 = 1 \\ 1 + 0 = 1 \\ 1 + 1 = 10 \quad (\text{escribo } 0 \text{ y llevo } 1) \end{array}$$

Escribamos la tabla de verdad de una columna, sin acarreo de entrada, con s como el bit de resultado y c como el acarreo que sale:

a	b	s	c
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Aplicando el método de los minterminos a cada columna de salida:

$$s = \neg a \cdot b + a \cdot \neg b = a \oplus b$$

$$c = a \cdot b$$

Eso es todo: un XOR y un AND. Se lo llama semisumador. Aparece el mismo XOR que apareció en el ejemplo de los dados, lo cual no sorprende, porque preguntar si dos bits difieren y sumarlos sin acarreo son la misma pregunta.

Para las columnas del medio hace falta también recibir el acarreo de la columna anterior. Con tres entradas, a, b y cin, la deducción es igual y da:

$$s = a \oplus b \oplus cin$$

$$cout = a \cdot b + cin \cdot (a \oplus b)$$

Esto es el sumador completo. El acarreo sale cuando los dos bits valen 1, o cuando venía un acarreo y al menos uno de los bits lo propaga.

Para sumar números de 4 bits ponemos 4 sumadores completos en fila y conectamos cada acarreo de salida con el de entrada del siguiente. Para 64 bits, lo mismo con 64. La estructura no cambia, solo se repite.

Producto binario

También se calcula como en la escuela, y sale más fácil que en decimal. Multiplicamos por cada dígito del segundo factor y sumamos los productos parciales desplazados.

La ventaja es que en base 2 no hay tabla de multiplicar. Cada dígito es 0 o 1, así que cada producto parcial es 0 o el primer factor tal cual, corrido tantas posiciones como indique la posición del dígito.

```

    1011    (11)
  ×  110    (6)
  -----
    0000    1011 × 0
    1011    1011 × 1, corrido 1
    1011    1011 × 1, corrido 2
  -----
  1000010  (66)

```

Un producto parcial se obtiene con un AND, un desplazamiento es solo cablear las señales corridas de lugar, y sumar ya sabemos hacerlo. Así que multiplicar no necesita nada nuevo: es AND más suma.

NAND y NOR, compuertas universales

En la práctica se usa un juego todavía más chico. NAND es un AND negado, y NOR es un OR negado:

a	b	a NAND b	a NOR b
0	0	1	1
0	1	1	0
1	0	1	0
1	1	0	0

Con NAND sola se arman las tres operaciones básicas:

```

¬a      = a NAND a
a · b   = ¬(a NAND b) = (a NAND b) NAND (a NAND b)
a + b   = ¬a NAND ¬b  = (a NAND a) NAND (b NAND b)

```

La última línea es De Morgan aplicada: $\neg(\neg a \cdot \neg b) = a + b$.

Con NOR pasa lo mismo, en espejo:

```

¬a      = a NOR a
a + b   = (a NOR b) NOR (a NOR b)
a · b   = (a NOR a) NOR (b NOR b)

```

Por eso se las llama compuertas universales. Cada una alcanza, sola, para construir cualquier circuito digital.

Que sean negadas no es un capricho. En la tecnología con la que se fabrican los chips, la compuerta natural es la negada: el AND se construye como un NAND seguido de un inversor. Fabricar todo con NAND usa menos transistores y simplifica el proceso.

Resumen

El camino completo, en una línea por paso:

- cualquier información se codifica con ceros y unos, eligiendo una tabla de equivalencias
- cualquier comportamiento se describe como una función que va de bits a bits
- cualquier función se puede escribir por enumeración en una tabla de verdad
- cualquier tabla de verdad se convierte en una expresión con NOT, AND y OR, por el método de los minterminos
- cualquier expresión se puede simplificar con el álgebra de Boole
- cualquier expresión se puede construir con dispositivos físicos, porque serie es AND, paralelo es OR y un contacto invertido es NOT
- alcanza incluso con una sola compuerta, NAND o NOR

De ahí salen los números, la suma, el producto y, subiendo capas, todo lo demás. La riqueza no viene de tener muchas piezas, sino de componer pocas piezas muchas veces.

Para pensar

1. Escribí la tabla de verdad de la función que indica si un dado de 6 caras sacó un número par. Deducí la expresión y simplificala.
2. Los códigos 110 y 111 no corresponden a ninguna cara del dado. Elegí para esas filas la salida que más te convenga y mostrá cuánto se acorta la expresión del ejercicio anterior.
3. Construí un XOR usando solo compuertas NAND. ¿Cuántas necesitás?
4. Verificá las dos leyes de De Morgan armando las tablas de verdad de los dos lados de cada igualdad.
5. Diseñá un circuito que compare dos números de 2 bits y devuelva 1 si son iguales. Compará tu solución con la del ejemplo de los dados.

El tipo `symbol` en JavaScript

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte complementario de la unidad de JavaScript

Cuando vimos los tipos básicos dejamos uno afuera. Los seis que ya conocés guardan un dato: un número guarda una cantidad, una cadena guarda texto, un booleano guarda una respuesta. El séptimo no guarda ningún dato. Guarda una identidad.

Ese es `symbol`, y este apunte trata sobre dos cosas. Primero, qué problema resuelve un valor que no vale por su contenido sino por ser él mismo. Segundo, los símbolos especiales, que son el mecanismo con el que JavaScript te deja modificar el comportamiento del propio lenguaje.

Guardá esta idea para el final, porque es la conclusión: los símbolos especiales son lo más parecido que tiene JavaScript a las interfaces de C#.

El problema que resuelve

Supongamos que escribís una librería de caché. Recibís objetos de otra persona y necesitás marcarlos con un identificador interno.

```
const usuario = { nombre: "Ada", email: "ada@ejemplo.com" };  
  
usuario.id = 42; // tu marca interna
```

El código funciona hasta que alguien te pasa un objeto que ya tiene un `id`. Ahí lo pisás. Y aunque elijas un nombre más raro, como `__cacheId__`, seguís apostando a que nadie más eligió el mismo.

El problema de fondo es que las claves de un objeto son cadenas, y dos cadenas iguales son la misma clave. No hay forma de fabricar un nombre y garantizar que sea tuyo.

Salvo que el nombre no sea una cadena:

```
const idCache = Symbol("idCache");  
  
usuario[idCache] = 42;
```

Ahora la clave no es el texto "idCache". Es un valor único que existe en tu módulo. Nadie puede escribirlo por accidente, porque no se puede escribir: solo se puede tener una referencia a él.

Crear un símbolo

Un símbolo se crea llamando a la función `Symbol`:

```
const s1 = Symbol();
const s2 = Symbol("descripción para depurar");

typeof s2; // "symbol"
s2.description; // "descripción para depurar"
s2.toString(); // "Symbol(descripción para depurar)"
```

La descripción es opcional y sirve solo para leer mejor los mensajes de error y la consola. No participa de la identidad:

```
Symbol("id") === Symbol("id"); // false
```

Esas son dos llamadas, así que son dos símbolos distintos, aunque digan lo mismo. Un símbolo solo es igual a sí mismo.

No se usa `new`:

```
new Symbol(); // TypeError: Symbol is not a constructor
```

Es un primitivo, no un objeto, igual que un número o un booleano. La diferencia es que no hay literal para escribirlo. No existe algo como `42` o `"hola"` para los símbolos. La única forma de obtener uno es llamar a la función y guardar el resultado.

Acordate del modelo mental de la clase de variables: la variable no es una caja, es una flecha que apunta a un valor. Con los símbolos eso se vuelve literal. El valor no tiene contenido que mirar, así que lo único que existe es la flecha. Si perdés la referencia, perdiste el símbolo para siempre.

No se convierte solo a texto

Este es el punto donde el tipo sorprende:


```
const s = Symbol("hola");

console.log("valor: " + s); // TypeError
console.log(`valor: ${s}`); // TypeError

console.log(String(s)); // "Symbol(hola)"
console.log(s.toString()); // "Symbol(hola)"
console.log(s); // funciona: console.log no concatena
```

Es el único primitivo que se niega a la conversión implícita a cadena. Y es a propósito.

Pensá qué pasaría si se convirtiera solo. Escribís `objeto["" + miSimbolo]` por descuido y en vez de un error obtenés una propiedad con la clave `"Symbol(hola)"`, que es una cadena común y corriente. Perdiste la unicidad sin enterarte. El lenguaje prefiere gritar.

La conversión explícita con `String` sí funciona, porque ahí estás diciendo que sabés lo que hacés. La conversión a número no funciona nunca:

```
Number(s); // TypeError
Boolean(s); // true, todos los símbolos son truthy
```

Discretos, no privados

En JavaScript solo hay dos tipos de clave de propiedad: cadenas y símbolos. Los índices numéricos de un arreglo también son cadenas por debajo.

Un símbolo se usa como clave con la sintaxis de índice, y también dentro de un literal de objeto entre corchetes:

```
const interno = Symbol("interno");

const config = {
  host: "localhost",
  puerto: 5432,
  [interno]: "esto no se serializa",
};

config[interno]; // "esto no se serializa"
```

Las propiedades con clave de símbolo son invisibles para casi todos los recorridos:

```
for (const clave in config) console.log(clave); // host, puerto
Object.keys(config); // ["host", "puerto"]
JSON.stringify(config); // {"host":"localhost","puerto":5432}
```

Pero no están escondidas. Quien las busque, las encuentra:

```
Object.getOwnPropertySymbols(config); // [Symbol(interno)]
Reflect.ownKeys(config); // ["host", "puerto", Symbol(interno)]
```

De ahí el título de la sección. Un símbolo no te da privacidad: te da discreción. Si lo que querés es que un campo sea inaccesible desde afuera, usá campos privados de clase con `#`, que sí son inalcanzables.

Hay una excepción que sorprende y conviene tener presente: la copia sí los arrastra.

```
const copia = { ...config };
Object.getOwnPropertySymbols(copia); // [Symbol(interno)]
```

Tanto el operador de propagación como `Object.assign` copian las propiedades de símbolo propias y enumerables. Solo la serialización las ignora.

El registro global de símbolos

A veces necesitás lo contrario de la unicidad: que dos partes del sistema obtengan el mismo símbolo sin compartir una variable. Para eso está el registro global.

```
const a = Symbol.for("agenda.id");
const b = Symbol.for("agenda.id");

a === b; // true
Symbol.keyFor(a); // "agenda.id"
Symbol.keyFor(Symbol("agenda.id")); // undefined
```

`Symbol.for` busca la clave en un registro global y devuelve el símbolo existente, o crea uno nuevo si no está. `Symbol.keyFor` hace el camino inverso, y devuelve `undefined` si el símbolo no vino del registro.

Sirve cuando dos copias de la misma librería conviven en un proyecto, o cuando una página y un `iframe` tienen que ponerse de acuerdo. La convención es prefijar la clave con el nombre del paquete, porque el registro es global de verdad y ahí las colisiones vuelven a existir.

Los símbolos especiales

Hasta acá los símbolos servían para que vos evitaras colisiones. Ahora viene el uso que justifica el tipo.

JavaScript tiene un puñado de símbolos ya creados, guardados como propiedades de la función `Symbol`. Se llaman símbolos bien conocidos. Cada uno marca un punto donde el lenguaje pregunta antes de actuar. Si tu objeto tiene una propiedad con ese símbolo, el motor la usa en vez de su comportamiento por omisión.

Acá está el paralelo con lo que ya saben de Programación III. En C#, para que un objeto se pueda recorrer con `foreach`, implementás `IEnumerable<T>`. Para que se pueda liberar con `using`, implementás `IDisposable`. El compilador reconoce la interfaz y habilita la sintaxis.

JavaScript no tiene interfaces. Lo que tiene son símbolos bien conocidos. Implementar `Symbol.iterator` es lo que en C# sería implementar `IEnumerable<T>`: la sintaxis del lenguaje empieza a funcionar sobre tu objeto.

Antes de ES6 esto se hacía con nombres mágicos, como `toString` y `valueOf`. El problema es evidente: si tu objeto necesitaba una propiedad llamada `toString` para otra cosa, estabas en problemas. Los símbolos resolvieron eso de raíz.

Symbol.iterator

Es el más importante de todos, y ya lo venís usando sin saberlo.

```
const rango = {
  desde: 1,
  hasta: 5,

  [Symbol.iterator]() {
    let actual = this.desde;
    const hasta = this.hasta;

    return {
      next: () =>
        actual <= hasta
          ? { value: actual++, done: false }
          : { value: undefined, done: true },
    };
  },
};
```

Con eso, un objeto cualquiera pasa a funcionar en todos los lugares donde esperabas un arreglo:

```
for (const n of rango) console.log(n); // 1 2 3 4 5

[...rango]; // [1, 2, 3, 4, 5]
Array.from(rango); // [1, 2, 3, 4, 5]
Math.max(...rango); // 5

const [primero, segundo] = rango; // 1, 2
```

El contrato es simple. El método devuelve un objeto con un método `next`, y cada llamada a `next` devuelve un objeto con `value` y `done`. Nada más.

Con un generador queda mucho más corto, porque el generador arma ese objeto por vos:

```
const rango = {
  desde: 1,
  hasta: 5,

  *[Symbol.iterator]() {
    for (let n = this.desde; n <= this.hasta; n++) yield n;
  },
};
```

Ahora mirá para atrás. Los arreglos, las cadenas, los `Set` y los `Map` que vimos en la clase de tipos compuestos se recorren con `for...of` porque todos traen `Symbol.iterator` implementado. No es una regla del `for...of`: es una propiedad de esos objetos.

Y por eso `for...in` no sirve para lo mismo. `for...in` recorre las claves de un objeto y no consulta ningún símbolo. Son dos mecanismos distintos que se parecen en la sintaxis.

Symbol.asyncIterator

La versión asíncrona del mismo contrato. El método devuelve promesas en lugar de valores, y se recorre con `for await`:

```
const cuentaRegresiva = {
  async *[Symbol.asyncIterator]() {
    for (let n = 3; n > 0; n--) {
      await new Promise((listo) => setTimeout(listo, 1000));
      yield n;
    }
  },
};

for await (const n of cuentaRegresiva) console.log(n); // 3, 2, 1
```

Lo vas a volver a ver cuando trabajemos con respuestas HTTP que llegan de a pedazos. Un cuerpo de respuesta se lee así.

Symbol.toPrimitive

Este conecta directo con la clase de conversión de tipos. Cuando JavaScript necesita un primitivo y le diste un objeto, primero consulta este símbolo.

```
class Dinero {
  constructor(monto, moneda) {
    this.monto = monto;
    this.moneda = moneda;
  }

  [Symbol.toPrimitive](contexto) {
    if (contexto === "number") return this.monto;
    return `${this.moneda} ${this.monto.toFixed(2)}`;
  }
}

const precio = new Dinero(1500, "ARS");

+precio; // 1500
precio * 2; // 3000
`${precio}`; // "ARS 1500.00"
precio + ""; // "ARS 1500.00"
```

El parámetro que recibe el método dice para qué se está pidiendo la conversión, y tiene tres valores posibles:

- `"number"` cuando la operación es aritmética, como `+valor`, `valor * 2` o una comparación con `<`
- `"string"` cuando se necesita texto, como en una interpolación o en `String(valor)`
- `"default"` cuando el operador acepta cualquiera de los dos, que en la práctica es el `+` binario y el `==`

Sin este símbolo, el motor usa `valueOf` y `toString` en un orden que depende del contexto. Con él, no hay ambigüedad: `Symbol.toPrimitive` le gana a los dos.

Si venís de C#, esto es una conversión implícita definida por el usuario, con la diferencia de que acá el mismo método atiende todos los destinos y se entera del contexto por parámetro.

Symbol.toStringTag

Define la etiqueta que aparece en la representación interna de un objeto:

```
Object.prototype.toString.call([]); // "[object Array]"
Object.prototype.toString.call(new Map()); // "[object Map]"

class Agenda {
  get [Symbol.toStringTag]() {
    return "Agenda";
  }
}

Object.prototype.toString.call(new Agenda()); // "[object Agenda]"
String(new Agenda()); // "[object Agenda]"
```

Es el mecanismo detrás del viejo truco de usar `Object.prototype.toString.call(valor)` para distinguir tipos que `typeof` no distingue, porque `typeof` devuelve `"object"` para un arreglo, una fecha y un mapa por igual.

Symbol.hasInstance

Redefine qué contesta `instanceof`:

```
class Par {
  static [Symbol.hasInstance](valor) {
    return typeof valor === "number" && valor % 2 === 0;
  }
}

4 instanceof Par; // true
7 instanceof Par; // false
```

Fijate que `4` no es un objeto y nunca fue construido con `Par`. Normalmente `instanceof` recorre la cadena de prototipos, pero acá esa pregunta la contestás vos. Es como poder redefinir el operador `is` de C#.

Usalo con cuidado. Un `instanceof` que miente es difícil de depurar justamente porque nadie sospecha del operador.

Symbol.dispose

Es el más nuevo y el que más les va a sonar. Marca cómo se libera un recurso cuando sale de alcance, y se combina con la declaración `using`:

```
class Conexion {
  constructor(nombre) {
    this.nombre = nombre;
    console.log("abro", nombre);
  }

  [Symbol.dispose]() {
    console.log("cierro", this.nombre);
  }
}

function consultar() {
  using conexion = new Conexion("agenda");
  // ... trabajo con la conexión
} // acá se llama solo a Symbol.dispose
```

Es `IDisposable` con `using`, tal cual lo vieron en C#, y llega quince años más tarde. También existe `Symbol.asyncDispose`, que se usa con `await using` para liberaciones que devuelven una promesa.

Es reciente: funciona en Node 24 y en los navegadores actuales, pero todavía no lo vas a encontrar en código viejo.

Los que quedan

Los demás símbolos bien conocidos aparecen poco en código de aplicación, pero conviene saber qué operación intercepta cada uno:

Símbolo	Qué operación intercepta
<code>Symbol.iterator</code>	<code>for...of</code> , propagación, desestructuración
<code>Symbol.asyncIterator</code>	<code>for await...of</code>
<code>Symbol.toPrimitive</code>	conversión a número o a cadena
<code>Symbol.toStringTag</code>	<code>Object.prototype.toString</code>
<code>Symbol.hasInstance</code>	<code>instanceof</code>
<code>Symbol.dispose</code>	<code>using</code>
<code>Symbol.asyncDispose</code>	<code>await using</code>
<code>Symbol.species</code>	qué constructor usan <code>map</code> , <code>filter</code> y <code>slice</code>
<code>Symbol.isConcatSpreadable</code>	si <code>concat</code> aplanar el valor
<code>Symbol.match</code>	<code>String.prototype.match</code>
<code>Symbol.matchAll</code>	<code>String.prototype.matchAll</code>
<code>Symbol.replace</code>	<code>String.prototype.replace</code>
<code>Symbol.search</code>	<code>String.prototype.search</code>
<code>Symbol.split</code>	<code>String.prototype.split</code>
<code>Symbol.unscopables</code>	qué nombres ignora la sentencia <code>with</code>

El resto de esta sección los explica uno por uno. No hace falta que los memorices: alcanza con que los reconozcas cuando aparezcan en el código de una librería.

Symbol.species

Decide qué constructor usan los métodos que devuelven una colección nueva. En un arreglo son `map` , `filter` , `slice` , `splice` y `concat` . En una promesa son `then` , `catch` y `finally` .

El problema aparece cuando extendés un tipo del lenguaje:

```
class Registro extends Array {}

const r = Registro.from([1, 2, 3]);

r instanceof Registro; // true
r.map((n) => n * 2) instanceof Registro; // true
```


Por omisión, `map` construye el resultado con el constructor del objeto original, así que te devuelve un `Registro` y no un arreglo común. Casi siempre es lo que querés. A veces no, porque tu constructor pide argumentos que `map` no le va a pasar, o porque el resultado ya no es un registro sino una lista de números.

`Symbol.species` te deja cambiar esa decisión, y se declara como un getter estático:

```
class Registro extends Array {
  static get [Symbol.species]() {
    return Array;
  }
}

const r = Registro.from([1, 2, 3]);

r instanceof Registro; // true
r.map(n => n * 2) instanceof Registro; // false
r.map(n => n * 2) instanceof Array; // true
```

El objeto original sigue siendo un `Registro`. Lo que cambió es el tipo de las copias que produce.

Es el símbolo menos querido de la lista. Complica los motores, casi nadie lo usa, y hubo propuestas en el comité para sacarlo del lenguaje. Sigue ahí porque quitarlo rompería código existente.

Symbol.isConcatSpreadable

Controla si `concat` aplana un valor o lo agrega entero. La regla por omisión mira si el valor es un arreglo:

```
[1, 2].concat([3, 4]); // [1, 2, 3, 4] lo aplana
[1, 2].concat("34"); // [1, 2, "34"] lo agrega entero
```

Este símbolo invierte cada caso. Un arreglo que no quiere aplanarse:

```
const bloque = [3, 4];
bloque[Symbol.isConcatSpreadable] = false;

[1, 2].concat(bloque); // [1, 2, [3, 4]]
```

Y un objeto que no es un arreglo pero quiere comportarse como uno:

```
const falsoArreglo = {
  0: "a",
  1: "b",
  length: 2,
  [Symbol.isConcatSpreadable]: true,
};

["inicio"].concat(falsoArreglo); // ["inicio", "a", "b"]
```

Para que el segundo caso funcione, el objeto necesita la propiedad `length` y claves numéricas, porque `concat` recorre de `0` a `length - 1`.

Ojo con el alcance: este símbolo solo lo consulta `concat`. Ni `flat`, ni el operador de propagación, ni `for...of` lo miran.

Los cinco símbolos de expresiones regulares

Empecemos con una pregunta que parece tonta. Cuando escribís `"a1b2".split(/\d/)`, quién hace el trabajo.

La respuesta intuitiva es que lo hace `split`, y que las expresiones regulares son un caso especial contemplado adentro del método. La respuesta real es la contraria: `split` delega.

`String.prototype.split` recibe el separador y antes de hacer nada le pregunta si tiene un método en `Symbol.split`. Si lo tiene, lo llama y devuelve lo que ese método devuelva. Solo si no lo tiene, aplica el algoritmo de separar por texto.

Por eso funciona con expresiones regulares: `RegExp.prototype` implementa `Symbol.split`. La expresión regular no es un caso especial dentro de `split`, es un objeto que implementa el símbolo. Y como el símbolo es público, cualquier objeto tuyo puede hacer lo mismo.

Los cinco métodos de `String` que delegan así son estos:

Llamada	Símbolo que consulta	Qué recibe	Qué debe devolver
<code>texto.match(x)</code>	<code>x[Symbol.match]</code>	el texto	las coincidencias o <code>null</code>
<code>texto.matchAll(x)</code>	<code>x[Symbol.matchAll]</code>	el texto	un iterador de coincidencias
<code>texto.replace(x, r)</code>	<code>x[Symbol.replace]</code>	el texto y el reemplazo	el texto resultante
<code>texto.search(x)</code>	<code>x[Symbol.search]</code>	el texto	el índice, o <code>-1</code>
<code>texto.split(x, n)</code>	<code>x[Symbol.split]</code>	el texto y el límite	un arreglo

Un ejemplo que resuelve algo real. Separar por varios caracteres a la vez es incómodo con una expresión regular cuando los separadores vienen en tiempo de ejecución, porque hay que escaparlos. Con `Symbol.split` armás un objeto que lo hace:

```
class Separadores {
  constructor(...caracteres) {
    this.caracteres = caracteres;
  }

  [Symbol.split](texto) {
    let partes = [texto];

    for (const caracter of this.caracteres) {
      partes = partes.flatMap((parte) => parte.split(caracter));
    }

    return partes.filter((parte) => parte !== "");
  }
}

"uno,dos;tres|cuatro".split(new Separadores(",", ";", "|"));
// ["uno", "dos", "tres", "cuatro"]
```

El objeto no es una expresión regular, no hereda de `RegExp` y no tiene nada que ver con ella. Encaja igual, porque el contrato es el símbolo y no el tipo. Esto es exactamente lo mismo que pasaba con `Symbol.iterator`, y es la idea que se repite en todo el apunte.

`Symbol.match` tiene además un segundo trabajo, y es el que explica un error raro. Los métodos `startsWith`, `endsWith` e `includes` se niegan a recibir una expresión regular:

```
"decí hola".startsWith(/hola/); // TypeError
```

Para averiguar si el argumento es una expresión regular, el lenguaje no mira el tipo: mira si tiene `Symbol.match`. Así que se puede engañar:

```
const re = /hola/;
re[Symbol.match] = false;

"decí hola".startsWith(re); // false, y ya no lanza error
"/hola/".startsWith(re); // true
```

Al desactivar el símbolo, el objeto deja de ser reconocido como expresión regular, y `startsWith` lo convierte a texto: compara contra la cadena `"/hola/"`. Es una curiosidad, no una técnica. Sirve para entender que en JavaScript los tipos del lenguaje se reconocen por lo que implementan, no por lo que son.

Symbol.unscopables

Este es el único de la lista que no vas a usar nunca, y aun así vale la pena por lo que enseña.

Declara qué propiedades de un objeto quedan afuera cuando ese objeto entra en un bloque `with`. La sentencia `with` mete las propiedades de un objeto en el alcance del bloque, está prohibida en modo estricto, y como los módulos son siempre estrictos, en la práctica ya no existe.

El problema que resolvió fue de compatibilidad. Cuando ES6 agregó métodos nuevos a `Array.prototype`, como `includes`, `values` o `keys`, había páginas en producción con código así:

```
const includes = "soy una variable externa";
const numeros = [1, 2, 3];

with (numeros) {
  console.log(includes);
}
```

Antes de ES6 ese `includes` se resolvía a la variable externa, porque los arreglos no tenían un método con ese nombre. El día que el método existió, el mismo código pasó a resolverse al método y las páginas se rompieron.

La solución fue una lista de excepciones publicada en el propio prototipo:

```
Array.prototype[Symbol.unscopables];
// { at: true, copyWithin: true, entries: true, fill: true,
//   find: true, findIndex: true, flat: true, flatMap: true,
//   includes: true, keys: true, values: true, ... }
```

Con esa lista, `with` saltea esos nombres y el ejemplo de arriba vuelve a imprimir la variable externa.

Guarda el caso como ejemplo de la primera clase, cuando hablamos de que JavaScript no puede romper la web. Agregar un método a un prototipo del lenguaje parece gratis y no lo es. El comité tuvo que inventar un símbolo entero para poder agregar `includes` a los arreglos sin voltear sitios que ya andaban.

El tipo symbol en TypeScript

Cuando pasemos a TypeScript vas a encontrar dos tipos, y la diferencia importa:

```
let cualquiera: symbol = Symbol();

const CLAVE: unique symbol = Symbol("clave");
```

El tipo `symbol` describe a cualquier símbolo. El tipo `unique symbol` describe a uno en particular, y es el único que podés usar como clave en la declaración de un tipo:

```
interface Config {
  host: string;
  [CLAVE]: string;
}
```

`unique symbol` solo se puede declarar con `const` y asignando directamente `Symbol()` o `Symbol.for()`. La razón es que TypeScript necesita poder rastrear ese símbolo hasta una única declaración, y una variable reasignable no le da esa garantía.

Cuándo usarlo

Usá símbolos para tres cosas:

- implementar un símbolo bien conocido, que es de lejos el caso más frecuente
- agregar metadatos a objetos que no son tuyos, sin riesgo de pisar nada
- guardar estado interno que no querés que aparezca en el JSON

Evitalos para otras tres:

- privacidad real, porque `Object.getOwnPropertySymbols` los expone y los campos `#` no
- valores que tengan que sobrevivir a `JSON.stringify` o a `structuredClone`, porque no sobreviven
- enumeraciones, donde un objeto congelado con `Object.freeze` o un tipo de TypeScript rinde más

La prueba rápida es preguntarte si el valor tiene que cruzar la frontera del proceso. Si va a viajar por la red o a guardarse en disco, un símbolo es la herramienta equivocada, porque su identidad vive solo en la memoria de este programa.

Ejercicios

1. Escribí un objeto `semana` que se pueda recorrer con `for...of` y devuelva los siete días. Resóvelo primero con un objeto iterador escrito a mano y después con un generador.

2. Explicá sin ejecutar qué imprime `console.log(Symbol("a") === Symbol("a"))` y qué imprime `console.log(Symbol.for("a") === Symbol.for("a"))`.
3. Agregá a un objeto una propiedad con clave de símbolo y comprobá con código que no aparece en `Object.keys`, que no aparece en `JSON.stringify` y que sí aparece en la copia hecha con propagación.
4. Escribí una clase `Temperatura` que devuelva los grados como número y una cadena con el símbolo de grado como texto, usando `Symbol.toPrimitive`. Probala con `+t` y con una interpolación.
5. Escribí una clase `Vocal` que haga que `"a" instanceof Vocal` devuelva `true` y `"b" instanceof Vocal` devuelva `false`.
6. Tomás encuentra este código y no entiende por qué falla: `const s = Symbol("x"); console.log("El símbolo es " + s);`. Explicale qué pasa, por qué el lenguaje lo decidió así y cómo se arregla.

Expresiones regulares en JavaScript

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte complementario de la unidad de JavaScript

Hasta ahora, cuando buscaste algo dentro de un texto, buscaste un contenido exacto.

```
texto.includes("hola")
```

 pregunta si esas cuatro letras están ahí.

Pero casi todo lo que necesitás en una aplicación web no tiene contenido exacto. Un email no es una cadena concreta: es cualquier cadena con arroba en el medio y un punto después. Un teléfono no es un número fijo: es cualquier secuencia de dígitos con espacios, guiones y paréntesis salpicados. Una fecha, un CUIT, un código postal, un nombre de archivo, todos son lo mismo: una forma, no un valor.

Las expresiones regulares son el lenguaje para describir formas. Este apunte arranca por qué son y de dónde salen, después cómo se escriben, y termina en recetas que vas a copiar y pegar en el proyecto de la materia.

Qué es una expresión regular

Una expresión regular describe un conjunto de cadenas.

Esa es toda la definición y conviene tomársela literal. La expresión `/ab/` no es una instrucción para buscar. Es el nombre de un conjunto que tiene un solo elemento, la cadena `"ab"`. La expresión `/ab*/` nombra un conjunto infinito: `"a"`, `"ab"`, `"abb"`, `"abbb"` y así para siempre.

Cuando escribís `patron.test(texto)` no le estás pidiendo al motor que busque. Le estás preguntando si el texto pertenece al conjunto que describe el patrón.

Ese cambio de perspectiva ayuda cuando una expresión no anda. La pregunta deja de ser "por qué no encuentra lo que quiero" y pasa a ser "qué conjunto describí realmente", que casi siempre resulta ser más grande de lo que pensabas.

Las tres operaciones que lo generan todo

Un conjunto de cadenas se construye con tres operaciones, y solo tres. Todo lo demás que vas a ver en este apunte es abreviatura.

La primera es la concatenación. Si ponés dos patrones uno al lado del otro, describís las cadenas que tienen el primero seguido del segundo:

```
/ab/.test("ab"); // true
```

La segunda es la alternancia. La barra vertical describe la unión de dos conjuntos, o sea las cadenas que están en uno o en el otro:

```
/gato|perro/.test("perro"); // true
```

La tercera es la repetición. El asterisco describe cero o más copias de lo que tiene a la izquierda:

```
/ab*/.test("a"); // true  
/ab*/.test("abbb"); // true
```

Con esas tres se puede describir cualquier conjunto regular. El resto de la sintaxis existe para no escribir tanto:

- `a+` es lo mismo que `aa*`
- `a?` es lo mismo que `(a|)`
- `[abc]` es lo mismo que `(a|b|c)`
- `a{2,3}` es lo mismo que `(aa|aaa)`
- `\d` es lo mismo que `[0-9]`, que a su vez es `(0|1|2|3|4|5|6|7|8|9)`

Nada de eso agrega poder. Agrega comodidad. Si alguna vez te perdés en un patrón, traducilo mentalmente a estas tres operaciones y vuelve a tener sentido.

La máquina que las ejecuta

Un motor de expresiones regulares es, en su forma pura, un autómata finito. Lee el texto de izquierda a derecha, un carácter por vez, y lo único que recuerda es en qué estado está. La cantidad de estados es fija y se decide cuando se compila el patrón.

De ahí sale la consecuencia más importante y la que más se ignora: una expresión regular no puede contar.

No tiene una memoria que crezca, así que no puede llevar la cuenta de cuántos paréntesis abrió para exigir que se cierren todos. No puede verificar que un HTML esté bien anidado. No puede validar una expresión aritmética con paréntesis adentro de paréntesis.

Cuando alguien te diga que las expresiones regulares no sirven para parsear HTML, esta es la razón técnica. No es una opinión de estilo: HTML anidado no es un lenguaje regular, así que ninguna expresión regular lo describe.

Ahora, el motor de JavaScript no es un autómata finito puro. Tiene funciones que se salen de la teoría, como las retrorreferencias y las miradas, y las implementa con retroceso: prueba un camino, y si falla, vuelve atrás y prueba otro. Guardate ese dato, porque al final del apunte explica por qué un patrón de tres líneas puede colgar un servidor.

Cómo se escribe una en JavaScript

Hay dos formas. La literal va entre barras y se compila cuando el motor lee el archivo:

```
const patron = /\d+/g;
```

La otra usa el constructor y recibe una cadena, así que se compila en tiempo de ejecución:

```
const patron = new RegExp("\\d+", "g");
```

Fijate la doble barra invertida. En la cadena, `"\\d"` produce los dos caracteres `\d`, que son los que el motor necesita. Es la fuente de errores más común con el constructor.

La regla para elegir es simple: si el patrón es fijo, usá la forma literal, que se lee mejor y se compila una sola vez. Usá el constructor solo cuando el patrón se arme en tiempo de ejecución, por ejemplo a partir de lo que el usuario escribió en un buscador.

Y si armás un patrón con texto del usuario, escapalo antes. Si no, un usuario que busca `precio (final)` te rompe la aplicación, porque los paréntesis significan otra cosa:

```
function escapar(texto) {  
  return texto.replace(/[*+?^${}()|[\]\\]/g, "\\$&");  
}  
  
const patron = new RegExp(escapar(loQueEscribio), "gi");
```

Las versiones nuevas del lenguaje traen `RegExp.escape`, que hace lo mismo. Mientras no puedas contar con ella en todos los navegadores que te interesan, dejá la función de arriba.

Los métodos que las usan

Antes de la sintaxis conviene saber quién las ejecuta, porque la mitad de los problemas vienen de elegir mal el método.

Llamada	Qué devuelve
<code>patron.test(texto)</code>	<code>true</code> o <code>false</code>
<code>texto.search(patron)</code>	la posición de la primera coincidencia, o <code>-1</code>
<code>texto.match(patron)</code>	sin <code>g</code> , la primera coincidencia con sus grupos; con <code>g</code> , un arreglo de textos sin grupos
<code>texto.matchAll(patron)</code>	un iterador con todas las coincidencias y sus grupos, y exige la bandera <code>g</code>
<code>texto.replace(patron, r)</code>	el texto con la primera coincidencia reemplazada, o todas si el patrón lleva <code>g</code>
<code>texto.replaceAll(patron, r)</code>	el texto con todas reemplazadas, y exige <code>g</code> si el patrón es una expresión regular
<code>texto.split(patron)</code>	un arreglo de partes
<code>patron.exec(texto)</code>	la próxima coincidencia, o <code>null</code> , avanzando entre llamadas si lleva <code>g</code>

Para preguntar si algo cumple una forma, usá `test`. Para sacar datos, usá `matchAll`. Ese par cubre casi todo:

```
const texto = "Pedidos: 12, 7 y 340 unidades";

/\d+/.test(texto); // true

[...texto.matchAll(/\d+/g)].map((c) => Number(c[0]));
// [12, 7, 340]
```

Cada coincidencia que devuelve `matchAll` o `exec` es un arreglo con extras. La posición `0` tiene el texto completo que coincidió, las siguientes tienen los grupos, y además trae `index` con la posición y `groups` con los grupos nombrados.

Literales y escapes

Casi todos los caracteres se representan a sí mismos. La letra `a` describe la letra `a`.

Los que no lo hacen son los metacaracteres, que tienen significado propio:

```
. * + ? ^ $ | \ ( ) [ ] { }
```

Para describir uno de esos caracteres tal cual, pónale una barra invertida adelante:

```
/3\\.14/.test("3.14"); // true
/3.14/.test("3x14"); // true, porque el punto significa "cualquier carácter"
```

El segundo caso es el error clásico. Un punto sin escapar acepta cualquier cosa, así que un patrón para validar precios acepta basura sin que te enteres.

Clases de caracteres

Los corchetes describen un carácter que puede ser cualquiera de los que están adentro:

```
/[aeiou]/.test("ritmo"); // false
/[gr[ia]s]/.test("gris"); // true
```

Adentro de los corchetes podés usar rangos con guion, y podés negar todo el conjunto poniendo

`^` al principio:

```
/[a-z]/; // una minúscula de la a a la z
/[0-9a-fA-F]/; // un dígito hexadecimal
/[^0-9]/; // cualquier cosa que no sea un dígito
```

Adentro de los corchetes casi ningún metacarácter conserva su significado. El punto es un punto y el asterisco es un asterisco. Solo hay que cuidar cuatro: el `]` que cierra, la barra invertida, el `^` cuando va primero y el guion cuando queda entre dos caracteres.

Los atajos y una advertencia sobre el español

Hay abreviaturas para las clases más usadas:

Atajo	Qué describe	Equivale a
<code>d</code>	un dígito	<code>[0-9]</code>
<code>D</code>	cualquier cosa menos un dígito	<code>[^0-9]</code>
<code>w</code>	un carácter de palabra	<code>[A-Za-z0-9_]</code>
<code>W</code>	lo contrario de <code>w</code>	<code>[^A-Za-z0-9_]</code>
<code>s</code>	un espacio en blanco	espacio, tabulación, salto de línea
<code>S</code>	lo contrario de <code>s</code>	
<code>.</code>	cualquier carácter menos el salto de línea	

Mirá con atención la definición de `\w`. Son las letras del inglés. No incluye la eñe ni las vocales acentuadas:

```
"Muñoz".match(/\w+/)[0]; // "Mu"
"José".match(/\w+/)[0]; // "Jos"
```

Esto es una fuente de bugs enorme en cualquier aplicación en castellano, y aparece cuando ya está en producción y un cliente se llama Peña. La solución es pedir letras de verdad con una propiedad Unicode, que necesita la bandera `u`:

```
"Muñoz".match(/\p{L}+/u)[0]; // "Muñoz"
"José".match(/\p{L}+/u)[0]; // "José"
```

Tomá `\p{L}` como el reemplazo por omisión de `\w` en todo lo que escribas para usuarios hispanohablantes.

Cuantificadores

Un cuantificador dice cuántas veces se repite lo que tiene a la izquierda:

Cuantificador	Cuántas veces
<code>*</code>	cero o más
<code>+</code>	una o más
<code>?</code>	cero o una
<code>{3}</code>	exactamente 3
<code>{2,}</code>	2 o más
<code>{2,4}</code>	entre 2 y 4

Se aplican al elemento inmediatamente anterior, que puede ser un carácter, una clase o un grupo entero:

```
/ho+la/.test("hooooola"); // true
/[0-9]{4}/.test("2026"); // true
/(ja)+/.test("jajaja"); // true
```

El error frecuente es olvidar el grupo. `/ja+/` describe una jota seguida de varias aes, no varias risas.

Codicia y pereza

Por omisión, un cuantificador es codicioso: se lleva todo lo que puede y recién después devuelve lo mínimo necesario para que el resto del patrón encaje.

```
const html = "<b>hola</b> <i>chau</i>";  
  
html.match(/<.+>/)[0]; // "<b>hola</b> <i>chau</i>"
```

Eso sorprende la primera vez. Pediste algo entre `<` y `>`, y te devolvió desde el primer `<` hasta el último `>`, porque el punto también acepta esos caracteres y el `+` agarró todo.

Agregando un signo de pregunta después del cuantificador lo volvéis perezoso: toma lo mínimo y crece solo si hace falta.

```
html.match(/<.+?>/)[0]; // "<b>"
```

La regla práctica es que cuando extraés algo delimitado, casi siempre querés la versión perezosa. La alternativa, muchas veces mejor, es prohibir el delimitador dentro de la parte que capturarás:

```
html.match(/<[>]+>/)[0]; // "<b>"
```

Esta última es más rápida que la perezosa, porque no necesita retroceder. Volvemos sobre eso al final.

Anclas y límites de palabra

Un patrón por omisión puede coincidir en cualquier parte del texto. Las anclas lo atan a una posición:

- `^` marca el principio del texto
- `$` marca el final del texto
- `\b` marca un límite de palabra
- `\B` marca la ausencia de un límite de palabra

La diferencia entre validar y buscar es exactamente esta:

```
/\d{4}/.test("abc 2026 xyz"); // true, encontró 4 dígitos en el medio  
/^d{4}$/.test("abc 2026 xyz"); // false  
/^d{4}$/.test("2026"); // true
```

Cuando validás un campo de formulario, anclá siempre con `^` y `$`. Sin anclas, `/\d{4}/` acepta cualquier basura que contenga cuatro dígitos en algún lado.

Los límites de palabra sirven para buscar palabras enteras:

```
"la casa y la casona".match(/\bcasa\b/g); // ["casa"]  
"la casa y la casona".match(/casa/g); // ["casa", "casa"]
```

Un `\b` no consume ningún carácter. Solo afirma que en esa posición cambia de un carácter de palabra a uno que no lo es. Y como se define en términos de `\w`, arrastra el mismo problema con la ñe.

Alternancia y grupos

La barra vertical separa alternativas, y tiene la precedencia más baja de toda la sintaxis. Eso confunde:

```
/^gato|perro$/; // "empieza con gato" o "termina con perro"  
/^(gato|perro)$/; // exactamente "gato" o exactamente "perro"
```

Casi siempre querés la segunda. Los paréntesis limitan el alcance de la alternancia, igual que en aritmética.

Los paréntesis hacen dos cosas a la vez: agrupan y capturan. Si solo necesitás agrupar, usá un grupo sin captura, que se escribe con `(?:)`:

```
/(?:ja)+/; // agrupa, no captura
```

No es solo cosmética. Con muchos grupos, los que no capturan hacen que el motor trabaje menos y que la numeración de los que sí capturan no se corra cada vez que agregás un paréntesis.

Grupos de captura y grupos nombrados

Un grupo que captura guarda la parte del texto que coincidió, para que la puedas leer después:

```
const patron = /(\d{2})\(/(\d{2})\(/(\d{4})/;
const c = "Vence el 15/08/2026".match(patron);

c[0]; // "15/08/2026"
c[1]; // "15"
c[2]; // "08"
c[3]; // "2026"
```

Depender de los números es frágil: agregás un paréntesis al principio y se corren todos. Poneles nombre:

```
const patron = /(?<dia>\d{2})\(/(?<mes>\d{2})\(/(?<anio>\d{4})/;
const { groups } = "Vence el 15/08/2026".match(patron);

groups.dia; // "15"
groups.anio; // "2026"
```

La sintaxis `(?<nombre>...)` es la misma de C#, así que esto ya lo vieron con `System.Text.RegularExpressions`.

Los grupos nombrados se combinan muy bien con la desestructuración que vimos en tipos compuestos:

```
const {
  groups: { dia, mes, anio },
} = "15/08/2026".match(patron);
```

Retrorreferencias

Una retrorreferencia exige que algo que ya apareció vuelva a aparecer igual. Se escribe `\1` para el primer grupo, o `\k<nombre>` si el grupo tiene nombre:

```
/(\w)\1/.test("casa"); // false
/(\w)\1/.test("carro"); // true, la doble erre
```

El uso práctico más común es detectar palabras repetidas al corregir un texto:

```
const patron = /\b(\p{L}+)\s+\1\b/giu;

"esto tiene tiene un error".match(patron); // ["tiene tiene"]
```

Acá se rompe la teoría, dicho sea de paso. Un autómata finito no puede recordar una palabra de largo arbitrario para compararla después. Las retrorreferencias son una de las funciones que sacan al motor del terreno regular y lo obligan a retroceder.

Miradas hacia adelante y hacia atrás

Una mirada verifica una condición sin consumir texto. Son cuatro:

Sintaxis	Qué exige
<code>(?=...)</code>	que lo que sigue coincida
<code>(?!...)</code>	que lo que sigue no coincida
<code>(?<=...)</code>	que lo anterior coincida
<code>(?<!=...)</code>	que lo anterior no coincida

La palabra clave es "sin consumir". Después de una mirada, el motor sigue parado en el mismo lugar donde estaba.

Eso permite exigir varias condiciones sobre el mismo texto a la vez, que es el caso de uso estrella:

```
const fuerte = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\w\s]).{10,}$;/;

fuerte.test("hola1234"); // false
fuerte.test("HolaMundo1!"); // true
```

Leelo de izquierda a derecha, parado en el principio del texto. Exige que en algún lugar más adelante haya una minúscula. Volvé al principio. Exige que haya una mayúscula. Volvé al principio. Y así con el dígito y con el símbolo. Recién al final consume el texto y le pide 10 caracteres o más.

Sin miradas habría que escribir todas las combinaciones de orden posibles, que son muchísimas.

Las miradas hacia atrás sirven para extraer sin arrastrar el contexto:

```
"El total es $1500".match(/(?<=\$)\d+/)[0]; // "1500"
```

El signo peso no queda en el resultado, porque nunca se consumió.

Las banderas

Las banderas van después de la barra final y cambian cómo se aplica el patrón:

Bandera	Qué hace
<code>g</code>	busca todas las coincidencias, no solo la primera
<code>i</code>	ignora mayúsculas y minúsculas
<code>m</code>	hace que <code>^</code> y <code>\$</code> valgan por línea y no por texto entero
<code>s</code>	hace que el punto también acepte el salto de línea
<code>u</code>	interpreta el patrón por puntos de código Unicode y habilita <code>p{...}</code>
<code>v</code>	como <code>u</code> , y además permite operaciones de conjuntos entre clases
<code>y</code>	obliga a que la coincidencia empiece justo en <code>lastIndex</code>
<code>d</code>	agrega las posiciones de inicio y fin de cada grupo

La bandera `m` es la que hace posible procesar un archivo de texto línea por línea con un solo patrón:

```
const log = `GET /api/contactos 200 12ms
POST /api/contactos 201 34ms
GET /api/contactos/7 404 3ms`;

const patron =
  /^(?<metodo>GET|POST|PUT|DELETE) (?<ruta>\S+) (?<estado>\d{3}) (?<ms>\d+)ms$/gm;

for (const { groups } of log.matchAll(patron)) {
  if (groups.estado.startsWith("4")) console.log(groups.ruta);
}
// /api/contactos/7
```

La trampa de la bandera g

Una expresión regular con `g` guarda estado. La propiedad `lastIndex` recuerda dónde terminó la última búsqueda, y la próxima llamada arranca desde ahí.

Con `matchAll` eso es invisible y funciona bien. Con `test` en un bucle produce uno de los bugs más desconcertantes del lenguaje:

```
const patron = /a/g;

patron.test("a"); // true
patron.test("a"); // false
patron.test("a"); // true
```

El mismo texto, el mismo patrón, y el resultado alterna. La primera llamada encuentra la `a` y deja `lastIndex` en 1. La segunda arranca en la posición 1, no encuentra nada, devuelve `false` y reinicia `lastIndex` en 0.

Hay tres formas de evitarlo:

- no le pongas `g` a un patrón que vas a usar con `test`, porque `test` no la necesita
- creá el patrón adentro de la función, así cada llamada arranca con uno nuevo
- si tenés que reusarlo, poné `patron.lastIndex = 0` antes de cada uso

La primera es la buena. Reservá `g` para `matchAll`, `replace` y `replaceAll`.

Reemplazar

El segundo argumento de `replace` puede ser un texto con marcadores especiales:

Marcador	Qué inserta
<code>@@CODE0@@amp;</code>	todo el texto que coincidió
<code>\$1</code> , <code>\$2</code>	el grupo número 1, 2
<code>@@CODE0@@lt;nombre></code>	el grupo nombrado
<code>` \$ ``</code>	lo que había antes de la coincidencia
<code> </code>	lo que había después
<code>\$</code>	un signo peso literal

Convertir una fecha de formato argentino a formato ISO se resuelve en una línea:

```
"15/08/2026".replace(
  /^(?<dia>\d{2})\/(?<mes>\d{2})\/(?<anio>\d{4})$/,
  "$<anio>-<mes>-<dia>",
);
// "2026-08-15"
```

Cuando el reemplazo necesita lógica, pasá una función. Recibe el texto completo, después cada grupo, después la posición, el texto original y, si hay grupos nombrados, un objeto con ellos:

```
const precios = "Camisa $1500, pantalón $2300";

precios.replace(/\$(\d+)/g, (todo, monto) => {
  const conIva = Math.round(Number(monto) * 1.21);
  return `${conIva}`;
});

// "Camisa $1815, pantalón $2783"
```

La función se llama una vez por coincidencia. Es la puerta para hacer cualquier cosa que el patrón solo no puede.

Unicode

Sin la bandera `u`, el motor trabaja de a unidades de 16 bits, y los caracteres que necesitan más de una se parten al medio:

```
"café ☹".match(/./gu).length; // 6
```

Con `u` habilitás además las propiedades Unicode, que son mucho más expresivas que los atajos clásicos:

- `\p{L}` cualquier letra
- `\p{Lu}` una letra mayúscula
- `\p{N}` cualquier dígito, incluidos los de otros alfabetos
- `\p{P}` un signo de puntuación
- `\p{Script=Latin}` una letra del alfabeto latino

Poné `u` por costumbre en todo patrón nuevo. Casi nunca molesta y evita sorpresas con nombres, emojis y texto pegado desde Word.

Recetas para el proyecto de la materia

Estas son las que vas a usar en la agenda de contactos.

Validar un email sin volverse loco. La expresión completa del estándar tiene miles de caracteres y no vale la pena. Esta alcanza, porque el que valida de verdad es el mail de confirmación que le mandás después:

```
const esEmail = (valor) => /^[^s@]+@[^s@]+\.[^s@]+$/i.test(valor.trim());
```

Dejar un teléfono en dígitos, sin importar cómo lo hayan escrito:

```
const soloDigitos = (tel) => tel.replace(/\D+/g, "");  
  
soloDigitos("+54 (381) 155-123456"); // "54381155123456"
```

Validar un CUIT por su forma, antes de verificar el dígito verificador:

```
const formaDeCuit = /^d{2}-d{8}-d$/;
```

Convertir un nombre en un identificador para la URL:

```
function aSlug(texto) {  
  return texto  
    .normalize("NFD")  
    .replace(/[\u0300-\u036f]/g, "")  
    .toLowerCase()  
    .replace(/^[a-z0-9]+/g, "-")  
    .replace(/^-+|-+$/g, "");  
}  
  
aSlug("Café Martínez – Sucursal Yerba Buena");  
// "cafe-martinez-sucursal-yerba-buena"
```

El truco de las dos primeras líneas merece explicación. `normalize("NFD")` separa cada letra acentuada en dos caracteres, la letra pelada y el acento. El rango `[\u0300-\u036f]` son justamente los acentos, así que el `replace` los borra y queda la letra sola.

Separar un contacto pegado desde un mail:

```
const patron = /^(?<nombre>[^<]+?)\s*<(?<email>[^>]+)>$/;  
  
const { groups } = "Ada Lovelace <ada@ejemplo.com>".match(patron);  
groups.nombre; // "Ada Lovelace"  
groups.email; // "ada@ejemplo.com"
```

Resaltar el término buscado en una lista de resultados:

```
function resaltar(texto, busqueda) {  
  const patron = new RegExp(escapar(busqueda), "gi");  
  return texto.replace(patron, "<mark>${}</mark>");  
}
```

Acordate de escapar también el HTML del texto antes de insertarlo en la página. Resaltar no te exime de eso, y lo vamos a ver en la unidad de seguridad.

El costo escondido: el retroceso catastrófico

Probá esto en la consola, pero guardá lo que estés haciendo:

```
const patron = /^(a+)+$/;

patron.test("aaaaaaaaaaaaaaaaaaaaaaaaaaaaa!");
```

El navegador se congela. No es un error del patrón: la respuesta correcta es `false`, y el motor la va a dar, pero después de probar todos los caminos posibles.

La causa es la ambigüedad. El texto tiene 29 aes, y el patrón pide grupos de una o más aes, repetidos una o más veces. Hay una cantidad exponencial de formas de repartir esas aes en grupos, y como el signo de exclamación final hace fallar todas, el motor las prueba una por una antes de rendirse.

Esto tiene nombre propio en seguridad: se llama ReDoS, y es un ataque de denegación de servicio. Si tu servidor valida con un patrón así lo que escribe un usuario, alcanza con un formulario bien elegido para clavar el proceso. Y como Node atiende todo en un solo hilo, se clava para todos los usuarios a la vez.

Tres reglas para no caerte:

- desconfiá de un cuantificador adentro de un grupo que también tiene cuantificador, como `(a+)+` o `(\d*)*`
- desconfiá de alternativas que aceptan lo mismo, como `(a|aa)+`, porque generan la misma ambigüedad
- preferí prohibir un carácter a permitir cualquiera, o sea `[^>]+` en vez de `.+?`

Acá hay una diferencia con C# que conviene marcar. .NET te da un tiempo máximo por búsqueda y un modo sin retroceso, así que podés protegerte desde afuera. JavaScript no tiene nada de eso: no hay tiempo límite, ni grupos atómicos, ni cuantificadores posesivos. La única defensa es el patrón que escribiste, y validar el largo de la entrada antes de aplicarlo.

Cuándo no usar una expresión regular

Son la herramienta equivocada más seguido de lo que parece.

No las uses para HTML ni XML. Ya vimos por qué: el anidamiento no es regular. En el navegador tenés `DOMParser`, y en el servidor hay librerías.

No las uses para CSV de verdad. Un CSV puede tener comas adentro de comillas, y comillas escapadas adentro de comillas. Cada vez que parcheés el patrón va a aparecer un caso nuevo.

No las uses cuando el lenguaje ya trae un analizador. Para una dirección web está la clase `URL`, que te da el dominio, la ruta y los parámetros ya separados y bien decodificados. Para fechas está `Date` y está `Intl`.

Y no las uses cuando una operación de cadenas alcanza. `texto.includes("hola")` se entiende de un vistazo y `/hola/.test(texto)` no agrega nada.

La regla es que una expresión regular gana cuando el patrón tiene variabilidad. Si estás buscando algo fijo, buscalo con los métodos de cadena.

Cómo depurarlas

Tres hábitos que ahorran horas.

Construirlas de a poco. Empezá con la parte más chica que puedas probar, verificá que ande, y recién ahí agregale la siguiente. Un patrón de 60 caracteres escrito de un tirón no hay forma de arreglarlo.

Usá una herramienta que las explique. En regex101.com pegás el patrón y el texto, y te muestra qué hizo cada parte y cuántos pasos le llevó. Ese contador de pasos es la mejor alarma temprana de un retroceso catastrófico.

Y escribí tests. Una expresión regular es exactamente la clase de código que parece andar hasta que llega un dato raro. Poné los casos que tienen que dar verdadero, los que tienen que dar falso, y sobre todo los del borde: el vacío, el que tiene acentos y el que es larguísimo.

JavaScript no tiene una bandera para escribir patrones en varias líneas con comentarios, como sí la tiene C#. Cuando uno se te vaya de largo, armalo por partes con nombres que se entiendan:

```
const dia = "(?<dia>\\d{2})";
const mes = "(?<mes>\\d{2})";
const anio = "(?<anio>\\d{4})";

const fecha = new RegExp(`^${dia}/${mes}/${anio}$`);
```

Chuleta

Símbolo	Qué significa
<code>.</code>	cualquier carácter menos el salto de línea
<code>d w s</code>	dígito, carácter de palabra, espacio
<code>D W S</code>	los tres anteriores negados
<code>p{L}</code>	una letra de cualquier idioma, con la bandera <code>u</code>
<code>[abc] [^abc]</code>	uno de esos, ninguno de esos
<code>[a-z]</code>	un rango
<code>*</code> <code>+</code> <code>?</code>	cero o más, una o más, cero o una
<code>{2}</code> <code>{2,}</code> <code>{2,4}</code>	exactamente, al menos, entre
<code>*?</code> <code>+?</code>	las versiones perezosas
<code>^</code> <code>\$</code>	principio y final
<code>b</code>	límite de palabra
<code> </code>	alternativa
<code>(...)</code>	grupo que captura
<code>(?:...)</code>	grupo que no captura
<code>(?<n>...)</code>	grupo nombrado
<code>1</code> <code>k<n></code>	retorreferencia
<code>(?=...)</code> <code>(?!...)</code>	mirada adelante, positiva y negativa
<code>(?<=...)</code> <code>(?<!=...)</code>	mirada atrás, positiva y negativa

Ejercicios

1. Escribí un patrón que valide un código postal argentino con el formato de una letra, cuatro dígitos y tres letras, como `T4000ABC`. Anclalo y probalo con un valor válido, uno con menos dígitos y uno que tenga el código correcto en el medio de otro texto.
2. Extraé todos los precios de este texto y devolvé la suma: `"Camisa $1500, pantalón $2300, medias $450"`.
3. Escribí una función `nombrePropio(texto)` que ponga en mayúscula la primera letra de cada palabra usando `replace` con una función. Tiene que funcionar con `"maría de los ángeles peña"`.

4. Explicá sin ejecutar por qué `/^\d{2}-\d{8}-\d$/`.test("20-12345678-9 y algo más") devuelve `false`, y qué devolvería el mismo patrón sin el `$`.
5. Dado un texto con líneas del formato `apellido, nombre: email`, escribí un patrón con grupos nombrados y armá con `matchAll` un arreglo de objetos con las tres propiedades.
6. Mirá este patrón para validar un identificador: `/^[a-z]+[0-9]*+$/`. Encontrá la ambigüedad, explicá por qué se cuelga con una entrada larga que termina en un guion, y reescribilo sin el problema.
7. Escribí dos patrones que hagan lo mismo, uno con cuantificador perezoso y otro con clase negada, para extraer el contenido entre comillas dobles. Probá cuál de los dos hace menos pasos en regex101.

Funciones en JavaScript

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte de la quinta clase de JavaScript

Hasta ahora escribimos instrucciones que se ejecutan de arriba abajo. Guardamos datos en variables, los recorrimos con bucles y decidimos con `if`. Con eso alcanza para resolver un problema, pero no para escribir un programa.

El problema aparece cuando el mismo cálculo se repite en tres lugares con pequeñas diferencias. Podés copiarlo tres veces, y entonces cada corrección hay que hacerla tres veces, y tarde o temprano una se olvida.

Una función resuelve eso de dos maneras a la vez. Le pone un nombre a un cálculo, así que podés invocarlo en vez de repetirlo. Y separa lo que es fijo de lo que varía, así que un solo cálculo sirve para muchos casos.

Este apunte arranca por ahí, pero no termina ahí. En JavaScript una función es además un valor como cualquier otro, y esa idea es la base de todo lo que viene después: las clausuras, las funciones de orden superior y la forma de trabajar con colecciones que vas a usar en el resto de la materia.

Del código repetido a la función

Mirá este código:

```
const precioCamisa = 1500;
const totalCamisa = precioCamisa + precioCamisa * 0.21;

const precioPantalon = 2300;
const totalPantalon = precioPantalon + precioPantalon * 0.21;
```

Hay una idea repetida: sumar el impuesto. Lo que cambia entre los dos casos es un solo dato.

Una función captura la idea y deja afuera el dato:

```
function conIva(precio) {  
  return precio + precio * 0.21;  
}  
  
conIva(1500); // 1815  
conIva(2300); // 2783
```

Eso que quedó afuera se llama parámetro. Es el agujero que dejaste en el cálculo para que quien lo use lo llene.

Fijate qué ganaste. Si mañana el impuesto cambia, lo cambiás en un lugar. Si el cálculo tiene un error, el error está en un lugar. Y el nombre `conIva` explica la intención, cosa que la fórmula suelta no hacía.

Una función es un valor

Acá viene el primer principio del que cuelga todo el resto.

En JavaScript, una función no es una construcción especial del lenguaje que solo se puede declarar y llamar. Es un valor, del mismo modo que `42` es un valor y `"hola"` es un valor.

Eso significa que podés hacer con una función todo lo que hacés con cualquier otro dato:

```
const doble = function (n) {  
  return n * 2;  
};  
  
typeof doble; // "function"  
  
const alias = doble; // guardarla en otra variable  
alias(21); // 42  
  
const operaciones = [doble, conIva]; // meterla en un arreglo  
operaciones[0](21); // 42  
  
const calculadora = { doble }; // meterla en un objeto  
calculadora.doble(21); // 42
```

Volvé al modelo mental de la clase de variables: la variable no es una caja, es una flecha que apunta a un valor. Con las funciones vale igual. `doble` es una flecha que apunta a una función, y `alias` es otra flecha que apunta a la misma función. No se copió nada.

Que una función sea un valor no es un detalle de escritura. Es lo que permite que una función reciba a otra como argumento, o que devuelva una función nueva. En muchos lenguajes eso hay que habilitarlo con alguna construcción especial. Acá no hace falta ninguna, porque la función ya

es un dato.

Guardá esta idea, porque las dos secciones más importantes del apunte, clausuras y orden superior, son consecuencias directas de ella.

Las tres formas de escribir una función

Hay tres formas, y la diferencia entre ellas no es de estilo.

La declaración es la clásica:

```
function sumar(a, b) {  
  return a + b;  
}
```

La expresión guarda una función anónima en una variable:

```
const sumar = function (a, b) {  
  return a + b;  
};
```

Y la expresión con nombre le pone un nombre a la función, además del nombre de la variable:

```
const sumar = function sumarDosNumeros(a, b) {  
  return a + b;  
};
```

Ese nombre interno sirve para dos cosas: aparece en los mensajes de error, que se vuelven mucho más legibles, y permite que la función se llame a sí misma aunque después cambies la variable.

La cuarta forma, la flecha, tiene su propia sección porque no es solo otra sintaxis.

Por qué una anda antes de estar escrita

Probá esto:

```
saludar("Ada"); // "Hola, Ada"  
  
function saludar(nombre) {  
  console.log(`Hola, ${nombre}`);  
}
```

Y después probá esto:

```
despedir("Ada"); // ReferenceError

const despedir = function (nombre) {
  console.log(`Chau, ${nombre}`);
};
```

La diferencia es el izado, que ya vimos con `var`, `let` y `const`. Antes de ejecutar un bloque, el motor lo recorre y registra lo que hay adentro.

Las declaraciones de función se registran completas, con su cuerpo incluido. Por eso podés llamarlas desde arriba.

Las expresiones no. Ahí lo que se registra es la variable, con las reglas de `const`: existe pero no se puede tocar hasta que la ejecución pasa por su línea.

En la práctica esto no cambia mucho si ordenás el código de forma razonable. Conviene conocerlo porque explica un error que de otro modo parece imposible.

La forma flecha

La flecha es una sintaxis más corta para escribir una función, y se convirtió en la forma habitual de pasar una función como argumento.

Empecemos por la versión completa, que es igual a una expresión de función con otra escritura:

```
const sumar = (a, b) => {
  return a + b;
};
```

A partir de ahí vienen las abreviaturas, y conviene verlas una por una porque se combinan.

Si el cuerpo es una sola expresión, sacás las llaves y el `return`. El valor de la expresión se devuelve solo:

```
const sumar = (a, b) => a + b;
```

Si hay un solo parámetro, podés sacar los paréntesis:

```
const doble = (n) => n * 2;
const doble = n => n * 2; // las dos son válidas
```

Muchos equipos escriben siempre los paréntesis por consistencia, y porque TypeScript los va a necesitar cuando le pongas el tipo al parámetro. Nosotros vamos a usarlos siempre.

Si no hay parámetros, los paréntesis vacíos son obligatorios:

```
const ahora = () => new Date();
```

Y acá está la trampa que a todos les pasa una vez. Si querés devolver un objeto literal con la forma corta, tenés que envolverlo en paréntesis:

```
const crear = (nombre) => { nombre: nombre }; // devuelve undefined  
const crear = (nombre) => ({ nombre: nombre }); // correcto
```

La razón es que las llaves son ambiguas. En el primer caso el motor las lee como el cuerpo de la función, no como un objeto. Los paréntesis le sacan la duda.

Las flechas se pueden encadenar, lo que hace que una función devuelva otra con muy poca ceremonia:

```
const multiplicar = (a) => (b) => a * b;
```

Eso se lee "dame un `a` y te devuelvo una función que espera un `b`". Volvemos sobre ese patrón al final.

Lo que la flecha no tiene

Una flecha no es solo azúcar sintáctica. Le faltan cosas a propósito.

No tiene `this` propio. Usa el del lugar donde fue escrita.

No tiene `arguments`.

No se puede usar con `new`, así que no sirve como constructor.

De esas tres, la primera es la que importa, y es la razón por la que las flechas se inventaron. Vamos a verla en detalle en la clase de objetos, pero la situación es esta:

```
const contador = {
  cuenta: 0,

  arrancarMal() {
    setInterval(function () {
      this.cuenta++; // this no es el contador
    }, 1000);
  },

  arrancarBien() {
    setInterval(() => {
      this.cuenta++; // this sigue siendo el contador
    }, 1000);
  },
};
```

Una función clásica recibe su `this` de cómo la llaman, y a la función que le pasás a `setInterval` la llama el temporizador. Una flecha no tiene `this` propio, así que usa el de afuera, que es el que vos esperabas.

La regla práctica, mientras tanto: usá flechas para funciones que pasás como argumento, y funciones clásicas o métodos para las que son parte de un objeto.

El alcance de las variables

El alcance de una variable es la parte del código donde ese nombre significa algo.

JavaScript usa alcance léxico, que también se llama estático. Quiere decir que el alcance se decide por dónde está escrito el código, y no por quién llama a quién en tiempo de ejecución. Podés determinar el alcance de cada variable mirando el archivo, sin ejecutarlo.

Hay tres niveles.

El alcance global es el de las variables declaradas fuera de todo. Existen en todo el programa.

El alcance de función es el de las variables declaradas con `var`. Viven en la función entera, sin importar en qué bloque las declaraste.

El alcance de bloque es el de `let` y `const`. Viven entre las llaves donde fueron declaradas.

```
function ejemplo() {
  if (true) {
    var conVar = "vivo en toda la función";
    let conLet = "vivo solo en este bloque";
  }

  console.log(conVar); // funciona
  console.log(conLet); // ReferenceError
}
```

Por eso usamos `const` por omisión y `let` cuando hace falta reasignar. `var` quedó para leer código viejo.

La cadena de alcances

Cuando el motor encuentra un nombre, lo busca primero en el bloque actual. Si no está, sale al bloque de afuera. Si tampoco está, sigue saliendo hasta llegar al alcance global. Si ahí tampoco está, tira `ReferenceError`.

```
const iva = 0.21; // global

function facturar(precio) {
  const descuento = 0.1; // alcance de la función

  if (precio > 1000) {
    const extra = 0.05; // alcance del bloque
    return precio * (1 + iva) * (1 - descuento - extra);
  }

  return precio * (1 + iva) * (1 - descuento);
}
```

Adentro del `if` se ven las tres. Afuera del `if` ya no se ve `extra`. Afuera de la función solo se ve `iva`.

La búsqueda va siempre hacia afuera, nunca hacia adentro. Una función no puede ver las variables locales de otra función, aunque la llame.

Si un nombre se repite en dos niveles, gana el más cercano y el de afuera queda tapado:

```
const nombre = "global";

function saludar() {
  const nombre = "local";
  console.log(nombre); // "local"
}
```

Eso se llama sombreado. No es un error, pero conviene evitarlo cuando confunde.

La clausura

Ahora juntá dos cosas que ya sabemos. Una función es un valor, así que puede sobrevivir al lugar donde nació. Y el alcance es léxico, así que la función sabe qué variables la rodeaban cuando se escribió.

La pregunta es qué pasa cuando la función sale de ahí y esas variables, en teoría, ya deberían haber desaparecido.

```
function contador() {
  let cuenta = 0;

  return () => ++cuenta;
}

const siguiente = contador();

siguiente(); // 1
siguiente(); // 2
siguiente(); // 3
```

La función `contador` terminó de ejecutarse en la primera línea de uso. Su variable `cuenta` debería haberse ido con ella. Sin embargo sigue viva, y encima recuerda su valor entre llamadas.

Eso es una clausura: una función junto con el entorno donde fue creada.

El mecanismo es más simple de lo que parece. Mientras exista una referencia a la función interna, el entorno que esa función necesita no se puede liberar. No es magia ni una excepción a las reglas: es la consecuencia de que el alcance sea léxico y de que las funciones sean valores.

Y cada llamada crea un entorno nuevo:


```
const a = contador();
const b = contador();

a(); // 1
a(); // 2
b(); // 1
```

`a` y `b` no comparten nada. Son dos clausuras distintas sobre dos variables `cuenta` distintas.

Para qué sirve una clausura

Tres usos que vas a escribir en la materia.

El primero es guardar estado privado. Las variables de la clausura no se pueden tocar desde afuera, porque no hay ningún nombre que las alcance:

```
function crearAgenda() {
  const contactos = [];

  return {
    agregar(contacto) {
      contactos.push(contacto);
      return contactos.length;
    },

    listar() {
      return [...contactos];
    },
  };
}

const agenda = crearAgenda();
agenda.agregar({ nombre: "Ada" });
agenda.listar(); // [{ nombre: "Ada" }]
agenda.contactos; // undefined
```

Esto es encapsulamiento, y lo conseguiste sin clases ni ninguna sintaxis especial. La variable es inalcanzable porque no existe, fuera de la función, ningún nombre que llegue hasta ella. Fijate el detalle de `listar`, que devuelve una copia: si devolviera el arreglo original, cualquiera podría modificarlo desde afuera y la privacidad sería falsa.

El segundo es fabricar funciones configuradas:

```
function conImpuesto(tasa) {
  return (precio) => precio * (1 + tasa);
}

const conIva = conImpuesto(0.21);
const conIngresosBrutos = conImpuesto(0.035);

conIva(1000); // 1210
```

La tasa queda fija adentro de la función devuelta. No hay que pasarla en cada llamada.

El tercero es recordar resultados para no recalcularlos:

```
function memorizar(fn) {
  const cache = new Map();

  return (arg) => {
    if (!cache.has(arg)) cache.set(arg, fn(arg));
    return cache.get(arg);
  };
}
```

El `Map` que vimos en tipos compuestos vive en la clausura. Cada función memorizada tiene el suyo.

El bucle que devuelve tres veces lo mismo

Este es el bug de clausuras más famoso, y sirve para entender el mecanismo por el lado del error:

```
const acciones = [];

for (var i = 0; i < 3; i++) {
  acciones.push(() => console.log(i));
}

acciones[0](); // 3
acciones[1](); // 3
acciones[2](); // 3
```

Uno espera 0, 1 y 2. Salen tres treses.

La causa es el alcance de `var`. Hay una sola variable `i` para toda la función, así que las tres flechas capturaron la misma variable, no tres copias de su valor. Cuando se ejecutan, el bucle ya terminó y esa única `i` vale 3.

Cambia `var` por `let` y anda:

```
for (let i = 0; i < 3; i++) {  
  acciones.push(() => console.log(i));  
}  
// 0, 1, 2
```

`let` crea una variable nueva en cada vuelta del bucle, así que cada flecha captura la suya. Esta es la razón concreta por la que dejamos `var` atrás, y muestra que una clausura captura variables, no valores.

Cómo recibe los parámetros

JavaScript no controla la cantidad de argumentos. Si faltan, los parámetros que sobran valen `undefined`. Si sobran, se ignoran:

```
function saludar(nombre, saludo) {  
  return `${saludo}, ${nombre}`;  
}  
  
saludar("Ada"); // "undefined, Ada"  
saludar("Ada", "Hola", "de más"); // "Hola, Ada"
```

Nada te avisa. No hay ningún control que compare la llamada con la declaración, ni cuando escribís el código ni cuando se ejecuta. Contemplar los argumentos que faltan es responsabilidad tuya.

Para eso están los valores por omisión:

```
function saludar(nombre, saludo = "Hola") {  
  return `${saludo}, ${nombre}`;  
}  
  
saludar("Ada"); // "Hola, Ada"
```

El valor por omisión se evalúa en cada llamada, no una sola vez al definir la función. Por eso este código es seguro y devuelve una lista nueva siempre:

```
function agregar(item, lista = []) {  
  lista.push(item);  
  return lista;  
}
```

Y como se evalúa en el momento, un parámetro puede usar a los anteriores:

```
function rango(desde, hasta = desde + 10) {  
  return [desde, hasta];  
}  
  
rango(5); // [5, 15]
```

Cuando la cantidad de argumentos es variable, juntalos con el resto:

```
function sumar(...numeros) {  
  return numeros.reduce((total, n) => total + n, 0);  
}  
  
sumar(1, 2, 3, 4); // 10
```

`...numeros` es un arreglo de verdad, con todos sus métodos. Va siempre último y solo puede haber uno por función.

Vas a encontrar `arguments` en código viejo, que es un objeto parecido a un arreglo con todos los argumentos. No lo uses: no es un arreglo, y las flechas no lo tienen.

Parámetros con nombre

Una función con cinco parámetros es una función que nadie puede llamar sin mirar la firma:

```
crearContacto("Ada", "ada@ejemplo.com", true, false, "clientes");
```

Nadie sabe qué significan ese `true` y ese `false`, y para averiguarlo hay que ir a leer la declaración. JavaScript no permite nombrar los argumentos en la llamada, pero combinando la desestructuración con los valores por omisión se consigue el mismo efecto:

```
function crearContacto({
  nombre,
  email,
  activo = true,
  favorito = false,
  etiqueta = "general",
} = {}) {
  return { nombre, email, activo, favorito, etiqueta };
}

crearContacto({
  nombre: "Ada",
  email: "ada@ejemplo.com",
  favorito: true,
});
```

Ahora la llamada se lee sola, el orden dejó de importar y agregar una opción nueva no rompe el código existente.

El `= {}` del final es importante: permite llamar a la función sin argumentos. Sin él, `crearContacto()` intentaría desestructurar `undefined` y tiraría un error.

Este patrón es el que vas a ver en casi todas las librerías modernas, y conviene adoptarlo apenas una función pasa de 3 parámetros. Vale la pena mirarlo despacio, porque es exactamente la sintaxis con la que vas a escribir componentes cuando lleguemos a React.

Cómo funciona la desestructuración en un parámetro

Estas dos funciones hacen lo mismo:

```
function mostrar(contacto) {
  const nombre = contacto.nombre;
  const email = contacto.email;
  console.log(`${nombre}: ${email}`);
}

function mostrar({ nombre, email }) {
  console.log(`${nombre}: ${email}`);
}
```

La segunda ahorra dos líneas, pero lo importante es entender qué son esas llaves, porque confunden la primera vez.

Las llaves en la lista de parámetros no crean un objeto. Son un patrón que describe la forma del objeto que va a llegar, y le dice al motor qué propiedades sacar y con qué nombre guardarlas. Los nombres de adentro tienen que coincidir con los de las propiedades.

La función sigue recibiendo un solo argumento. La desestructuración no convierte un objeto en varios parámetros: solo desarma el objeto apenas entra.

Renombrar, valores por omisión y anidamiento

Podés guardar una propiedad con otro nombre. A la izquierda va la propiedad que llega, a la derecha el nombre local que querés usar:

```
function mostrar({ nombre: nombreCompleto, email: correo }) {  
  console.log(nombreCompleto, correo);  
}
```

Se lee al revés de lo que uno espera, y es la parte que más cuesta. Sirve cuando el nombre que llega es feo, o cuando choca con una variable que ya tenés.

Cada propiedad puede tener su valor por omisión:

```
function mostrar({ nombre, activo = true, etiqueta = "general" }) {}
```

Ese valor entra en juego solo si la propiedad es `undefined`. Con `null`, con `0` o con una cadena vacía no se aplica, porque esos son valores presentes:

```
mostrar({ nombre: "Ada", activo: null }); // activo queda en null
```

Si el objeto tiene objetos adentro, el patrón se anida igual:

```
function enviar({ asunto, destinatario: { nombre, email } }) {  
  console.log(`${asunto} para ${nombre} <${email}>`);  
}
```

Acá hay dos trampas. La primera es que eso no crea ninguna variable `destinatario`: al ponerle dos puntos, `destinatario` pasa a ser el camino, no el destino. Si necesitás las dos cosas, nombralo dos veces:

```
function enviar({ destinatario, destinatario: { email } }) {}
```

La segunda es que si `destinatario` no viene, el patrón intenta desarmar `undefined` y la función explota. Se cubre con un valor por omisión en el nivel de adentro:

```
function enviar({ destinatario: { email } = {} }) {}
```

Esa es la misma idea del `= {}` que pusimos al final del patrón completo, aplicada un nivel más abajo.

Juntar el resto de las propiedades

Dentro de un patrón de objeto, los tres puntos juntan todo lo que no nombraste en un objeto nuevo:

```
function crearBoton({ texto, alHacerClic, ...resto }) {  
  console.log(texto); // "Guardar"  
  console.log(resto); // { tipo: "submit", ancho: "100%" }  
}  
  
crearBoton({  
  texto: "Guardar",  
  alHacerClic: guardar,  
  tipo: "submit",  
  ancho: "100%",  
});
```

Es la misma sintaxis de los parámetros rest, pero adentro del patrón. Sirve para tomar las propiedades que te interesan y pasar el resto a otro lado sin enumerarlas.

Desestructurar arreglos en los parámetros

Los arreglos se desarman igual, pero con corchetes y por posición en vez de por nombre:

```
function distancia([x1, y1], [x2, y2]) {  
  return Math.hypot(x2 - x1, y2 - y1);  
}  
  
distancia([0, 0], [3, 4]); // 5
```

También admite valores por omisión, saltar posiciones con una coma de más y juntar el resto:

```
function podio([oro, , bronce = "vacante", ...resto]) {}
```

Se usa menos que la de objetos, con una excepción enorme que vemos ahora mismo.

La sintaxis que vas a usar en React

Un componente de React es una función. Recibe un solo objeto con todo lo que le pasaron desde afuera, y a ese objeto se lo llama props por convención.

Sin desestructurar, se ve así:

```
function Boton(props) {  
  return <button onClick={props.alHacerClic}>{props.texto}</button>;  
}
```

Y desestructurando el parámetro, así:

```
function Boton({ texto, alHacerClic }) {  
  return <button onClick={alHacerClic}>{texto}</button>;  
}
```

No mires todavía la sintaxis de las etiquetas, que la vemos entera en la unidad de React. Mirá solo la lista de parámetros, que es lo que ya sabés leer.

Las dos versiones funcionan, y la segunda es la que vas a encontrar en todos lados. La razón no es que ahorre teclas: es que la lista de parámetros pasa a ser la documentación del componente. Leés la primera línea y sabés qué acepta.

Con valores por omisión, esa primera línea dice todavía más:

```
function Boton({  
  texto,  
  alHacerClic,  
  variante = "primario",  
  deshabilitado = false,  
}) {}
```

Ahí queda escrito, en un solo lugar, qué es obligatorio y qué es opcional con su valor por omisión.

React usa además una propiedad especial llamada `children`, que contiene lo que haya entre la etiqueta de apertura y la de cierre:


```
function Tarjeta({ titulo, children }) {
  return (
    <section>
      <h2>{titulo}</h2>
      {children}
    </section>
  );
}
```

Y el resto de las propiedades es el mecanismo con el que un componente propio reenvía a la etiqueta de adentro todo lo que no le interesa a él:

```
function Entrada({ etiqueta, ...resto }) {
  return (
    <label>
      {etiqueta}
      <input {...resto} />
    </label>
  );
}

<Entrada etiqueta="Email" type="email" required placeholder="tu@correo.com" />;
```

Mirá las dos apariciones de los tres puntos, porque son las dos caras de la misma idea. En el parámetro juntan lo que no nombraste. En la etiqueta lo reparten de nuevo, propiedad por propiedad, sobre el `input`. El componente se ocupa de la etiqueta visible y deja pasar `type`, `required` y `placeholder` sin saber que existían.

Falta una pieza, y es la desestructuración de arreglos:

```
const [contactos, setContactos] = useState([]);
```

`useState` devuelve un arreglo de dos elementos: el valor actual y la función que lo cambia. La desestructuración les pone nombre a los dos en una línea.

Y acá se ve por qué devuelve un arreglo y no un objeto. Como la desestructuración de arreglos es por posición, vos elegís los nombres. En un mismo componente podés escribir `useState` cinco veces y llamar a cada par como corresponda. Con un objeto habría que renombrar las mismas dos propiedades en cada llamada.

Cómo se pasan los argumentos

JavaScript pasa siempre una copia del valor. Lo que confunde es que en los objetos el valor es la flecha, no el objeto.

Con primitivos, la función trabaja sobre una copia y afuera no cambia nada:

```
function incrementar(n) {  
  n++;  
  return n;  
}  
  
let cuenta = 5;  
incrementar(cuenta); // 6  
cuenta; // 5
```

Con objetos, la función recibe una flecha nueva que apunta al mismo objeto. Si modificás el objeto, el cambio se ve afuera:

```
function activar(contacto) {  
  contacto.activo = true;  
}  
  
const ada = { nombre: "Ada", activo: false };  
activar(ada);  
ada.activo; // true
```

Pero si reasignás el parámetro, solo estás moviendo tu propia flecha, y afuera no pasa nada:

```
function reemplazar(contacto) {  
  contacto = { nombre: "Otro" };  
}  
  
reemplazar(ada);  
ada.nombre; // "Ada"
```

Es la misma distinción entre mutar y reasignar que vimos con `const` y con la copia de arreglos. Vale igual acá.

La consecuencia práctica: una función que modifica los objetos que recibe es más difícil de razonar. Cuando puedas, devolvé un objeto nuevo en vez de tocar el que te dieron.

Cómo devuelve valores

Una función devuelve un valor con `return`. Si no hay `return`, o si hay un `return` pelado, devuelve `undefined`:

```
function sinNada() {}  
sinNada(); // undefined
```

`return` corta la ejecución en el acto. Eso habilita el estilo de guardas que vimos en estructuras de control:

```
function descuento(cliente) {  
  if (!cliente) return 0;  
  if (!cliente.activo) return 0;  
  if (cliente.compras < 3) return 0;  
  
  return 0.15;  
}
```

Cada caso raro se resuelve y se sale. Lo que queda al final es el caso normal, sin anidamiento.

Hay una trampa con el punto y coma automático. Esto devuelve `undefined`:

```
function crear() {  
  return  
  {  
    ok: true;  
  }  
}
```

El motor inserta un punto y coma después del `return`, porque `return` no puede tener un salto de línea antes de su valor. La llave se queda sola y nunca se ejecuta. Poné siempre el valor en la misma línea que el `return`.

Para devolver varios valores hay dos caminos. Con un objeto, si los datos tienen nombre propio:

```
function analizar(texto) {  
  return {  
    palabras: texto.split(/\s+/).length,  
    caracteres: texto.length,  
  };  
}  
  
const { palabras, caracteres } = analizar("hola mundo");
```

Con un arreglo, si el orden es lo natural y quien lo recibe va a querer renombrarlos:

```
function dividir(a, b) {  
  return [Math.floor(a / b), a % b];  
}  
  
const [cociente, resto] = dividir(17, 5);
```

En los dos casos, la desestructuración del otro lado es lo que hace que se lea bien.

Las funciones de orden superior

Una función de orden superior es una función que recibe funciones como argumento, devuelve una función, o las dos cosas.

No es una categoría especial del lenguaje. Es lo que pasa naturalmente cuando las funciones son valores: si podés pasar un número, podés pasar una función.

El caso de recibirlas es el más común. Lo venís usando sin darle nombre:

```
[3, 1, 2].sort((a, b) => a - b);  
boton.addEventListener("click", () => console.log("clic"));  
setTimeout(() => console.log("después"), 1000);
```

En los tres casos, la función que pasás describe qué hacer, y la función de orden superior decide cuándo y cuántas veces hacerlo. Esa división de trabajo es la idea central.

El caso de devolverlas ya lo vimos con las clausuras. Sirve para configurar una función antes de usarla:

```
const porCampo = (campo) => (a, b) => a[campo].localeCompare(b[campo]);  
  
contactos.sort(porCampo("nombre"));  
contactos.sort(porCampo("ciudad"));
```

Y las dos cosas juntas dan los envoltorios, que agregan comportamiento a una función sin tocarla:

```
function conTiempo(fn) {
  return (...args) => {
    const inicio = performance.now();
    const resultado = fn(...args);
    console.log(`${fn.name} tardó ${(performance.now() - inicio).toFixed(1)}ms`);
    return resultado;
  };
}

const buscarConTiempo = conTiempo(buscarContactos);
```

La función original no se enteró de nada. Este patrón es el que usan por dentro los middleware de Express, que vamos a ver más adelante.

Las colecciones como una cadena de pasos

Los arreglos traen una familia de métodos que reciben una función. Todos siguen la misma idea: vos escribís qué hacer con un elemento, y el método se encarga de recorrer.

Tres cubren casi todo, y el resto son variantes.

filter selecciona. Recibe una función que responde sí o no, y devuelve un arreglo con los elementos que dijeron que sí.

map transforma. Recibe una función que convierte un elemento en otra cosa, y devuelve un arreglo del mismo largo con los resultados.

reduce resume. Recibe una función que combina lo acumulado con el elemento siguiente, y devuelve un solo valor.

Tomemos estos datos:

```
const contactos = [
  { nombre: "Ada Lovelace", ciudad: "Tucumán", compras: 5, activo: true },
  { nombre: "Grace Hopper", ciudad: "Salta", compras: 2, activo: true },
  { nombre: "Alan Turing", ciudad: "Tucumán", compras: 8, activo: false },
  { nombre: "Edsger Dijkstra", ciudad: "Salta", compras: 1, activo: true },
];
```

Como cada método devuelve un arreglo, se pueden encadenar. Cada eslabón recibe lo que devolvió el anterior:

```
const resultado = contactos
  .filter((c) => c.activo)
  .toSorted((a, b) => a.nombre.localeCompare(b.nombre))
  .map((c) => c.nombre);

// ["Ada Lovelace", "Edsger Dijkstra", "Grace Hopper"]
```

Eso se lee como una receta de arriba abajo: quedate con los activos, ordenalos por nombre, quedate solo con el nombre. Escribí la misma consulta con un bucle, un `if` adentro y un arreglo auxiliar, y vas a ver por qué esta forma se impuso.

Dos detalles del ejemplo. `toSorted` devuelve un arreglo nuevo, mientras que `sort` ordena el original en el lugar. Y `localeCompare` es lo que ordena bien los acentos y la ñe, cosa que comparar con `<` no hace.

La familia completa

Método	Qué hace	Qué devuelve
<code>filter</code>	se queda con los que cumplen una condición	un arreglo
<code>map</code>	transforma cada elemento	un arreglo del mismo largo
<code>flatMap</code>	transforma cada elemento y aplana un nivel	un arreglo
<code>reduce</code>	combina todos los elementos en un solo valor	lo que devuelvas
<code>toSorted</code>	ordena según un comparador	un arreglo nuevo
<code>toReversed</code>	invierte el orden	un arreglo nuevo
<code>find</code>	busca el primero que cumple	el elemento, o <code>undefined</code>
<code>findIndex</code>	busca la posición del primero que cumple	un número, o <code>-1</code>
<code>some</code>	pregunta si al menos uno cumple	<code>true</code> o <code>false</code>
<code>every</code>	pregunta si todos cumplen	<code>true</code> o <code>false</code>
<code>includes</code>	pregunta si un valor está	<code>true</code> o <code>false</code>
<code>slice</code>	corta un tramo	un arreglo
<code>at</code>	devuelve el elemento de una posición, y acepta negativos	el elemento
<code>Object.groupBy</code>	agrupa según una clave calculada	un objeto de arreglos

Algunos en acción:

```
// sumar una propiedad de todos
contactos.reduce((total, c) => total + c.compras, 0); // 16

// agrupar por ciudad
Object.groupBy(contactos, (c) => c.ciudad);
// { Tucumán: [...], Salta: [...] }

// quedarse con las ciudades sin repetir
[...new Set(contactos.map((c) => c.ciudad))]; // ["Tucumán", "Salta"]

// preguntar por el conjunto entero
contactos.some((c) => c.compras > 5); // true
contactos.every((c) => c.activo); // false
```

`reduce` es el que más cuesta al principio, así que vale detenerse. Su función recibe dos cosas: lo acumulado hasta ahora y el elemento actual. Lo que esa función devuelve pasa a ser lo acumulado de la vuelta siguiente. El segundo argumento de `reduce`, ese `0` del ejemplo, es el valor inicial del acumulador. No lo omitas nunca: sin él, un arreglo vacío tira error.

Cada paso arma un arreglo nuevo

Estos métodos hacen su trabajo en el momento. `filter` no anota la condición para aplicarla más tarde: recorre el arreglo entero, arma uno nuevo y lo devuelve.

En una cadena, eso significa un recorrido completo por eslabón:

```
contactos
  .filter((c) => c.activo)
  .map((c) => c.nombre)
  .filter((n) => n.startsWith("A"));
```

Ese código recorre tres veces y crea dos arreglos intermedios que se usan una vez y se tiran.

Con cuatro contactos no importa. Con cuatro millones sí, y ahí hay dos salidas. Una es hacer todo el trabajo en un solo `reduce`. La otra es usar un generador, que produce un valor por vez, recién cuando se lo piden:

```
function* soloActivos(items) {
  for (const item of items) {
    if (item.activo) yield item;
  }
}

for (const contacto of soloActivos(contactos)) {
  console.log(contacto.nombre);
}
```

Ahí no se arma ningún arreglo intermedio. Cada contacto se produce y se consume antes de que se calcule el siguiente. El generador funciona en el `for...of` gracias al `Symbol.iterator` que vimos en el apunte de símbolos.

Una aclaración más, que va a importar cuando lleguemos a bases de datos: todos estos métodos trabajan sobre datos que ya están en memoria. Filtrar del lado del servidor va a ser una decisión que escribas en la consulta. `filter` no puede hacerlo por vos, porque para filtrar necesita que los datos hayan viajado enteros hasta acá.

Cuándo encadenar y cuándo no

El encadenamiento se lee muy bien mientras cada paso tenga un propósito claro. Tres o cuatro eslabones está bien. Diez, no.

Cuando una cadena se vuelve difícil de seguir, cortala en variables con nombre:

```
const activos = contactos.filter((c) => c.activo);
const deTucuman = activos.filter((c) => c.ciudad === "Tucumán");
const nombres = deTucuman.map((c) => c.nombre);
```

Los nombres intermedios documentan lo que pasa mejor que cualquier comentario.

Y una recomendación que vale para todas estas funciones: la función que le pasás no debería modificar nada de afuera. `map` existe para transformar, no para provocar efectos. Si adentro de un `map` estás escribiendo en otro arreglo o tocando el DOM, lo que necesitás es un `for...of`.

Ejercicios

1. Escribí `promedio(...numeros)` con parámetros rest, que devuelva 0 si no recibe ninguno. Resolvela con `reduce`.

2. Convertí estas tres funciones a la forma flecha con la abreviatura más corta posible que siga siendo legible: una que suma dos números, una que devuelve un objeto con el nombre recibido, y una sin parámetros que devuelve la fecha de hoy.
3. Escribí `crearRegistro()` que devuelva un objeto con los métodos `anotar(mensaje)` y `verTodo()`, donde el arreglo de mensajes sea inaccesible desde afuera. Demostrá con código que no se puede tocar.
4. Explicá sin ejecutar qué imprime este código y por qué: `const fns = []; for (var i = 0; i < 3; i++) fns.push(() => i); console.log(fns.map((f) => f()));`. Después arreglalo de dos maneras distintas.
5. Escribí `configurarSaludo({ saludo = "Hola", signo = "!" } = {})` que devuelva una función que saluda a un nombre. Usala para crear un saludo formal y uno informal.
6. Con el arreglo de contactos del apunte, escribí en una sola cadena la consulta que devuelve, para cada ciudad, la suma de compras de sus contactos activos.
7. Escribí `unaSolaVez(fn)`, una función de orden superior que devuelva una función que ejecuta a `fn` la primera vez y en las siguientes devuelve el resultado guardado, sin volver a ejecutarla.

Gestión de archivos en Node.js

Tutorial desde primeros principios: qué es un archivo, cómo lo lee y escribe un programa, cómo se codifica el texto y cómo buscar, copiar, renombrar, listar y extraer información.

El mismo código corre en Windows, macOS y Linux. Node esconde casi todas las diferencias, pero no todas. Cuando queda una diferencia real, aparece marcada como nota entre sistemas.

Los ejemplos necesitan Node 22 o superior y módulos ES: archivos `.mjs`, o `"type": "module"` en el `package.json`.

Parte 1: qué es un archivo

Un archivo es una secuencia de bytes

Un archivo es eso y nada más: una tira de bytes numerados desde 0. El sistema operativo no sabe si son un poema, una foto o un programa. Solo sabe cuántos bytes hay y dónde están.

Esto vale igual en los tres sistemas. Todo lo demás que asociamos con un archivo vive afuera del archivo:

- el nombre vive en la carpeta que lo contiene
- el tamaño, las fechas y los permisos viven en un registro de metadatos aparte
- el tipo no existe para el sistema: la extensión es una convención

Cada sistema le pone otro nombre a ese registro de metadatos. Linux y macOS lo llaman inodo. NTFS, el sistema de archivos de Windows, lo llama entrada de la tabla maestra de archivos. La idea es la misma en los tres: contenido por un lado, ficha de datos por otro.

Una carpeta es una tabla de nombres

Una carpeta también es un archivo, pero con contenido reglamentado: una tabla que asocia nombres con registros de metadatos.

```
carpeta
  "apunte.txt" → registro 4021
  "foto.png"   → registro 4022
  "sub"        → registro 4023 (que es otra carpeta)
```

De acá salen tres conclusiones que después explican el comportamiento de las funciones de Node:

- renombrar es barato: cambiás una fila de la tabla, no movés bytes
- un mismo contenido puede tener varios nombres, llamados enlaces duros, que existen en los tres sistemas
- borrar un nombre no borra los bytes mientras quede otro nombre apuntando al mismo registro

Primera diferencia entre sistemas: archivos abiertos

Qué pasa si borrás un archivo que otro programa tiene abierto:

- en Linux y macOS el nombre desaparece de la tabla, pero los bytes siguen vivos hasta que el último programa cierre el archivo
- en Windows la operación falla, con el código `EBUSY` o `EPERM`, porque el sistema bloquea el archivo mientras está abierto

Esto también afecta a renombrar y a mover. Es la causa número uno de scripts que andan en la notebook del docente con macOS y fallan en la máquina del alumno con Windows. La regla práctica: cerrá siempre lo que abrís, con `finally`.

La extensión es una convención, casi

Para leer y escribir, la extensión no significa nada: podés guardar una imagen en `notas.txt` y Node la escribe sin quejarse.

Nota entre sistemas: lo que hace ejecutable a un archivo cambia. En Linux y macOS es un permiso del registro de metadatos. En Windows lo decide la extensión: `.exe`, `.bat`, `.ps1`. Por eso en Windows la extensión pesa más que en el resto.

No existe el archivo de texto

Un archivo de texto es un archivo binario cuyos bytes decidimos interpretar como caracteres, con una tabla de conversión que llamamos codificación. Si te equivocás de tabla, leés otra cosa. Lo vemos en la parte 4.

Parte 2: nombres y rutas

Acá nace la mayoría de los errores que aparecen solo en un sistema. Conviene verlo antes de escribir código.

Mayúsculas y minúsculas

- Linux distingue: `Datos.txt` y `datos.txt` son dos archivos distintos
- Windows no distingue, pero conserva lo que escribiste
- macOS, con el formato APFS de fábrica, tampoco distingue, aunque se puede formatear un disco para que sí

El síntoma típico: un proyecto anda en tu máquina y falla al subirlo a un servidor Linux, porque el código pide `Config.json` y el archivo se llama `config.json`.

La regla: escribí el nombre siempre igual y nunca uses las mayúsculas para diferenciar dos archivos.

Caracteres que no podés usar

- Windows prohíbe `< > : " / \ | ? *` y los caracteres de control, no admite nombres terminados en punto o en espacio, y reserva palabras completas como `CON`, `PRN`, `AUX`, `NUL`, `COM1` a `COM9` y `LPT1` a `LPT9`
- Linux y macOS solo prohíben la barra `/` y el byte nulo

Si generás nombres a partir de datos, y muchas veces vas a hacerlo, límpialos con el criterio más estricto de los tres:

```
const RESERVADOS = /^(con|prn|aux|nul|com[1-9]|lpt[1-9])(\..*)?$/i;

export function nombreSeguro(nombre) {
  let limpio = nombre
    .replace(/[<>:"/\|?*\\u0000-\\u001f]/g, '_') // prohibidos en Windows
    .replace(/[. ]+$/, ''); // punto o espacio final
  if (RESERVADOS.test(limpio)) limpio = `_${limpio}`;
  return limpio.slice(0, 120) || '_';
}

nombreSeguro('reporte 12/03/2026: ventas'); // 'reporte 12_03_2026_ ventas'
```

Un ejemplo típico: una fecha con barras en el nombre del archivo crea carpetas sin querer en Linux y falla en Windows.

Largo máximo

- Windows corta en 260 caracteres la ruta completa, salvo que el sistema tenga activadas las rutas largas
- Linux permite 255 bytes por nombre y unos 4096 por ruta
- macOS permite 255 caracteres por nombre

La regla: no anides carpetas muy profundas ni armes nombres larguísimos. Un límite propio de 120 caracteres por nombre te deja tranquilo en los tres.

Acentos y normalización

Unicode permite escribir `é` de dos formas: como un carácter único, llamado forma NFC, o como una `e` seguida de una tilde combinante, llamada forma NFD.

```
const nfc = 'José'.normalize('NFC');
const nfd = 'José'.normalize('NFD');
console.log(nfc === nfd);           // false
console.log(nfc.length, nfd.length); // 4 5
```

Se ven idénticos en pantalla y son strings distintos. macOS históricamente guarda los nombres en NFD, así que un nombre leído con `readdir` puede no coincidir con el mismo nombre escrito a mano en tu código.

La regla: normalizá los dos lados antes de comparar.

```
const igual = (a, b) => a.normalize('NFC') === b.normalize('NFC');
```

Archivos ocultos

En Linux y macOS un archivo está oculto si su nombre empieza con punto. En Windows es un atributo del registro de metadatos, que Node no expone. Si filtrás ocultos, filtrá por el punto inicial y asumí que en Windows el filtro es aproximado.

Parte 3: rutas con el módulo path

Las rutas son texto, y ahí siguen los errores. Usá siempre el módulo `path` en lugar de concatenar con `+`.

```
import path from 'node:path';

path.join('datos', 'sub', 'b.txt'); // 'datos/sub/b.txt' o 'datos\sub\b.txt'
path.resolve('datos', 'a.txt');     // ruta absoluta desde el directorio actual
path.basename('/tmp/a.txt');         // 'a.txt'
path.basename('/tmp/a.txt', '.txt'); // 'a'
path.extname('a.txt');               // '.txt'
path.dirname('datos/a.txt');         // 'datos'
path.parse('C:\\Users\\ale\\a.txt'); // { root, dir, base, ext, name }
```

`path` decide solo el separador correcto según dónde corra el programa. Esa es toda la portabilidad que necesitás en el 90% de los casos.

Qué significa absoluta en cada sistema

- en Linux y macOS una ruta absoluta empieza con `/`, y hay una sola raíz
- en Windows empieza con una letra de unidad, `C:\`, y cada unidad tiene su raíz
- Windows también admite rutas de red, llamadas UNC, con la forma

```
\\servidor\recurso\archivo.txt
```

Por eso en Windows existen rutas raras que en el resto no: `C:archivo.txt` es relativa al directorio actual de la unidad C.

Probar rutas de otro sistema

`path` trae dos versiones completas, y podés usarlas desde cualquier máquina. Sirven para las pruebas y para mostrar en clase cómo se ve la otra plataforma:

```
import path from 'node:path';

path.win32.join('datos', 'sub', 'b.txt'); // 'datos\sub\b.txt'
path.posix.join('datos', 'sub', 'b.txt'); // 'datos/sub/b.txt'
path.win32.sep;                          // '\'
path.posix.sep;                          // '/'
```

Nunca uses `path.posix` para tocar el disco. Es para manipular texto de rutas que sabés que son de estilo Unix, como las de una URL o las de un archivo zip.

Nunca escribas rutas fijas

Cambialas por funciones que devuelven lo correcto en cada sistema:

En vez de	Usá
<code>'/tmp'</code>	<code>os.tmpdir()</code>
<code>'/home/ale'</code> o <code>'C:\Users\ale'</code>	<code>os.homedir()</code>
<code>'carpeta/sub'</code>	<code>path.join('carpeta', 'sub')</code>
<code>ruta.split('/')</code>	<code>ruta.split(path.sep)</code>
<code>__dirname</code> en módulos ES	<code>import.meta.dirname</code>

```
import os from 'node:os';
import path from 'node:path';

const temporal = path.join(os.tmpdir(), 'mi-app', 'cache.json');
const plantilla = path.join(import.meta.dirname, 'plantillas', 'factura.html');
```

Una ruta relativa se resuelve contra la carpeta desde donde ejecutaste el programa, no contra el archivo donde está el código. `import.meta.dirname` te da la carpeta del módulo actual y viaja bien entre sistemas.

Rutas y URLs

Una URL de tipo `file:` siempre usa barras hacia adelante, aun en Windows. Convertí en lugar de recortar texto:

```
import { fileURLToPath, pathToFileURL } from 'node:url';

const ruta = fileURLToPath(import.meta.url);           // ruta del sistema
const url = pathToFileURL('C:\\datos\\a.txt');         // file:///C:/datos/a.txt
```

Rutas seguras

Si la ruta viene del usuario, `..` te saca de la carpeta permitida. Validá el resultado, no la entrada:

```
import path from 'node:path';

export function rutaSegura(base, entradaDelUsuario) {
  const raiz = path.resolve(base);
  const destino = path.resolve(raiz, entradaDelUsuario);
  if (destino !== raiz && !destino.startsWith(raiz + path.sep)) {
    throw new Error('Ruta fuera de la carpeta permitida');
  }
  return destino;
}
```

Comparar con `path.sep` en vez de con `/` es lo que hace que la validación también sirva en Windows.

Parte 4: cómo habla un programa con el disco

Las cuatro operaciones básicas

Por debajo de cualquier lenguaje hay cuatro pedidos al sistema operativo: abrir, leer, escribir y cerrar. Cambian los nombres internos, no la idea. Linux y macOS los llaman `open`, `read`, `write` y `close`. Windows los llama `CreateFile`, `ReadFile`, `WriteFile` y `CloseHandle`.

Al abrir, el sistema te devuelve una referencia: un número entero en Linux y macOS, un identificador en Windows. Node envuelve las dos cosas en un mismo objeto y las llama descriptor.

El descriptor tiene un cursor de posición. Cada lectura o escritura lo adelanta. Podés moverlo si querés leer el medio de un archivo sin leer el principio.

Este ejemplo hace exactamente eso y no usa ningún atajo:

```
import { open } from 'node:fs/promises';

const archivo = await open('datos/a.txt', 'r'); // abrir
try {
  const buffer = Buffer.alloc(4); // caja de 4 bytes vacía
  const { bytesRead } = await archivo.read(buffer, 0, 4, 5);
  //                               ↑   ↑   ↑   ↑
  //                               dónde guardar | | posición en el archivo
  //                               desde qué offset |
  //                               cuántos bytes leer
  console.log(bytesRead, buffer.toString('utf8'));
} finally {
  await archivo.close(); // cerrar siempre
}
```

Si el archivo contiene `hola\nmundo\n`, la salida es `4 mund`. Empezó en el byte 5 y trajo 4 bytes.

Ese `finally` no es adorno. En Windows, mientras el descriptor esté abierto, nadie puede borrar ni renombrar el archivo.

Todo lo que sigue en este tutorial es azúcar sobre estas cuatro operaciones.

Las tres APIs de fs

Node ofrece el mismo conjunto de funciones en tres presentaciones:

Forma	Cómo se importa	Cuándo usarla
Promesas	<code>import { readFile } from 'node:fs/promises'</code>	siempre que puedas
Callbacks	<code>import { readFile } from 'node:fs'</code>	código viejo o APIs de streams
Sincrónica	<code>import { readFileSync } from 'node:fs'</code>	scripts cortos y arranque de un programa

La versión sincrónica frena el único hilo de JavaScript hasta que el disco responde. En un script de línea de comandos no importa. En un servidor, una lectura sincrónica de 200 milisegundos deja a todos los clientes esperando.

Usá siempre el prefijo `node:` al importar. Deja claro que es un módulo del runtime y no un paquete de npm.

Parte 5: bytes y texto

Buffer, el arreglo de bytes de Node

`Buffer` es la representación de bytes crudos en Node. Es un arreglo de enteros de 0 a 255, y se comporta igual en los tres sistemas.

```
const b = Buffer.from('ñandú', 'utf8');
console.log(b);           // <Buffer c3 b1 61 6e 64 c3 ba>
console.log(b.length);    // 7 bytes
console.log('ñandú'.length); // 5 caracteres
```

Si lees un archivo sin indicar codificación, Node te devuelve un `Buffer`. Si indicás una, te devuelve un string ya decodificado.

```
import { readFile } from 'node:fs/promises';

const crudo = await readFile('datos/a.txt'); // Buffer
const texto = await readFile('datos/a.txt', 'utf8'); // string
```

Las codificaciones que vas a encontrar

Una codificación es una tabla que asigna números a caracteres, más una regla para escribir esos números como bytes.

Codificación	Bytes por carácter	Alcance
ASCII	1	128 caracteres, sin acentos ni ñe
Latin-1 (ISO-8859-1, <code>latin1</code>)	1	256 caracteres, alcanza para el español
Windows-1252	1	Latin-1 más comillas y guiones tipográficos
UTF-8	1 a 4	todo Unicode, compatible con ASCII
UTF-16	2 o 4	todo Unicode, es el formato interno de los strings de JavaScript

UTF-8 ganó porque los primeros 128 caracteres ocupan un byte y coinciden con ASCII. Un archivo en inglés pesa lo mismo que antes, y aun así podés escribir o 😊.

Nota entre sistemas: macOS y Linux escriben UTF-8 de fábrica. Windows todavía arrastra Windows-1252 en varias herramientas, entre ellas Excel al exportar CSV. Por eso los archivos con `Ã±` casi siempre vienen de una máquina con Windows.

Qué pasa si te equivocás de tabla

```
const bytes = Buffer.from('ñ', 'utf8'); // <Buffer c3 b1>
console.log(bytes.toString('latin1')); // Ã±
```

Eso es el mojibake: el texto ilegible que sale de leer bytes con la tabla equivocada. La palabra viene del japonés `𐐇` y significa algo parecido a caracteres transformados.

Se reconoce por el patrón. Casi siempre aparece una `Ã` de más:

- `Ã±` donde iba `ñ`
- `Ã©` donde iba `é`
- `â€œ` donde iba una comilla tipográfica

El motivo es directo. En UTF-8 las letras acentuadas del español empiezan con el byte `C3`, y ese byte, leído como Latin-1, es la letra `Ã`. El segundo byte aporta el símbolo raro que sigue.

Los bytes están bien; la tabla de lectura está mal. Nunca lo arregles reemplazando texto: volvé a leer con la codificación correcta.

Node decodifica de forma nativa `utf8`, `utf16le`, `latin1`, `ascii`, `base64` y `hex`. Para el resto usá `TextDecoder`, que acepta todas las codificaciones estándar:

```
import { readFile } from 'node:fs/promises';

const bytes = await readFile('datos/exportado.csv');
const texto = new TextDecoder('windows-1252').decode(bytes);
```

Guardá siempre en UTF-8. Es la única decisión que evita este problema para siempre.

Longitud: bytes, unidades de código y caracteres

Tres números distintos para la misma cadena:

```
const cara = '😄';
console.log(cara.length);           // 2  unidades UTF-16
console.log([...cara].length);      // 1  carácter real
console.log(Buffer.byteLength(cara)); // 4  bytes en UTF-8
```

Si vas a cortar texto, recorrela con el spread o con `Intl.Segmenter`. `slice` sobre índices puede partir un carácter al medio.

El BOM

Algunas herramientas de Windows escriben tres bytes al principio del archivo, `EF BB BF`, para anunciar que viene UTF-8. Node no los saca solo, así que aparecen como un carácter invisible al principio del string y rompen el primer encabezado de tu CSV.

```
function sinBOM(texto) {
  return texto.charCodeAt(0) === 0xfeff ? texto.slice(1) : texto;
}
```

Al revés, Excel en Windows suele necesitar ese BOM para abrir un CSV en UTF-8 sin romper los acentos. Si generás archivos para Excel, escribilo a propósito.

Fin de línea

Es la diferencia entre sistemas más vieja y más molesta:

- Linux y macOS terminan la línea con un byte, `\n`
- Windows usa dos, `\r\n`

Si partís un texto con `split('\n')`, en Windows te queda un `\r` colgado al final de cada línea. La comparación `línea === 'ok'` falla y no se ve por qué.

```
const lineas = texto.split(/\r?\n/); // sirve en los tres sistemas
```

Para escribir tenés dos criterios válidos. Si el archivo lo lee una persona en su máquina, usá `os.EOL`, que devuelve el separador local. Si el archivo lo lee otro programa, usá siempre `\n`: los tres sistemas lo entienden.

Nota entre sistemas: Git también participa. Con `core.autocrlf` activado convierte los finales de línea al guardar y al traer. Si ves un archivo entero marcado como modificado sin haberlo tocado, es eso.

Decodificar de a pedazos

Si leés un archivo grande por partes, un carácter UTF-8 de varios bytes puede quedar partido entre dos pedazos. La solución es un decodificador con estado:

```
const bytes = Buffer.from('ñandú');

// mal: cada pedazo se decodifica por su cuenta
const d1 = new TextDecoder();
console.log(d1.decode(bytes.subarray(0, 1))); // carácter roto

// bien: el decodificador recuerda el byte incompleto
const d2 = new TextDecoder();
let salida = d2.decode(bytes.subarray(0, 1), { stream: true });
salida += d2.decode(bytes.subarray(1), { stream: true });
console.log(salida); // ñandú
```

Cuando usás `createReadStream(ruta, 'utf8')` o `readline`, Node ya hace esto por vos.

La consola

Si el texto se ve bien en el archivo y mal en la terminal, el problema es la terminal, no tu código. En la consola vieja de Windows revisá la página de códigos con `chcp`; `chcp 65001` la pone en UTF-8. Windows Terminal y PowerShell 7 ya vienen bien de fábrica.

Parte 6: crear y escribir

Escribir un archivo entero

```
import { writeFile, appendFile, mkdir } from 'node:fs/promises';
import path from 'node:path';

await mkdir('salida', { recursive: true }); // crea la carpeta y sus padres
await writeFile(path.join('salida', 'notas.txt'), 'primera línea\n', 'utf8');
await appendFile(path.join('salida', 'notas.txt'), 'segunda línea\n', 'utf8');
```

`writeFile` crea el archivo si no existe y lo vacía si existe. `recursive: true` en `mkdir` no falla si la carpeta ya está.

Los flags de apertura

El flag decide qué pasa con lo que ya hay. Vale para `open`, `writeFile` y `createWriteStream`, y se comporta igual en los tres sistemas:

Flag	Qué hace
<code>r</code>	solo lectura, falla si no existe
<code>w</code>	escritura, crea o vacía
<code>wx</code>	escritura, falla si ya existe
<code>a</code>	agrega al final, crea si no existe
<code>ax</code>	agrega, falla si ya existe
<code>r+</code>	lectura y escritura, falla si no existe
<code>w+</code>	lectura y escritura, crea o vacía

```
// crear un archivo solo si no existe, sin pisar nada
try {
  await writeFile('salida/config.json', '{}', { flag: 'wx' });
} catch (err) {
  if (err.code !== 'EEXIST') throw err;
}
```

Muchas escrituras seguidas

Abrí una vez y escribí muchas veces. Cada llamada a `appendFile` abre y cierra el archivo, así que dentro de un bucle es costosa.

```
import { open } from 'node:fs/promises';

const archivo = await open('salida/log.txt', 'a');
try {
  for (let i = 0; i < 1000; i++) {
    await archivo.write(`línea ${i}\n`);
  }
} finally {
  await archivo.close();
}
```

Para volúmenes grandes conviene un stream, que además maneja la contrapresión cuando el disco no da abasto:

```
import { createWriteStream } from 'node:fs';
import { Readable } from 'node:stream';
import { pipeline } from 'node:stream/promises';

function* generar() {
  for (let i = 0; i < 1_000_000; i++) yield `línea ${i}\n`;
}

await pipeline(Readable.from(generar()), createWriteStream('salida/grande.txt'));
```

Escritura atómica

Si el programa se corta a la mitad de un `writeFile`, el archivo queda truncado. Para que nunca pase, escribís en un temporal y renombrás. El nombre dentro del mismo disco reemplaza el destino de una sola vez: quien lea, lee la versión vieja o la nueva, nunca una mezcla.

```
import { open, rename } from 'node:fs/promises';

export async function escribirAtomico(ruta, contenido) {
  const temporal = `${ruta}.${process.pid}.tmp`;
  const archivo = await open(temporal, 'wx');
  try {
    await archivo.writeFile(contenido, 'utf8');
    await archivo.sync(); // fuerza el volcado del cache a disco
  } finally {
    await archivo.close(); // en Windows, sin esto el rename falla
  }
  await rename(temporal, ruta);
}
```

Nota entre sistemas: en Windows el renombrar falla si el destino está abierto por otro programa, incluido el antivirus, que abre los archivos nuevos para revisarlos. Es un error transitorio, así que se resuelve reintentando:

```
async function renombrarConReintento(origen, destino, intentos = 5) {
  for (let i = 0; i < intentos; i++) {
    try {
      return await rename(origen, destino);
    } catch (err) {
      if (!['EBUSY', 'EPERM', 'EACCES'].includes(err.code) || i === intentos - 1) throw
err;
      await new Promise(r => setTimeout(r, 50 * 2 ** i));
    }
  }
}
```

Escribí siempre el temporal en la misma carpeta que el destino. Si lo ponés en `os.tmpdir()`, es probable que quede en otro disco y el renombrar falle con `EXDEV`.

Parte 7: leer

Leer todo de una vez

```
import { readFile } from 'node:fs/promises';

const texto = await readFile('datos/a.txt', 'utf8');
```

Es la forma más simple y sirve para archivos chicos. Tiene dos límites duros, iguales en los tres sistemas:

- el archivo entero entra en memoria, así que 2 GB de archivo son 2 GB de RAM
- un string de JavaScript no puede pasar de 512 MiB, y arriba de eso `readFile` con codificación tira un error

Leer línea por línea

Es la forma correcta para logs, CSV y cualquier archivo que no sepas cuánto pesa. La memoria usada es la de una línea.

```
import { createReadStream } from 'node:fs';
import { createInterface } from 'node:readline';

const rl = createInterface({
  input: createReadStream('datos/log.txt', 'utf8'),
  crlfDelay: Infinity // trata \r\n como un solo corte
});

let numero = 0;
for await (const linea of rl) {
  numero++;
  if (linea.includes('error')) console.log(`${numero}: ${linea}`);
}
```

`crlfDelay: Infinity` es lo que hace que este código lea igual un archivo escrito en Windows y uno escrito en Linux. Ponelo siempre.

Leer de a bloques

Cuando el archivo no es texto, o cuando querés controlar el tamaño del bloque:

```
import { createReadStream } from 'node:fs';

let bytes = 0;
for await (const bloque of createReadStream('datos/foto.png', { highWaterMark: 64 * 1024
})) {
  bytes += bloque.length; // bloque es un Buffer
}
console.log(bytes);
```

Leer un pedazo puntual

Ya lo viste en la parte 4: `archivo.read(buffer, offset, largo, posicion)`. Sirve para leer solo la cabecera de un archivo sin abrirlo entero, como hacemos en la parte 11 con el PNG.

Parte 8: listar carpetas

Listado simple

```
import { readdir } from 'node:fs/promises';

console.log(await readdir('datos'));           // ['a.txt', 'sub']
console.log(await readdir('datos', { recursive: true })); // incluye 'sub/b.txt'
```

Nota entre sistemas: el orden de `readdir` no está garantizado y cambia entre sistemas de archivos. Windows suele devolver alfabético, Linux devuelve el orden interno de la tabla. Si el orden importa, ordenalo vos.

```
const nombres = (await readdir('datos')).sort((a, b) => a.localeCompare(b, 'es'));
```

Listado con tipo

`withFileTypes` devuelve objetos `Dirent` en lugar de nombres. Evita un `stat` por entrada, que es la parte cara.

```
import { readdir } from 'node:fs/promises';

for (const entrada of await readdir('datos', { withFileTypes: true })) {
  console.log(entrada.name, entrada.isDirectory() ? 'carpeta' : 'archivo');
}
```

Recorrido propio

Un generador asíncronico te da control total, te deja podar ramas y no acumula todo el árbol en memoria:

```
import { readdir } from 'node:fs/promises';
import path from 'node:path';

export async function* recorrer(dir, omitir = new Set(['node_modules', '.git'])) {
  for (const entrada of await readdir(dir, { withFileTypes: true })) {
    const ruta = path.join(dir, entrada.name);
    if (entrada.isDirectory()) {
      if (omitir.has(entrada.name)) continue;
      yield* recorrer(ruta, omitir);
    } else if (entrada.isFile()) {
      yield ruta;
    }
  }
}

for await (const ruta of recorrer('.')) console.log(ruta);
```

Preguntá `isFile()` en vez de suponer. Una carpeta puede contener enlaces simbólicos, sockets y dispositivos, y esas entradas rompen cualquier lectura ingenua.

Nota entre sistemas: los tres tienen enlaces simbólicos, pero en Windows crearlos requiere permisos especiales o el modo de desarrollador. Además, los accesos directos `.lnk` de Windows no son enlaces para el sistema de archivos: son archivos comunes que solo el Explorador interpreta. Node los ve como lo que son, archivos de datos.

Parte 9: buscar

Hay tres formas de buscar, según por qué preguntás.

Buscar por nombre

Node trae `glob` desde la versión 22. Devuelve un iterador asíncronico, así que empieza a producir resultados antes de terminar el recorrido.

```
import { glob } from 'node:fs/promises';

for await (const ruta of glob('**/*.txt', { cwd: 'datos', exclude: ['**/*.tmp'] })) {
  console.log(ruta);
}
```

Los comodines son los del shell:

- `*` cualquier cosa menos el separador de carpetas

- `**` cualquier cosa, incluidas carpetas
- `?` un carácter
- `{a,b}` una alternativa

Los patrones se escriben siempre con la barra `/`, también en Windows. Es una convención de los globs, no una ruta del sistema. Las rutas que devuelve, en cambio, sí son del sistema: armalas y compará con `path`, nunca con texto.

Buscar por contenido

Un grep casero, línea por línea, que funciona con archivos de cualquier tamaño:

```
import { createReadStream } from 'node:fs';
import { createInterface } from 'node:readline';

export async function buscarEnArchivo(ruta, patron) {
  const rl = createInterface({
    input: createReadStream(ruta, 'utf8'),
    crlfDelay: Infinity
  });
  const hallazgos = [];
  let numero = 0;
  for await (const linea of rl) {
    numero++;
    if (patron.test ? patron.test(linea) : linea.includes(patron)) {
      hallazgos.push({ ruta, numero, linea });
    }
  }
  return hallazgos;
}
```

Combinalo con el generador `recorrer` y tenés un buscador de proyecto completo en 20 líneas.

Buscar por metadatos

```
import { stat } from 'node:fs/promises';

const haceUnaSemana = Date.now() - 7 * 24 * 60 * 60 * 1000;

for await (const ruta of recorrer('.')) {
  const info = await stat(ruta);
  if (info.size > 1_000_000 && info.mtimeMs > haceUnaSemana) {
    console.log(ruta, info.size);
  }
}
```

Buscar duplicados

Dos archivos con el mismo hash tienen, a todo fin práctico, el mismo contenido. El hash se calcula por stream, así que no importa el tamaño:

```
import { createReadStream } from 'node:fs';
import { createHash } from 'node:crypto';
import { pipeline } from 'node:stream/promises';

async function hashDe(ruta) {
  const hash = createHash('sha256');
  await pipeline(createReadStream(ruta), hash);
  return hash.digest('hex');
}

const porHash = new Map();
for await (const ruta of recorrer('.')) {
  const h = await hashDe(ruta);
  porHash.set(h, [...(porHash.get(h) ?? []), ruta]);
}
for (const [h, rutas] of porHash) {
  if (rutas.length > 1) console.log('duplicados:', rutas);
}
```

Un truco para acelerarlo: comparar primero el tamaño con `stat`, que es casi gratis, y calcular el hash solo para los que coinciden.

Parte 10: copiar, mover, renombrar y borrar

Copiar

```
import { copyFile, cp } from 'node:fs/promises';
import { constants } from 'node:fs';

await copyFile('datos/a.txt', 'salida/a.txt');
await copyFile('datos/a.txt', 'salida/a.txt', constants.COPYFILE_EXCL); // falla si
existe
await cp('datos', 'respaldo', { recursive: true }); // carpeta
entera
```

Al copiar se copian los bytes y las fechas, no siempre los permisos. En Windows los permisos son listas de control de acceso que no tienen equivalente en el modelo de Unix, así que el destino hereda los de su carpeta.

Renombrar y mover

Son la misma operación: cambiar la entrada de nombre en la tabla de la carpeta. Por eso mover dentro del mismo disco es instantáneo aunque el archivo pese 4 GB.

```
import { rename, copyFile, unlink } from 'node:fs/promises';

await rename('salida/viejo.txt', 'salida/nuevo.txt');
```

Cruzar de disco es otra cosa: ahí no hay una tabla común y `rename` falla con `EXDEV`. Pasa al mover de `C:` a `D:` en Windows, a un disco externo en macOS, o entre particiones en Linux. Y también al usar Docker, donde cada volumen es otro sistema de archivos.

```
export async function mover(origen, destino) {
  try {
    await rename(origen, destino);
  } catch (err) {
    if (err.code !== 'EXDEV') throw err;
    await copyFile(origen, destino);
    await unlink(origen);
  }
}
```

Dos advertencias más. `rename` pisa el destino sin avisar si ya existe. Y en Windows falla si el origen o el destino están abiertos, con `EBUSY` o `EPERM`: usá el reintento de la parte 6.

Borrar

```
import { rm, unlink } from 'node:fs/promises';

await unlink('salida/a.txt'); // un archivo
await rm('salida/temp', { recursive: true, force: true }); // carpeta, sin fallar si no está
```

`force: true` significa "no me avises si no existe". Es cómodo y peligroso: se come el `ENOENT` que te habría avisado de un error de tipeo en la ruta.

Nota entre sistemas: borrar no manda a la papelera en ninguno de los tres. La papelera la implementa el explorador de archivos, no el sistema. Si querés ese comportamiento, mové a una carpeta propia o usá una librería como `trash`.

No preguntes si existe

Esto parece razonable y está mal:

```
if (existsSync(ruta)) {
  const texto = await readFile(ruta, 'utf8'); // puede fallar igual
}
```

Entre la pregunta y la respuesta, otro proceso pudo borrar el archivo. Es una condición de carrera clásica, llamada TOCTOU. La forma correcta es intentar la operación y manejar el error:

```
try {
  const texto = await readFile(ruta, 'utf8');
} catch (err) {
  if (err.code !== 'ENOENT') throw err;
  // el archivo no está: usá un valor por defecto
}
```

Parte 11: extraer información

Metadatos con stat

```
import { stat } from 'node:fs/promises';

const info = await stat('datos/a.txt');
console.log(info.size);           // bytes, confiable en los tres sistemas
console.log(info.mtime);         // última modificación del contenido (Date)
console.log(info.birthtime);     // fecha de creación
console.log(info.isFile(), info.isDirectory(), info.isSymbolicLink());
```

Notas entre sistemas sobre estos campos:

- `size` y `mtime` funcionan igual en todos lados
- `birthtime` es real en Windows y macOS; en Linux depende del sistema de archivos y a veces devuelve la fecha del último cambio de metadatos
- la precisión del reloj cambia: NTFS guarda hasta 100 nanosegundos, otros sistemas redondean al segundo, así que no compares fechas por igualdad exacta

Sobre los permisos, `info.mode` es un número pensado para el modelo de Unix:

```
console.log((info.mode & 0o777).toString(8)); // 644, 755, y así
```

En Linux y macOS eso es exacto. En Windows, Node informa un valor aproximado que solo distingue si el archivo es de solo lectura. Por eso `chmod` en Windows apenas cambia ese atributo y no hace nada más. Si tu programa necesita permisos finos en Windows, el módulo `fs` no alcanza.

`stat` sigue los enlaces simbólicos y te informa del destino. `lstat` te informa del enlace en sí. Si recorres árboles, usá `lstat` para no entrar en bucles infinitos con enlaces circulares.

Estadísticas de texto

```
import { createReadStream } from 'node:fs';
import { createInterface } from 'node:readline';

export async function contar(ruta) {
  const rl = createInterface({
    input: createReadStream(ruta, 'utf8'),
    crlfDelay: Infinity
  });
  let lineas = 0, palabras = 0, caracteres = 0;
  for await (const linea of rl) {
    lineas++;
    caracteres += [...linea].length;
    if (linea.trim()) palabras += linea.trim().split(/\s+/).length;
  }
  return { lineas, palabras, caracteres };
}
```

El conteo de caracteres no coincide con el tamaño en bytes que informa `stat`, por dos razones: los acentos ocupan 2 bytes en UTF-8 y los saltos de línea ocupan 2 en Windows. Es un buen ejercicio para hacer visible la codificación.

Saber qué es un archivo de verdad

La extensión miente. El contenido no. Casi todos los formatos empiezan con una firma de pocos bytes, llamada número mágico, que es la misma en cualquier sistema:

```
import { open } from 'node:fs/promises';

const FIRMAS = [
  { tipo: 'png', bytes: [0x89, 0x50, 0x4e, 0x47] }, // .PNG
  { tipo: 'jpg', bytes: [0xff, 0xd8, 0xff] },
  { tipo: 'gif', bytes: [0x47, 0x49, 0x46, 0x38] }, // GIF8
  { tipo: 'pdf', bytes: [0x25, 0x50, 0x44, 0x46] }, // %PDF
  { tipo: 'zip', bytes: [0x50, 0x4b, 0x03, 0x04] } // PK, también docx, xlsx y jar
];

export async function tipoReal(ruta) {
  const archivo = await open(ruta, 'r');
  try {
    const cabecera = Buffer.alloc(8);
    await archivo.read(cabecera, 0, 8, 0);
    for (const { tipo, bytes } of FIRMAS) {
      if (cabecera.subarray(0, bytes.length).equals(Buffer.from(bytes))) return tipo;
    }
    return 'desconocido';
  } finally {
    await archivo.close();
  }
}
```

Un paso más: el ancho y el alto de un PNG están en la cabecera, en los bytes 16 a 23, como enteros de 4 bytes con el más significativo primero.

```
export async function medidasPNG(ruta) {
  const archivo = await open(ruta, 'r');
  try {
    const cabecera = Buffer.alloc(24);
    await archivo.read(cabecera, 0, 24, 0);
    return { ancho: cabecera.readUInt32BE(16), alto: cabecera.readUInt32BE(20) };
  } finally {
    await archivo.close();
  }
}
```

Leíste 24 bytes de un archivo de 8 MB y obtuviste el dato. Eso es lo que hacen por dentro las librerías de metadatos.

El orden de los bytes, llamado endianness, es del formato, no de la máquina. PNG guarda el byte más significativo primero y por eso se lee con `readUInt32BE` en cualquier sistema. Otros formatos, como BMP, guardan al revés y se leen con `readUInt32LE`.

Formatos de datos

JSON entero, cuando el archivo entra en memoria:

```
const config = JSON.parse(await readFile('config.json', 'utf8'));
```

NDJSON, un objeto JSON por línea, que es el formato correcto para archivos grandes:

```
for await (const linea of rl) {  
  if (!linea.trim()) continue;  
  const registro = JSON.parse(linea);  
}
```

CSV a mano, solo si el archivo es simple y lo controlás vos:

```
const [encabezado, ...filas] = texto.trim().split(/\r?\n/);  
const columnas = encabezado.split(',');  
const datos = filas.map(fila => Object.fromEntries(  
  fila.split(',').map((valor, i) => [columnas[i], valor])  
));
```

Esto se rompe con comillas, comas dentro de un campo y saltos de línea dentro de una celda. Se rompe todavía más con un CSV exportado por Excel en español, que usa punto y coma como separador y viene en Windows-1252. Para CSV real usá una librería probada, como `csv-parse` o `papaparse`. Es un buen ejercicio para mostrarle a un alumno por qué existen esas librerías.

Parte 12: errores

Node normaliza los errores del sistema operativo. Windows devuelve sus propios números, pero la propiedad `code` que ves en JavaScript es la misma en los tres sistemas. Ramificá por `code`, nunca por el texto del mensaje ni por `errno`, que sí cambia.

Código	Qué pasó
<code>ENOENT</code>	la ruta no existe
<code>EEXIST</code>	ya existe y pediste no pisar
<code>EACCES</code>	no tenés permiso
<code>EPERM</code>	operación no permitida, en Windows suele ser un archivo abierto o de solo lectura
<code>EBUSY</code>	el archivo está en uso, casi siempre en Windows
<code>EISDIR</code>	es una carpeta y esperabas un archivo
<code>ENOTDIR</code>	un tramo de la ruta no es una carpeta
<code>ENOTEMPTY</code>	la carpeta tiene contenido
<code>EMFILE</code>	demasiados archivos abiertos a la vez
<code>ENOSPC</code>	no queda espacio en el disco
<code>EXDEV</code>	quisiste renombrar entre dos discos
<code>ENAMETOOLONG</code>	el nombre o la ruta pasa el límite del sistema

```
try {
  await readFile(ruta, 'utf8');
} catch (err) {
  if (err.code === 'ENOENT') return null;
  if (err.code === 'EACCES') throw new Error(`Sin permiso para leer ${ruta}`);
  throw err;
}
```

Cuidado con la concurrencia

Esto revienta con `EMFILE` cuando hay muchos archivos, porque abre todos a la vez:

```
const contenidos = await Promise.all(rutas.map(r => readFile(r, 'utf8')));
```

El sistema limita cuántos archivos puede tener abiertos un proceso. El límite es más bajo en macOS que en Linux, así que este error aparece antes en una Mac. Procesá en lotes:

```

async function enLotes(items, tamaño, tarea) {
  const salida = [];
  for (let i = 0; i < items.length; i += tamaño) {
    salida.push(...await Promise.all(items.slice(i, i + tamaño).map(tarea)));
  }
  return salida;
}

const contenidos = await enLotes(rutas, 50, r => readFile(r, 'utf8'));

```

Parte 13: vigilar cambios

```

import { watch } from 'node:fs/promises';

const ac = new AbortController();
setTimeout(() => ac.abort(), 60_000);

try {
  for await (const evento of watch('datos', { recursive: true, signal: ac.signal })) {
    console.log(evento.eventType, evento.filename);
  }
} catch (err) {
  if (err.name !== 'AbortError') throw err;
}

```

Es la parte de `fs` donde los sistemas más se diferencian, porque cada uno tiene su propio mecanismo: `ReadDirectoryChangesW` en Windows, `FSEvents` en macOS e `inotify` en Linux.

Aspecto	Windows	macOS	Linux
<code>recursive</code>	nativo	nativo	desde Node 20, armado sobre vigilancias simples
Nombre del archivo en el evento	sí	sí	sí, no siempre en subcarpetas
Eventos repetidos por un guardado	frecuentes	frecuentes	frecuentes
Límite propio	ninguno visible	descriptores abiertos	<code>max_user_watches</code> del kernel

Tres advertencias antes de construir algo encima:

- un solo guardado puede disparar 2 o 3 eventos, porque el editor escribe un temporal y renombra

- los nombres pueden llegar en NFD desde macOS: normalizá antes de comparar
- vigilar un árbol grande en Linux puede agotar el límite del kernel y fallar con `ENOSPC`

Si lo necesitás en serio, agregá un retardo antirrebote de 100 milisegundos o usá `chokidar`, que normaliza todo esto.

Diferencias entre sistemas, resumen

Tema	Windows	macOS	Linux
Separador de rutas	<code>`</code>	<code>/</code>	<code>/</code>
Raíz	una por unidad, <code>C:</code>	única, <code>/</code>	única, <code>/</code>
Mayúsculas en nombres	no distingue	no distingue por defecto	distingue
Caracteres prohibidos	<code>` < > : " /</code>	<code>? * `</code> y control	<code>/</code> y nulo <code>/</code> y nulo
Nombres reservados	<code>CON</code> , <code>NUL</code> , <code>COM1</code> , y así	ninguno	ninguno
Largo de ruta	260 salvo configuración	1024	4096
Fin de línea	<code>rn</code>	<code>n</code>	<code>n</code>
Codificación habitual	UTF-8 y Windows-1252	UTF-8	UTF-8
Normalización de nombres	NFC	suele guardar NFD	tal cual se escribió
Borrar o renombrar lo abierto	falla	permitido	permitido
Permisos en <code>stat().mode</code>	solo lectura o escritura	modelo Unix completo	modelo Unix completo
Fecha de creación	sí	sí	depende del sistema de archivos
<code>watch</code> recursivo	nativo	nativo	desde Node 20

Reglas para código que corre en los tres

1. Armá rutas con `path.join` y `path.resolve`, nunca con `+` ni con barras escritas a mano.
 2. Reemplazá las rutas fijas por `os.tmpdir()`, `os.homedir()` e `import.meta.dirname`.
 3. Leé y escribí siempre en UTF-8, y decodificá lo ajeno con `TextDecoder`.
 4. Cortá líneas con `/\r?\n/` y usá `crlfDelay: Infinity` en `readline`.
 5. Escribí `\n` en los archivos que lee otro programa, y `os.EOL` en los que lee una persona.
 6. Cerrá cada archivo que abras, dentro de un `finally`.
 7. Ramificá por `err.code`, nunca por el mensaje del error.
 8. Reintentá ante `EBUSY`, `EPERM` y `EACCES` al renombrar o borrar.
 9. Normalizá los nombres con `normalize('NFC')` antes de compararlos.
 10. Limpiá los nombres generados con el criterio de Windows, que es el más estricto.
 11. Ordená vos el resultado de `readdir` si el orden importa.
 12. Probá alguna vez en otro sistema, o al menos en el mismo con Docker.
-

Ejercicios

1. Escribí un script que reciba una carpeta y muestre los 10 archivos más pesados, con tamaño en KB y fecha de modificación.
 2. Escribí un contador de líneas, palabras y caracteres que funcione con un archivo de 5 GB sin agotar la memoria, y explicá por qué el conteo no coincide con el tamaño que informa `stat`.
 3. Tomá un CSV exportado por Excel en Windows-1252, con punto y coma como separador, y convertilo a UTF-8 separado por comas.
 4. Escribí un buscador de texto que recorra un árbol de carpetas, omita `node_modules` y muestre archivo, número de línea y contenido.
 5. Escribí un detector de duplicados que compare primero por tamaño y solo calcule el hash cuando dos archivos pesen igual.
 6. Escribí una función `guardarConRespaldo` que renombre el archivo anterior a `.bak` antes de escribir el nuevo, de forma atómica y con reintentos.
 7. Escribí un script que detecte archivos con la extensión cambiada, comparando la extensión con el número mágico.
 8. Escribí un validador que reciba una lista de nombres y avise cuáles fallarían en Windows y por qué.
 9. Escribí un organizador que mueva archivos a carpetas por año y mes de modificación, y que resuelva el caso de mover entre dos discos.
-

Resumen

Tarea	Función
leer todo	<code>readFile(ruta, 'utf8')</code>
leer por líneas	<code>readline</code> sobre <code>createReadStream</code>
leer un pedazo	<code>handle.read(buffer, offset, largo, posicion)</code>
escribir	<code>writeFile</code> , <code>appendFile</code>
escribir sin riesgo	temporal con <code>wx</code> , <code>sync</code> y <code>rename</code>
crear carpeta	<code>mkdir(ruta, { recursive: true })</code>
listar	<code>readdir(ruta, { withFileTypes: true })</code>
buscar por nombre	<code>glob('**/*.txt')</code>
buscar por contenido	recorrer y filtrar línea por línea
copiar	<code>copyFile</code> , <code>cp</code> con <code>recursive</code>
mover o renombrar	<code>rename</code> , con respaldo para <code>EXDEV</code>
borrar	<code>unlink</code> , <code>rm</code> con <code>recursive</code>
metadatos	<code>stat</code> , <code>lstat</code>
rutas	<code>path.join</code> , <code>path.resolve</code> , <code>path.extname</code>
carpetas del sistema	<code>os.tmpdir()</code> , <code>os.homedir()</code>

Cuatro ideas para llevarte:

- un archivo son bytes, y esa idea es idéntica en Windows, macOS y Linux
- el texto es una interpretación de esos bytes y siempre necesita una codificación explícita
- las diferencias entre sistemas no están en leer y escribir, sino en cómo se nombran las rutas, cómo terminan las líneas y qué se puede hacer con un archivo abierto
- no preguntes si algo existe: intentá la operación y manejá el `code` del error

Una aclaración final: nada de esto corre en el navegador. Una página web solo accede a los archivos que el usuario elige con un `input type="file"`, o a través de la API File System Access, y siempre con permiso explícito. El acceso libre al disco es propio de Node, en cualquiera de los tres sistemas.

El tipo number en JavaScript

Apunte de clase

De qué se trata

JavaScript tiene un solo tipo para los números: `number`. No hay `int`, `long`, `float` ni `double` por separado. Todo número, tenga o no parte decimal, se guarda igual: como un flotante de 64 bits del estándar IEEE 754.

```
typeof 42;           // 'number'
typeof 3.14;         // 'number'
typeof -0.5;         // 'number'
42 === 42.0;         // true, es el mismo valor
```

Eso simplifica mucho el lenguaje y trae dos consecuencias que hay que conocer: los decimales no siempre son exactos, y los enteros grandes tienen un límite. Las vemos más adelante.

Existe un segundo tipo numérico, `bigint`, para enteros de tamaño arbitrario. Es un tipo aparte y no se mezcla con `number`.

Formas de escribir literales

El mismo valor se puede escribir de varias maneras. La base solo cambia cómo lo escribimos: adentro, el número guardado es el mismo.

```
const decimal = 255;
const hexadecimal = 0xff;           // prefijo 0x, dígitos 0 a 9 y a a f
const octal = 0o377;                // prefijo 0o, dígitos 0 a 7
const binario = 0b11111111;         // prefijo 0b, dígitos 0 y 1

decimal === hexadecimal;             // true, los cuatro son 255
```

Los prefijos no distinguen mayúsculas de minúsculas: `0XFF` y `0xff` son lo mismo. La forma habitual es prefijo en minúscula y dígitos hexadecimales en mayúscula, `0xFF`.

Cuidado con la forma vieja del octal: un cero adelante, como `017`, todavía se interpreta como octal en modo no estricto. Es una fuente de errores conocida y por eso está desaconsejada. Usá siempre `0o17`.

Para los decimales y la notación científica:

```
3.14
.5      // válido, equivale a 0.5
5.      // válido, equivale a 5
1e6     // 1000000, el exponente es potencia de 10
1.5e-3  // 0.0015
```

Desde ES2021 se puede usar el guion bajo como separador visual. El intérprete lo ignora:

```
const poblacion = 1_000_000;
const mascara = 0b1010_0001;
const tarjeta = 0xFF_EC_DE_5E;
```

Los literales de `bigint` llevan una `n` al final:

```
const grande = 9007199254740993n;
typeof grande;      // 'bigint'
grande + 1n;        // funciona
grande + 1;         // TypeError: no se pueden mezclar tipos
```

Valores especiales

Tres valores del tipo `number` no son números comunes.

`Infinity` y `-Infinity` aparecen al desbordar o al dividir por cero. JavaScript no lanza excepción al dividir por cero:

```
1 / 0;      // Infinity
-1 / 0;     // -Infinity
1e400;     // Infinity, se pasó del máximo representable
```

`NaN` significa "no es un número" y aparece cuando una operación aritmética no tiene resultado numérico:

```
0 / 0;      // NaN
Number('hola'); // NaN
Math.sqrt(-1); // NaN
typeof NaN;  // 'number', aunque suene contradictorio
```


`NaN` tiene una propiedad única: no es igual a nada, ni siquiera a sí mismo. Por eso no se lo puede detectar comparando.

```
NaN === NaN;           // false
Number.isNaN(x);       // así se pregunta
isNaN(x);              // la versión global, evitala
```

La función global `isNaN` primero convierte su argumento a número, así que da respuestas engañosas: `isNaN('hola')` devuelve `true` y `isNaN('')` devuelve `false`. `Number.isNaN` no convierte nada y solo devuelve `true` para el `NaN` verdadero.

Precisión y límites

Los flotantes de 64 bits guardan el número en base 2. Algunos decimales que en base 10 son exactos, en base 2 son periódicos, igual que un tercio en decimal.

```
0.1 + 0.2;           // 0.30000000000000004
0.1 + 0.2 === 0.3;   // false
```

No es un error del lenguaje: pasa en todo lenguaje que use IEEE 754, incluido C#. Para comparar decimales conviene usar una tolerancia:

```
Math.abs(a - b) < Number.EPSILON;
```

Para dinero, la solución práctica es trabajar en enteros: guardar centavos en vez de pesos, y dividir solo al mostrar.

Con los enteros el problema es otro: hay un rango donde son exactos y afuera dejan de serlo.

```
Number.MAX_SAFE_INTEGER;           // 9007199254740991, o sea 2^53 - 1
9007199254740992 === 9007199254740993; // true, ya no distingue
Number.isSafeInteger(x);            // para verificar
```

Si necesitás enteros más grandes, como identificadores de base de datos, usá `bigint` o guardalos como cadena.

Conversión explícita

Cuando queremos pasar algo a número, tenemos varias herramientas y cada una sirve para un caso distinto.

`Number(x)` convierte el valor completo. Si sobra cualquier cosa, da `NaN`:

```
Number('42');           // 42
Number('3.14');          // 3.14
Number(' 42 ');          // 42, ignora espacios alrededor
Number('42px');          // NaN
Number('');               // 0
Number(true);             // 1
Number(false);            // 0
Number(null);             // 0
Number(undefined);       // NaN
Number([]);               // 0
Number([5]);              // 5
Number([1, 2]);           // NaN
Number({});               // NaN
```

El más `+` unario hace exactamente lo mismo que `Number(x)`, en forma abreviada:

```
+'42';           // 42
+true;           // 1
```

`parseInt(cadena, base)` lee desde el principio y corta en el primer carácter que no sirve. Es lo contrario de `Number`: acepta que sobre texto.

```
parseInt('42px');      // 42
parseInt('3.9');        // 3, corta en el punto
parseInt('px42');       // NaN, no arrancó con dígito
parseInt('ff', 16);     // 255
parseInt('1010', 2);    // 10
```

Pasá siempre la base, aunque sea 10. Sin ella, una cadena que empieza con `0x` se interpreta en hexadecimal y el resultado sorprende.

`parseInt` espera una cadena. Si le pasás un número, primero lo convierte a texto, y ahí aparecen efectos raros:

```
parseInt(0.0000005);    // 5, porque el número se escribe como '5e-7'
```

`parseFloat(cadena)` es igual pero acepta la parte decimal:

```
parseFloat('3.14 metros'); // 3.14
parseFloat('1.5e3');         // 1500
```

Para truncar o redondear un número que ya es número, la herramienta correcta es `Math`:

```
Math.trunc(3.9);    // 3, corta la parte decimal
Math.trunc(-3.9);   // -3
Math.floor(-3.9);   // -4, redondea hacia abajo
Math.round(3.5);    // 4
Math.ceil(3.1);     // 4
```

En el sentido inverso, `toString(base)` pasa de número a cadena en la base que pidas:

```
(255).toString(2);    // '11111111'
(255).toString(16);   // 'ff'
(10).toString();      // '10'
```

`toFixed` también devuelve una cadena, no un número. Es para mostrar, no para calcular:

```
const precio = 3.14159;
precio.toFixed(2);      // '3.14', es texto
+precio.toFixed(2);     // 3.14, ahora sí es número
```

Conversión implícita

Acá aparece el tipado débil. Los operadores aritméticos aceptan cualquier valor y lo convierten solos a número, con una excepción grande: el `+`.

El `+` está sobrecargado. Si alguno de los dos operandos es una cadena, concatena. Si no, suma.

```
5 + 2;           // 7
'5' + 2;         // '52', concatena
5 + '2';         // '52'
'5' + true;      // '5true'
```

Todos los demás operadores aritméticos convierten a número, siempre:

```
'5' - 2;        // 3
'5' * '2';       // 10
'6' / '2';       // 3
'5' % 2;         // 1
'hola' * 2;      // NaN
```

Como el `+` se evalúa de izquierda a derecha, el orden cambia el resultado:

```
1 + 2 + '3';    // '33', primero suma 3 y después concatena
'1' + 2 + 3;    // '123', concatena todo
```

De acá sale un consejo práctico: convertí a número apenas entra el dato, no en el medio de la cuenta. Todo lo que viene de un formulario, de un `prompt`, de un archivo de texto o de una URL llega como cadena.

```
const cantidad = Number(input.value);
if (Number.isNaN(cantidad)) { /* avisar del error */ }
```

Los operadores de comparación también convierten, y ahí hay una trampa clásica:

```
'10' > 9;        // true, compara 10 con 9 como números
'10' > '9';       // false, compara texto contra texto, carácter por carácter
```

Con dos cadenas, JavaScript compara alfabéticamente, y el carácter `'1'` viene antes que `'9'`. El mismo problema aparece al ordenar un arreglo de números con `sort` sin función de comparación.

Otra rareza conocida, con `null`:

```
null == 0;       // false
null > 0;         // false
null >= 0;        // true
```

No hay contradicción: `>=` convierte `null` a 0, y `==` tiene una regla propia que solo empareja `null` con `undefined`. La conclusión de siempre: usá `===`, que no convierte nada.

Operadores aritméticos

Son 7, más los dos de incremento:

Operador	Nombre	Ejemplo
<code>+</code>	suma	<code>2 + 3</code> da 5
<code>-</code>	resta	<code>5 - 2</code> da 3
<code>*</code>	producto	<code>4 * 3</code> da 12
<code>/</code>	división	<code>7 / 2</code> da 3.5
<code>%</code>	resto	<code>7 % 2</code> da 1
<code>**</code>	potencia	<code>2 ** 10</code> da 1024
<code>-</code>	negación unaria	<code>-x</code>
<code>++</code>	incremento	<code>i++</code> o <code>++i</code>
<code>--</code>	decremento	<code>i--</code> o <code>--i</code>

Tres detalles.

La división nunca es entera. `7 / 2` da 3.5, no 3. Para división entera usá `Math.trunc(7 / 2)`.

El `%` devuelve el resto, no el módulo matemático. Con negativos, el signo lo pone el dividendo:

```
7 % 3;      // 1
-7 % 3;     // -1, no 2
```

El `**` es asociativo a derecha, al revés que los demás, y no se puede combinar con el menos unario sin paréntesis:

```
2 ** 3 ** 2;    // 512, es 2 ** (3 ** 2)
-2 ** 2;        // SyntaxError
(-2) ** 2;      // 4
```

En `i++` y `i--` importa la posición. En forma sufija devuelve el valor viejo y después incrementa; en forma prefija incrementa primero. Si el resultado no se usa, da lo mismo.

```
let i = 5;
console.log(i++); // imprime 5, ahora i vale 6
console.log(++i); // imprime 7
```

Precedencia de operadores

Cuando una expresión mezcla operadores, el lenguaje aplica primero los de mayor precedencia. De mayor a menor:

Nivel	Operadores
1	<code>()</code> agrupación
2	<code>++</code> <code>--</code> sufijos
3	<code>!</code> <code>~</code> <code>+</code> <code>-</code> unarios, <code>typeof</code>
4	<code>**</code>
5	<code>*</code> <code>/</code> <code>%</code>
6	<code>+</code> <code>-</code>
7	<code><<</code> <code>>></code> <code>>>></code>
8	<code><</code> <code><=</code> <code>></code> <code>>=</code>
9	<code>==</code> <code>!=</code> <code>===</code> <code>!==</code>
10	<code>&</code>
11	<code>^</code>
12	<code> </code>
13	<code>&&</code>
14	<code> </code> <code>??</code>
15	<code>=</code> <code>+=</code> <code>-=</code> y demás asignaciones

Dos cosas para retener de esta tabla.

Lo esperable se cumple: `2 + 3 * 4` da 14, porque el producto va primero.

Lo inesperado es que los operadores de bits están más abajo que los de comparación. Eso rompe expresiones que parecen obvias:

```
flags & LEER === LEER;    // se agrupa como flags & (LEER === LEER)
(flags & LEER) === LEER;  // así se escribe
```

Ante la duda, poné paréntesis. Nadie los cuenta y se leen mejor que la tabla.

Operadores de comparación

Operador	Significado
<code>===</code>	igual en valor y tipo, sin conversión
<code>!==</code>	distinto en valor o tipo
<code>==</code>	igual con conversión previa
<code>!=</code>	distinto con conversión previa
<code><</code> <code><=</code> <code>></code> <code>>=</code>	orden

Usá `===` y `!==` por defecto. El único uso razonable de `==` es `x == null`, que da `true` para `null` y para `undefined` a la vez.

Todos devuelven un booleano genuino. Con `NaN` de por medio, todas las comparaciones dan `false`, incluida la de igualdad.

Operadores de bits

Los operadores de bits trabajan sobre la representación binaria del número. Sirven para empaquetar varios datos chicos en un solo valor, para trabajar con máscaras y para hablar con APIs de bajo nivel.

Cómo se convierten los flotantes a enteros

Acá pasa algo importante. Los números son flotantes de 64 bits, pero los operadores de bits no trabajan con flotantes. Antes de operar, JavaScript convierte cada operando a un entero de 32 bits con signo, en complemento a 2. Al terminar, convierte el resultado de vuelta a `number`.

La conversión hace dos cosas: descarta la parte decimal truncando hacia cero, y se queda con los 32 bits de menor peso.

De ahí sale el truco de la barra cero. Como `x | 0` no cambia ningún bit, el único efecto visible es la conversión a entero:

```
3.9 | 0;      // 3
-3.9 | 0;     // -3, trunca hacia cero
~3.9;        // 3, el mismo truco con doble negación de bits
```

Es rápido y compacto, pero tiene un límite: solo funciona dentro del rango de 32 bits. Afuera, el resultado es basura.

```
(2 ** 31) | 0;      // -2147483648, se desbordó
3000000000 | 0;    // -1294967296
```

Por eso, en código para leer, es mejor `Math.trunc(x)`. El truco conviene conocerlo porque aparece por todos lados en código ajeno.

Los operadores

Operador	Nombre	Qué hace
<code>&</code>	AND	1 donde ambos bits son 1
<code> </code>	OR	1 donde al menos un bit es 1
<code>^</code>	XOR	1 donde los bits son distintos
<code>~</code>	NOT	invierte todos los bits
<code><<</code>	desplazamiento a izquierda	corre los bits y rellena con ceros
<code>>></code>	desplazamiento a derecha con signo	corre los bits y rellena con el bit de signo
<code>>>></code>	desplazamiento a derecha sin signo	corre los bits y rellena con ceros

Son los mismos AND, OR y NOT del álgebra de Boole, aplicados bit por bit en paralelo sobre los 32 bits.

```
const a = 0b1100;  // 12
const b = 0b1010;  // 10

a & b;      // 0b1000, o sea 8
a | b;      // 0b1110, o sea 14
a ^ b;      // 0b0110, o sea 6
~a;         // -13, porque ~x es siempre -(x + 1)
```

Para ver el resultado en binario conviene formatearlo:

```
(a & b).toString(2).padStart(4, '0');  // '1000'
```

Los desplazamientos multiplican y dividen por potencias de 2:


```
5 << 1;    // 10, equivale a 5 * 2
5 << 3;    // 40, equivale a 5 * 8
20 >> 2;    // 5, equivale a Math.trunc(20 / 4)
-8 >> 1;    // -4, mantiene el signo
-8 >>> 1;   // 2147483644, rellena con ceros y el signo se pierde
```

La diferencia entre `>>` y `>>>` solo se nota con números negativos. `-1 >>> 0` devuelve 4294967295 y es el modismo habitual para leer un valor de 32 bits como si no tuviera signo.

Ejemplo: opciones simultáneas con banderas de bits

Este es el uso clásico. Queremos representar un conjunto de opciones que pueden estar activas al mismo tiempo. En vez de varias variables booleanas, usamos un solo número y le asignamos un bit a cada opción.

Es lo mismo que hace Unix con los permisos de archivos: el 6 de `chmod 640` es lectura más escritura.

Definimos las opciones con un bit encendido cada una:

```
const LEER      = 1 << 0;    // 0b0001, o sea 1
const ESCRIBIR  = 1 << 1;    // 0b0010, o sea 2
const BORRAR    = 1 << 2;    // 0b0100, o sea 4
const ADMIN     = 1 << 3;    // 0b1000, o sea 8
```

Cada valor tiene un solo bit en 1, y ninguno coincide con otro. Escribirlos con `<<` deja claro que son potencias de 2 y evita el error de repetir un valor.

Combinar varias opciones se hace con OR, porque OR enciende bits:

```
let permisos = LEER | ESCRIBIR;    // 0b0011, o sea 3
```

Consultar si una opción está activa se hace con AND, porque AND filtra. Se llama aplicar una máscara: el resultado conserva solo los bits que están en los dos lados.

```
(permisos & ESCRIBIR) !== 0;    // true
(permisos & BORRAR) !== 0;      // false
```

Los paréntesis no son opcionales, por lo que vimos en la tabla de precedencia.

Las cuatro operaciones básicas sobre el conjunto:

```

permisos |= BORRAR;           // prender: 0b0011 pasa a 0b0111
permisos &= ~BORRAR;          // apagar: vuelve a 0b0011
permisos ^= ADMIN;            // alternar: si estaba lo apaga, si no lo prende
permisos = 0;                 // limpiar todo

```

La de apagar merece atención. `~BORRAR` invierte todos los bits, así que da un número con unos en todas las posiciones menos la de `BORRAR`. Al hacer AND con eso, todos los bits sobreviven salvo el que queríamos apagar.

Preguntas más elaboradas:

```

const REQUERIDOS = LEER | ESCRIBIR;

(permisos & REQUERIDOS) === REQUERIDOS; // tiene todos los requeridos
(permisos & REQUERIDOS) !== 0;           // tiene al menos uno
permisos === 0;                          // no tiene ninguno

```

Y una función para mostrar el estado en forma legible:

```

const NOMBRES = { LEER, ESCRIBIR, BORRAR, ADMIN };

function describir(permisos) {
  return Object.entries(NOMBRES)
    .filter(([bit]) => (permisos & bit) !== 0)
    .map([nombre]) => nombre
    .join(', ') || 'sin permisos';
}

describir(LEER | ADMIN); // 'LEER, ADMIN'

```

Las ventajas son el tamaño y la velocidad: 32 opciones entran en un número, las consultas son una instrucción de máquina, y todo el conjunto viaja como un solo valor.

Las limitaciones también son claras. Entran 32 banderas como máximo, porque los operadores trabajan en 32 bits. Y el código es más difícil de leer que un objeto con propiedades booleanas.

Hoy, para lógica de negocio propia, un `Set` o un objeto de booleanos suele ser mejor opción. Las banderas de bits siguen siendo la forma correcta cuando hay que hablar con una API que las usa, como los módulos de archivos de Node, WebGL o los eventos del navegador.

Resumen

- hay un solo tipo `number`, flotante de 64 bits, más `bigint` para enteros grandes

- los literales se escriben en decimal, hexadecimal con `0x`, octal con `0o` y binario con `0b`, y admiten `_` como separador
- `Infinity` y `NaN` son valores del tipo `number`; `NaN` no es igual a nada y se detecta con `Number.isNaN`
- los decimales no son exactos y los enteros son seguros hasta $2^{53} - 1$
- convertí con `Number`, `parseInt` con base, `parseFloat` o `+` unario, y volvé a texto con `toString(base)`
- el `+` concatena si hay una cadena; el resto de los operadores aritméticos convierten a número
- la comparación entre cadenas es alfabética, así que convertí antes de comparar
- los operadores de bits convierten a entero de 32 bits con signo, de donde sale el truco de `| 0`
- con `|`, `&`, `~` y `<<` se arma un conjunto de opciones simultáneas en un solo número

Para pensar

1. Escribí el número 202 como literal decimal, hexadecimal, octal y binario, y verificá con `===` que son el mismo valor.
2. Explicá por qué `'5' + 3` da `'53'` pero `'5' - 3` da 2.
3. Ordená el arreglo `[10, 9, 100, 1]` con `sort()` sin argumentos y explicá el resultado.
4. Agregá una opción `COMPARTIR` al ejemplo de permisos y escribí las expresiones para prenderla, apagarla y consultarla.
5. Escribí una función `contarBits(n)` que devuelva cuántos bits en 1 tiene un número, usando `&` y `>>`.
6. Averiguá qué devuelve `(permisos & ESCRIBIR)` cuando la opción está activa, y explicá por qué comparamos con `!== 0` en vez de con `=== true`.

Expresiones regulares en JavaScript

Programación IV: Desarrollo Web — Etapa 1, fundamentos técnicos Apunte complementario de la unidad de JavaScript

Hasta ahora buscaste contenido exacto. `texto.includes("hola")` pregunta si esas cuatro letras están ahí.

Pero casi nada de lo que necesitás validar en una aplicación web tiene contenido exacto. Un email no es una cadena concreta: es cualquier cadena con arroba en el medio y un punto después. Un teléfono no es un número fijo: es cualquier secuencia de dígitos con espacios, guiones y paréntesis salpicados. Una fecha, un CUIT, un código postal, un nombre de archivo: todos son una forma, no un valor.

Las expresiones regulares son el lenguaje para describir formas. Este apunte arranca por lo que son, sigue por cómo se escriben, explica cómo funciona el motor por dentro y termina en recetas que vas a copiar al proyecto de la materia.

Qué es una expresión regular

Una expresión regular describe un conjunto de cadenas.

Conviene tomar esa definición literal. La expresión `/ab/` no es una instrucción para buscar. Es el nombre de un conjunto con un solo elemento: la cadena `"ab"`. La expresión `/ab*/` nombra un conjunto infinito: `"a"`, `"ab"`, `"abb"`, `"abbb"` y así para siempre.

Cuando escribís `patron.test(texto)` no le pedís al motor que busque. Le preguntás si alguna parte de `texto` pertenece al conjunto que nombra `patron`. Todo lo demás son variaciones de esa misma pregunta: dónde empieza la parte que pertenece, cuáles son todas las partes que pertenecen, con qué reemplazarlas.

Pensarlas como conjuntos evita el error más común, que es leerlas como si fueran un programa que camina el texto. El motor sí camina el texto, y más adelante vamos a ver cómo, pero el significado de la expresión no depende de ese recorrido.

Las tres operaciones que generan todo

Toda la sintaxis, con sus decenas de símbolos, sale de tres operaciones sobre conjuntos. Si entendés las tres, el resto es notación abreviada.

Concatenación: pegar un conjunto después de otro. Si un conjunto tiene "a" y el otro tiene "b", la concatenación tiene "ab". Se escribe poniendo las cosas una al lado de la otra.

```
/ab/.test("ab");    // true
/ab/.test("ba");    // false
```

Alternancia: la unión de dos conjuntos. Se escribe con la barra vertical.

```
/gato|perro/.test("tengo un perro");    // true
```

Repetición: todas las repeticiones de un conjunto, incluida la vacía. Se escribe con el asterisco.

```
/ab*/.test("a");        // true, cero repeticiones de b
/ab*/.test("abbb");     // true
```

Con esas tres alcanza para describir cualquier lenguaje regular. Falta una pieza de notación, no de teoría: los paréntesis, que sirven para decir a qué se aplica cada operación.

```
/gato|perros/;          // gato, o perros
/(gato|perro)s/;        // gatos, o perros
/ab*/;                  // a seguida de cero o más b
/(ab)*/;                // cero o más repeticiones de ab
```

El asterisco y la barra tienen precedencia distinta, igual que la multiplicación y la suma. La repetición se aplica al último elemento; la alternancia parte la expresión entera en dos. Cuando dudes, poné paréntesis.

Cómo se escribe una expresión regular en JavaScript

Hay dos formas de crear el objeto.

El literal va entre barras y se compila cuando el archivo se carga:

```
const patron = /\d+/g;
```

El constructor recibe una cadena y se compila en tiempo de ejecución:

```
const patron = new RegExp("\\d+", "g");
```

Las dos producen un objeto `RegExp` idéntico. La diferencia práctica es el escape. En el literal escribís la expresión tal cual. En el constructor la expresión viaja dentro de una cadena, así que las barras invertidas de la expresión hay que duplicarlas: la cadena `"\\d"` contiene los dos caracteres que el motor necesita ver.

Usá el literal siempre que el patrón sea fijo. Usá el constructor solo cuando el patrón se arma en tiempo de ejecución, típicamente porque una parte viene de una variable:

```
function buscarPalabra(texto, palabra) {  
  const patron = new RegExp(`\\b${palabra}\\b`, "iu");  
  return patron.test(texto);  
}
```

Ese código tiene un problema que vamos a arreglar más adelante: si `palabra` trae un punto, un paréntesis o un asterisco, esos caracteres no se buscan literalmente, se interpretan como sintaxis.

Cómo se hace la pregunta

El patrón por sí solo no hace nada. Hay siete formas de consultarlo y cada una responde una pregunta distinta.

`test` devuelve si hay al menos una coincidencia. Es la que querés para validar:

```
/^\d{4}$/.test("2026");    // true
```

`search` devuelve la posición de la primera coincidencia, o -1:

```
"pedido 4471".search(/\d/);    // 7
```

`match` sin la bandera `g` devuelve la primera coincidencia con todo su detalle: el texto que coincidió, los grupos, la posición. Con la bandera `g` devuelve solo el arreglo de textos que coincidieron, sin detalle:

```
"tel 4451-2233".match(/(\d{4})-(\d{4})/);  
// ["4451-2233", "4451", "2233", index: 4, ...]  
  
"a1 b2 c3".match(/\d/g);  
// ["1", "2", "3"]
```

`matchAll` devuelve todas las coincidencias con todo su detalle. Exige la bandera `g` y devuelve un iterador, así que se recorre con `for of` o se expande con el operador de propagación:

```
for (const m of "a1 b2".matchAll(/(\w)(\d)/g)) {  
  console.log(m[1], m[2], m.index);  
}
```

`replace` reemplaza la primera coincidencia, o todas si el patrón tiene `g`. `replaceAll` reemplaza todas y exige `g` cuando le pasás una expresión regular:

```
" hola mundo ".replace(/\s+/g, " ").trim(); // "hola mundo"
```

`split` parte la cadena usando el patrón como separador:

```
"uno, dos ,tres".split(/\s*,\s*/); // ["uno", "dos", "tres"]
```

`exec` es la forma cruda: devuelve una coincidencia por llamada y avanza. Sirve para recorridos manuales, pero `matchAll` cubre casi todos los casos con menos filo.

El objeto con estado

Un patrón con la bandera `g` o `y` guarda una propiedad `lastIndex` y la modifica cada vez que lo usás con `test` o `exec`. Esto sorprende a todo el mundo una vez:

```
const patron = /a/g;  
patron.test("aa"); // true, lastIndex queda en 1  
patron.test("aa"); // true, lastIndex queda en 2  
patron.test("aa"); // false, y lastIndex vuelve a 0
```

La regla práctica es simple: no reutilices un patrón con `g` para validar. Decláralo sin `g` si vas a usar `test`, o creá el literal dentro de la función. Un patrón con `g` declarado como constante en el módulo y usado en un bucle de validación es un error difícil de encontrar, porque falla de forma intermitente.

Clases de caracteres

Los corchetes nombran un conjunto de un solo carácter. Adentro, casi todos los símbolos pierden su significado especial.

```

/[aeiou]/;      // una vocal
/[a-z]/;        // una letra minúscula, por rango
/[a-zA-Z0-9]/;  // varios rangos juntos
/[^0-9]/;       // cualquier carácter que no sea un dígito

```

El acento circunflejo al principio niega la clase. En cualquier otra posición es un carácter común. El guion es un rango entre dos caracteres, salvo al principio o al final, donde es un guion literal: `[-+]` son los dos signos.

Hay atajos para las clases más usadas:

atajo	equivale a	significado
<code>d</code>	<code>[0-9]</code>	dígito
<code>D</code>	<code>[^0-9]</code>	no dígito
<code>w</code>	<code>[A-Za-z0-9_]</code>	carácter de palabra
<code>W</code>	<code>[^A-Za-z0-9_]</code>	no carácter de palabra
<code>s</code>	espacios, tabulaciones, saltos de línea	espacio en blanco
<code>S</code>	lo contrario	no espacio en blanco

Fijate que `\w` es ASCII puro. No incluye la ñe ni las vocales acentuadas. Para texto en castellano, `\w` es casi siempre la elección equivocada; más abajo está la alternativa.

El punto es una clase especial: coincide con cualquier carácter menos los saltos de línea. Con la bandera `s` también coincide con los saltos. Dentro de los corchetes el punto es un punto literal, así que `[.]` y `\.` son lo mismo.

Cuantificadores

Los cuantificadores repiten lo que tienen a la izquierda.

```

/a?/;          // cero o una
/a*/;          // cero o más
/a+/;          // una o más
/a{3}/;        // exactamente tres
/a{2,}/;       // dos o más
/a{2,5}/;      // entre dos y cinco

```


Todos son codiciosos: agarran la mayor cantidad posible y después devuelven caracteres si hace falta para que el resto de la expresión funcione. Agregarles un signo de pregunta los vuelve perezosos: agarran la menor cantidad posible y van sumando solo si hace falta.

La diferencia se ve mejor con un ejemplo. Sobre el texto `"<b>hola</b>"`:

```
"<b>hola</b>".match(/<.+>/) [0];    // "<b>hola</b>"
"<b>hola</b>".match(/<.+?>/) [0];    // "<b>"
```

El codicioso se comió todo hasta el último `>` y funcionó igual, porque quedaba un `>` para cerrar. El perezoso se detuvo en el primero. Cuando extraés algo delimitado, casi siempre querés el perezoso.

Anclas y bordes

Las anclas no consumen caracteres. Afirman algo sobre la posición.

`^` afirma que estás al principio del texto. `$` afirma que estás al final. Con la bandera `m` pasan a valer por línea en vez de por texto completo.

Esta distinción es la diferencia entre buscar y validar:

```
/\d{4}/.test("abc 2026 def");    // true, hay cuatro dígitos ahí
/^d{4}$/.test("abc 2026 def");    // false, el texto entero no es cuatro dígitos
```

Toda validación de formulario lleva `^` y `$`. Sin ellos, cualquier basura alrededor pasa el control.

`\b` afirma un borde de palabra: la posición donde un `\w` toca un no `\w`. Sirve para buscar una palabra completa:

```
/\bsol\b/.test("hace sol");        // true
/\bsol\b/.test("solamente");       // false
```

Y acá se cobra que `\w` sea ASCII. Como la ñe y las acentuadas no son caracteres de palabra para el motor, los bordes caen en lugares raros:

```
/\bcafé\b/.test("tomé café con leche");    // false
```

Falla porque entre la `é` y el espacio no hay transición: los dos son no palabra, así que no hay borde. Para texto en castellano conviene evitar `\b` y usar miradas, que están más abajo.

Grupos

Los paréntesis, además de agrupar para la precedencia, capturan lo que coincidió adentro. Se numeran de izquierda a derecha por el paréntesis que abre, empezando en 1.

```
const m = "2026-08-24".match(/(\d{4})-(\d{2})-(\d{2})/);
m[0];    // "2026-08-24"
m[1];    // "2026"
m[2];    // "08"
```

Los grupos con nombre se leen mucho mejor y no se rompen cuando agregás un paréntesis en el medio:

```
const patron = /(?!<anio>\d{4})-(?!<mes>\d{2})-(?!<dia>\d{2})/;
const { anio, mes, dia } = "2026-08-24".match(patron).groups;
```

Si agrupás solo por precedencia y no necesitás el contenido, usá el grupo sin captura. Es más claro y evita numeración inútil:

```
/(?:ab)+/;
```

Una referencia hacia atrás pide que algo se repita igual, no que tenga la misma forma:

```
/(\w)\1/.test("aa");           // true
/(\w)\1/.test("ab");           // false
/(?!<c>\w)\k<c>/test("aa");     // true, con nombre
```

En el reemplazo, los grupos vuelven como `$1`, `$2` o `@@CODE2@@lt;nombre>`:

```
"2026-08-24".replace(/(\d{4})-(\d{2})-(\d{2})/, "$3/$2/$1");
// "24/08/2026"
```

Y si el reemplazo necesita lógica, pasale una función. Recibe la coincidencia, los grupos, la posición y el texto completo:

```
"precio: 1200".replace(/\d+/g, (n) => Number(n) * 1.21);
// "precio: 1452"
```

Miradas

Una mirada afirma qué hay alrededor sin consumirlo. Son cuatro y se leen como su nombre indica.

```
/hola(?:\s)/;      // hola, si le sigue un espacio
/hola(?:!\s)/;      // hola, si no le sigue un espacio
/(?<=\$)\d+/;       // dígitos, si vienen después de un signo pesos
/(?<!\$)\d+/;        // dígitos, si no vienen después de un signo pesos
```

Lo que está adentro de la mirada no forma parte de la coincidencia. Por eso sirven para recortar con precisión:

```
"vale $1200".match(/(?:<=\$)\d+/)[0];    // "1200", sin el signo
```

Y también para pedir varias condiciones sobre el mismo texto al mismo tiempo, porque cada mirada empieza a evaluar desde la misma posición:

```
const segura = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d){8,}$/;
segura.test("Contra1234");    // true
segura.test("contra1234");    // false, falta mayúscula
```

Esa es la forma idiomática de validar una contraseña con varias reglas. Cada mirada es una regla independiente y las agregás o quitás sin tocar el resto.

Banderas

Las banderas van después de la barra de cierre y cambian el modo de trabajo del motor.

bandera	efecto
<code>g</code>	busca todas las coincidencias, no solo la primera
<code>i</code>	ignora mayúsculas y minúsculas
<code>m</code>	<code>^</code> y <code>\$</code> valen por línea
<code>s</code>	el punto también coincide con saltos de línea
<code>u</code>	modo Unicode, habilita <code>p{...}</code>
<code>v</code>	modo Unicode extendido, con operaciones entre clases
<code>y</code>	modo pegajoso, la coincidencia debe empezar en <code>lastIndex</code>
<code>d</code>	agrega las posiciones de inicio y fin de cada grupo

La bandera `y` sirve para escribir analizadores léxicos, donde cada token tiene que empezar exactamente donde terminó el anterior. La bandera `d` sirve para resaltar coincidencias en pantalla, porque te da los índices de cada grupo y no solo el texto.

Unicode

Sin la bandera `u`, el motor trabaja con unidades de código de 16 bits. Cualquier carácter fuera del plano básico, como un emoji o una letra de un alfabeto extendido, ocupa dos unidades y el punto coincide con la mitad.

```
"😄".length;           // 2
"😄".match(/./)[0].length; // 1, media coincidencia
"😄".match(/./u)[0].length; // 2, el carácter completo
```

Con `u` habilitás además las propiedades Unicode, que son la respuesta correcta al problema de `\w` y el castellano:

```
/^\p{L}+$/u.test("Núñez");           // true, solo letras
(/^\p{Lu}/u.test("Núñez"));           // true, empieza con mayúscula
/^\p{L}\p{Nd}+$/u.test("año2026"); // letras y dígitos de cualquier alfabeto
```

Las propiedades más útiles para validar datos de personas son `\p{L}` para letras, `\p{Lu}` y `\p{Ll}` para mayúsculas y minúsculas, `\p{Nd}` para dígitos decimales, `\p{P}` para puntuación y `\p{Diacritic}` para los signos que se combinan con una letra.

La bandera `v`, más reciente, agrega operaciones entre clases. La diferencia con `--` y la intersección con `&&` evitan clases enormes escritas a mano:

```
/^\p{L}--\p{Script=Han}+$/v; // letras, menos las chinas
```

Poné siempre `u` cuando el texto venga de un usuario. No cuesta nada y evita una familia entera de errores.

Cómo funciona el motor por dentro

Hasta acá vimos qué significa cada símbolo. Ahora hace falta el modelo mecánico, porque explica tanto los límites como los problemas de rendimiento.

El modelo teórico de una expresión regular es un autómata finito: una máquina con una cantidad fija de estados que lee el texto de izquierda a derecha, un carácter por vez, y cambia de estado según lo que lee. No tiene memoria más allá del estado en el que está.

De ahí sale el límite más importante: una expresión regular no puede contar. No puede verificar que los paréntesis de una fórmula estén balanceados, porque para eso necesitaría llevar la cuenta de cuántos abrió, y esa cuenta no tiene límite. No puede validar HTML anidado por la misma razón. Cada vez que alguien intenta, escribe un monstruo de 200 caracteres que funciona con los casos que probó y falla con el siguiente.

La regla es clara: si la estructura se anida sin límite, necesitás un analizador, no una expresión regular. Para HTML usá el DOM. Para JSON usá `JSON.parse`. Para CSV con comillas usá una librería.

El motor de JavaScript, además, no es un autómata puro. Implementa la búsqueda con retroceso: prueba una alternativa, y si más adelante falla, vuelve atrás y prueba la siguiente. Ese retroceso es lo que permite las referencias hacia atrás y las miradas, que un autómata puro no puede expresar. También es lo que puede hacer explotar el tiempo de ejecución.

Retroceso catastrófico

Cuando dos cuantificadores anidados describen el mismo texto de muchas formas distintas, la cantidad de combinaciones que el motor prueba antes de rendirse crece de forma exponencial.

```
const malo = /^(a+)+$/;  
malo.test("aaaaaaaaaaaaaaaaaaaaaaaaaaaaa!");
```

Con 30 caracteres, ese `test` puede tardar varios segundos. Con 40, minutos. El texto no coincide por el signo de admiración final, pero antes de saberlo el motor prueba todas las maneras de repartir 30 letras `a` entre los dos cuantificadores.

En el navegador congela una pestaña. En el servidor es peor: Node ejecuta el patrón en el hilo principal, así que una sola petición con un texto armado a propósito deja de responder a todas las demás. El ataque tiene nombre propio, ReDoS, y es de los más fáciles de meter sin darse cuenta.

Las señales de alarma son tres: un cuantificador dentro de otro, alternativas que pueden coincidir con lo mismo, y patrones armados con datos del usuario. La corrección casi siempre es simplificar:

```
const bueno = /^a+$/;    // describe el mismo conjunto, sin ambigüedad
```

Cuando el patrón viene de afuera, validá que el texto de entrada tenga un largo máximo antes de aplicarlo, y desconfiá de cualquier expresión regular que aceptes como configuración.

Escapar texto literal

Volvamos al problema que dejamos pendiente. Si insertás texto del usuario dentro de un patrón, sus caracteres especiales se interpretan como sintaxis. Un punto pasa a coincidir con cualquier cosa, y un paréntesis suelto rompe la compilación con un error.

Los runtimes recientes traen `RegExp.escape`, que resuelve esto en una línea. Si no podés contar con él, la función es corta:

```
function escapar(texto) {
  return texto.replace(/[\.*+?${}()|\\|\/]/g, "\\$&");
}

const patron = new RegExp(escapar(entradaDelUsuario), "giu");
```

El `@CODE0@amp;` en el reemplazo significa la coincidencia completa, así que cada carácter especial queda precedido por una barra invertida.

Recetas para el proyecto de la materia

Estas son las que vas a necesitar en la agenda de contactos. Están pensadas para copiarse, pero leé la advertencia de cada una.

Email, versión pragmática:

```
const EMAIL = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
```

No intentes escribir el email perfecto. La definición formal del estándar admite comillas, comentarios y direcciones que ningún sistema real acepta, y la expresión que la cubre entera tiene miles de caracteres. Lo único que un patrón puede darte es un filtro de errores de tipeo. La única validación real de un email es mandarle un correo.

Teléfono argentino, normalizando primero:

```
function normalizarTelefono(texto) {
  const soloDigitos = texto.replace(/\D/g, "");
  return /^(?:54)?9?\d{10}$/.test(soloDigitos) ? soloDigitos : null;
}
```

La estrategia general es esa: primero limpiás con `replace`, después validás sobre el resultado limpio. Un patrón que acepte todas las formas de escribir un teléfono, con y sin código de país, con paréntesis, guiones o espacios, es inmantenible.

Fecha en formato ISO:

```
const ISO = /^(?<anio>\d{4})-(?<mes>\d{2})-(?<dia>\d{2})$/;

function parsearFecha(texto) {
  const m = ISO.exec(texto);
  if (!m) return null;
  const { anio, mes, dia } = m.groups;
  const fecha = new Date(Number(anio), Number(mes) - 1, Number(dia));
  return fecha.getMonth() === Number(mes) - 1 ? fecha : null;
}
```

Acá está la lección más importante de todo el apunte, así que vale repetirla. La expresión regular valida la forma, no la validez. "2026-02-31" tiene forma perfecta y no existe. La forma la controla el patrón; la validez la controla el código.

Lo mismo pasa con el CUIT. El patrón `/^\d{2}-?\d{8}-?\d$/` verifica la forma, pero el dígito verificador se calcula, y ese cálculo no se puede escribir como expresión regular.

Nombre de persona, sin ASCII de por medio:

```
const NOMBRE = /^\\p{Lu}\\p{L}+(?:[ '-]\\p{Lu}?\\p{L}+)*$/u;
NOMBRE.test("María del Carmen"); // true
NOMBRE.test("O'Brien"); // true
```

Espacios de más en cualquier campo de texto:

```
const limpio = entrada.trim().replace(/\\s+/g, " ");
```

Convertir un título en identificador para la URL:

```
function aSlug(texto) {
  return texto
    .normalize("NFD")
    .replace(/\\p{Diacritic}/gu, "")
    .toLowerCase()
    .replace(/^[a-z0-9]+/g, "-")
    .replace(/^-+|\\-+$/g, "");
}

aSlug("Programación IV: Desarrollo Web"); // "programacion-iv-desarrollo-web"
```

La normalización a NFD separa cada letra acentuada en letra más acento, y recién entonces se pueden borrar los acentos con una propiedad Unicode.

Extraer datos de texto libre, con `matchAll`:

```
const texto = `
Ana Pérez - ana@ejemplo.com - 4451-2233
Luis Gómez - luis@ejemplo.com - 4452-8877
`;

const LINEA = /^(?<nombre>.+) - (?<email>\S+@\S+) - (?<tel>[\d-]+)$/gm;
const contactos = [...texto.matchAll(LINEA)].map((m) => m.groups);
```

Tres cosas que hacen funcionar ese patrón: la bandera `m` hace que `^` y `$` valgan por línea, el `+` perezoso en el nombre evita que se coma los guiones separadores, y los grupos con nombre te dan un objeto listo para usar.

Resaltar una búsqueda del usuario en una lista de resultados:

```
function resaltar(texto, busqueda) {
  const patron = new RegExp(`(${escapar(busqueda)})`, "gi");
  return texto.replace(patron, "<mark>$1</mark>");
}
```

Esa función combina casi todo el apunte: escape de texto literal, constructor con patrón dinámico, banderas de búsqueda global e insensible a mayúsculas, grupo de captura y referencia en el reemplazo.

Cómo depurar una expresión que no funciona

Cuando un patrón falla, no lo mires fijo. Trabajalo.

Partilo en pedazos y probá cada uno por separado. Si `/^(?<a>\d+)-(?<b>\w+)$/` no coincide, probá primero `/^\d+/,` después `/-/`, después el resto.

Armalo por partes con el constructor, que admite comentarios y saltos de línea en el código fuente:

```
const anio = "\\d{4}";
const mes = "\\d{2}";
const ISO = new RegExp(`^${anio}-${mes}-${mes}$`);
```

Usá grupos con nombre desde el principio. Un `m.groups.mes` que sale `undefined` te dice exactamente qué parte no coincidió; un `m[3]` no te dice nada.

Y desconfiá de tu patrón cuando funciona. Probalo con la entrada vacía, con espacios al principio y al final, con acentos, con un texto diez veces más largo del esperado y con el caso que creés imposible. La mayoría de los patrones que fallan en producción pasaron todas las pruebas del autor.

Ejercicios

1. Escribí tres expresiones regulares que describan el mismo conjunto de cadenas: las que tienen entre 2 y 4 letras `a` seguidas. Explicá cuál preferís y por qué.
2. Explicá sin ejecutar por qué `/^\d+$/ .test("12\n")` devuelve `true` en algunos motores y qué agregarías para evitar la sorpresa. Verificalo después.
3. Escribí una función `contarPalabras(texto)` que funcione bien con texto en castellano, incluidas las palabras con ñe y con acento. No uses `\w`.
4. Dado el texto `"total: $1.200,50 y envío: $300"`, extraé los dos importes como números de JavaScript. Vas a necesitar una mirada, un reemplazo y una conversión.
5. Explicá por qué `/^(\d+)*$/` es peligrosa y escribí una expresión equivalente que no lo sea. Medí las dos con `console.time` sobre una cadena de 30 dígitos terminada en una letra.
6. Escribí `validarCuit(texto)` que acepte las formas `20-12345678-3` y `20123456783`, y que además verifique el dígito verificador. Decidí qué parte resuelve la expresión regular y qué parte resuelve el código.
7. Escribí `plantilla(texto, datos)` que reemplace cada `{{clave}}` del texto por `datos[clave]`, y que deje intacto el marcador si la clave no existe. Usá una función de reemplazo y un grupo con nombre.
8. Tomá el patrón de contactos de la sección de recetas y hacelo tolerante: que acepte teléfono ausente, espacios de más alrededor de los guiones separadores y líneas vacías intercaladas.